CORRECTIONS AFTER THE FIRST BUILD (read this first)

MOCKUPS ARE THE LAW. Build exactly what the mockup pictures and the numbers in this document show: the same spacing, sizes, icons, colours, corners, order of items and texts. **Do not invent anything that is not described here.** Do not add, remove, move, resize, recolour or rename anything because it seems better. Do not use a library default look (Material default colours, shapes, icon sets, fonts, elevations) where this document gives a value. If something is missing or two things seem to disagree, keep the existing behaviour of the app, copy the closest mockup, and ask. Before you call a screen finished, put your screenshot next to the mockup picture and compare spacing, icons and text one by one. Everything in the pictures that is a name, number or list is an example and must come from real data.

These points come from the first screenshots of the build. They override anything shorter or different further down.

1. **Colour.** The accent is blurple `#5865F2`. The build uses bright blue on the selected rail tile, Sign in button, links and tab chip. Use only `CcTheme.colors.accent`. Search the code for any hard-coded blue and remove it.
2. **Panel action buttons.** In the panel title area, New file and New folder (and any refresh or other action) are each a **38 by 38 rounded button, radius 12, grey field colour `#E9EBEF`**, with the icon 18 inside. Bare icons are wrong. Same for the header actions on full screens.
3. **Swipe.** The sidebar opens with a right swipe from **anywhere on the editor**, not only the edge. It works when the swipe is mostly horizontal and the code cannot scroll further left. The left 40 dp edge always works. See Part 1, section 3, and `SidebarHost` in Part 2.
4. **File tree.** Names must always be visible. Indent is 10 dp per level (not 16), no vertical guide lines, folders with a single folder inside are joined (`src/main`, `java/com/app`), and the name gets all the remaining width with an ellipsis.
5. **Buttons and text.** Buttons never wrap and never cut their label with "...". Use short labels ("Sign in", "Clone from link"). Empty-state sentences may wrap, buttons may not.
6. **Run card.** The panel is 234 dp wide. The Run button is a 40 by 40 green icon-only button (play icon). The project name column takes the rest and is cut with an ellipsis. Nothing overlaps. See Run card in Components and `RunCard` in Part 2.
7. **Dim layer and sidebar width.** Behind the sliding editor card use 30% of `#1E2332`, not a dark grey. The card must still look light. **The sidebar is 300 dp wide (rail 66 + panel 234), so the editor card slides 300 dp to the right and only a 60 dp strip of it stays visible.** In the first build the card stopped at 250 dp, which left the panel too narrow (buttons cut off, names squeezed). **The middle layer (the panel) must be a rounded card: both its top-left and its top-right corners are rounded, radius 26, exactly as in the mockups. In the first build the top-right corner of the panel was square. That is wrong. The editor card keeps its rounded left corners.** The panel's own top-left corner is also rounded, radius 26, exactly as in the mockups.
8. **Existing chips (for example the OPEN list).** Keep them, but they scroll sideways and cut long names with an ellipsis, never show half a word.
9. **Tool highlight.** In the rail, the highlighted tool must always match the panel that is showing. If a tool opens a full screen instead of a panel, do not move the highlight.

WHAT THE WORDS IN THE PROJECT LISTS MEAN

This explains the small texts under each project name (Welcome "Recent projects", the project switcher, and the Run card). All of it is read from real data. The examples in the pictures ("Build ok", "2 problems") are only examples of the kinds of text.

The project row

```
[B]  Blank app                        >
     Android · main · Build ok
```

Part	Meaning	Where it comes from
Coloured tile with a letter	The project's colour and the first letter of its name	Project record (`colorHex`, `name.first()`)
Small red or orange number on the tile	How many **problems** (errors and warnings) the project has right now. Red when there is at least one error, orange when only warnings. Hidden at 0	Real diagnostics of that project
Name	The project name	Project record
First word of the sub line (`Android`, `Website`)	The **project type**	Project config / template used
`main`	The **current git branch** of the project. Not shown when the project is not a git repository	Real git
`Build ok`	The **result of the last build**: it finished without errors	Last real build result
`Build failed`	The last build ended with an error (shown in red)	Last real build result
`Not built yet`	There is no build result yet (shown in muted grey)	No build record
`2 problems`	The project has 2 errors or warnings (shown in orange or red)	Real diagnostics
`2 days ago`, `4 days ago`	When the project was **last opened**	Project record, formatted by `relativeTime()`
`7 failed tests` (example)	Only if the app has a test runner and the last test run failed	Real last test result. Do not show it if the app has no tests
`>` chevron	Tap to open the project	
`6 projects` (top right of the list header)	How many projects exist	Real count

The one sub line rule

Build the sub line from pieces and join them with " · ". Show only the pieces that exist. Show each piece once. Never repeat the project name inside it.

Order: type, branch, status.

Status is the most important single thing about the project, chosen in this order:

1. `Build failed` if the last build failed (red).
2. `N problems` if there are problems (orange, or red if any is an error).
3. `Build ok` if the last build succeeded (green).
4. Otherwise the relative last-opened time (`2 days ago`, muted).

```
data class ProjectStatus(val text: String, val kind: Kind) { enum class Kind { Ok, Warning, Error, Muted } }

fun projectStatus(p: Project, lastBuild: BuildResult?, problems: Problems): ProjectStatus = when {
    lastBuild is BuildResult.Failed -> ProjectStatus("Build failed", ProjectStatus.Kind.Error)
    problems.total > 0 -> ProjectStatus("${problems.total} ${if (problems.total == 1) "problem" else "problems"}",
        if (problems.errors > 0) ProjectStatus.Kind.Error else ProjectStatus.Kind.Warning)
    lastBuild is BuildResult.Success -> ProjectStatus("Build ok", ProjectStatus.Kind.Ok)
    else -> ProjectStatus(relativeTime(p.lastOpened), ProjectStatus.Kind.Muted)
}

fun projectSubLine(p: Project, branch: String?, s: ProjectStatus): List<Pair<String, ProjectStatus.Kind>> =
    buildList {
        add(p.typeLabel to ProjectStatus.Kind.Muted)                // "Android" or "Website"
        if (branch != null) add(branch to ProjectStatus.Kind.Muted)  // "main"
        add(s.text to s.kind)
    }

@Composable
fun ProjectSubLine(parts: List<Pair<String, ProjectStatus.Kind>>) {
    val c = CcTheme.colors
    val text = buildAnnotatedString {
        parts.forEachIndexed { i, (t, kind) ->
            if (i > 0) append(" · ")
            withStyle(SpanStyle(color = when (kind) {
                ProjectStatus.Kind.Ok -> c.green; ProjectStatus.Kind.Warning -> c.amber
                ProjectStatus.Kind.Error -> c.red; ProjectStatus.Kind.Muted -> c.muted })) { append(t) }
        }
    }
    Text(text, style = CcType.caption.copy(fontSize = 12.5.sp), maxLines = 1, overflow = TextOverflow.Ellipsis)
}
```

The Run card text

In the panel's Run card the second line is `branch · state`. `main · ok` means: the branch is `main` and the last build is ok. Use the same words as above: `main · ok`, `main · failed`, `main · not built`, `main · building 42%` (while a build runs). The state word is green for ok, red for failed, muted otherwise. If the project has no git repository, show only the state.

Mistakes seen in the first build

- The same value appeared twice in the sub line (for example the name again). Show each piece once.
- Texts that are not real: `7 failed tests` or `2 problems` typed in. They must be computed.
- Text wrapping to a second line. The sub line is always one line with an ellipsis at the end.

DARK THEME COLOURS

The app can also show the dark theme (pictures at the end). Use the same tokens with these dark values. Switch with the Appearance setting (Light, Dark, Follow system). Nothing else changes: layout, sizes, shapes and icons are the same.

Token	Dark hex	Light hex
rail	#1E1F22	#E3E5EA
panel	#2B2D31	#F1F2F5
content	#313338	#FAFAFB
field	#383A40	#E9EBEF
selectedRow	#404249	#DCDFE5
border	#3F4147	#D5D8DE
divider	#26272B	#E3E5EA
toggleOff	#4A4D55	#B8BCC5
text	#F2F3F5	#1F2024
text2	#DBDEE1	#2E3036
textSoft	#B5BAC1	#4F535C
muted	#949BA4	#626772
faint	#6D6F78	#858A95
accent	#5865F2	#5865F2
green	#23A559	#248046
amber	#F0B232	#C98A0A
red	#F23F42	#DA373C
dim (scrim)	rgba(0,0,0,0.50)	rgba(30,35,50,0.30)

```
val DarkCcColors = LightCcColors.copy(
    rail = Color(0xFF1E1F22), panel = Color(0xFF2B2D31), content = Color(0xFF313338), field = Color(0xFF383A40),
    selectedRow = Color(0xFF404249), selectedFile = Color(0xFF404249), border = Color(0xFF3F4147), divider = Color(0xFF2627
2B),
    toggleOff = Color(0xFF4A4D55), text = Color(0xFFF2F3F5), text2 = Color(0xFFDBDEE1), textSoft = Color(0xFFB5BAC1),
    muted = Color(0xFF949BA4), faint = Color(0xFF6D6F78), green = Color(0xFF23A559), amber = Color(0xFFF0B232),
    red = Color(0xFFF23F42), dim = Color(0x80000000))

@Composable
fun CcTheme(mode: ThemeMode, content: @Composable () -> Unit) {
    val dark = when (mode) { ThemeMode.Dark -> true; ThemeMode.Light -> false; ThemeMode.System -> isSystemInDarkTheme() }
    CompositionLocalProvider(LocalCcColors provides if (dark) DarkCcColors else LightCcColors) { MaterialTheme(content = co
ntent) }
}
```

Syntax colours in the dark theme use brighter versions: keyword #C792EA, type #4EC9B0, string #CE9178, function #DCDCAA, number #B5CEA8, comment #6A9955, variable #9CDCFE, plain #DBDEE1.

PART 1. HOW THE APP WORKS (read before anything else)

MOCKUPS ARE THE LAW. Build exactly what the mockup pictures and the numbers in this document show: the same spacing, sizes, icons, colours, corners, order of items and texts. **Do not invent anything that is not described here.** Do not add, remove, move, resize, recolour or rename anything because it seems better. Do not use a library default look (Material default colours, shapes, icon sets, fonts, elevations) where this document gives a value. If something is missing or two things seem to disagree, keep the existing behaviour of the app, copy the closest mockup, and ask. Before you call a screen finished, put your screenshot next to the mockup picture and compare spacing, icons and text one by one. Everything in the pictures that is a name, number or list is an example and must come from real data.

This part explains the structure and the behaviour in plain words. Part 2 gives the Kotlin Jetpack Compose code for every piece. Part 3 is the visual spec (numbers, colours, components, screens). When the words and the code seem to disagree, the code in Part 2 is the reference.

1. The four layers (what is "full screen" and what is "half screen")

Your developer is confused because the app has four different kinds of screen. Each has its own rules.

Layer	What it is	Covers	How it opens	How it closes
1. Editor card	The code editor. This is the home screen	The whole phone	It is always there at the bottom	Never closes. Back at the editor asks to exit
2. Sidebar (the "two panels")	A thin tool rail plus a side panel, **behind** the editor card	The left 300 dp. The editor card slides right and stays visible on the right edge	Swipe right anywhere on the editor (no button)	Tap the editor card, swipe left, or Back
3. Full screens	Clone, Source control, History, Diff, Terminal session, Run, Preview, Settings and its pages, Project settings, Welcome, Start project	The whole phone, including the sidebar and rail	Tap an action in a panel or a row in Settings	Back button in the header, or system Back. One step back
4. Sheets, menus, dialogs	Project switcher, file menu, new command, confirm dialogs	Bottom part of the screen or a floating box over whatever is below	Tap on a button or long-press	Back, tap outside, swipe down

The two panels are NOT a minimised app. They are one thing: the sidebar. It is made of two parts side by side:

- **Rail:** 66 dp wide. Only icons. The tool list (Files, Search, GitHub, Terminal, Logcat, Problems) and Settings at the bottom. Project avatar on top.
- **Panel:** the rest of the 300 dp (234 dp). **The panel is a rounded card: its top-left AND top-right corners are rounded, radius 26** (the right corner is where the editor card starts). It shows the content of the selected tool: file tree for Files, results for Search, and so on.

The editor card is on top of the sidebar. When the user swipes, the card moves to the right and the sidebar is revealed under it. The card is still showing (a strip of the card on the right edge, with rounded left corners, radius 26), slightly smaller and rounded, and it is dimmed. Tapping it closes the sidebar. The sidebar is not a separate screen and it does not have its own header.

2. The state machine (the logic of the whole app)

Keep ONE source of truth for navigation. Do not let each screen decide for itself.

```
AppState
  project: Project?                // the open project (real, from the database)
  sidebarOpen: Boolean             // layer 2
  sidebarTool: Tool                // Files, Search, GitHub, Terminal, Logcat, Problems
  stack: List<Route>               // layer 3, empty when we are on the editor
  sheet: Sheet?                    // layer 4 (only one at a time)
  openFiles: List<OpenFile>        // editor tabs
  activeFile: OpenFile?
```

Rules, in the order the system Back button must handle them (the first one that applies wins):

1. If a sheet, menu or dialog is open: close it. Stop.
2. If the stack has a full screen: pop one route. Stop.
3. If the sidebar is open: close it. Stop.
4. If we are on the editor and the last Back was less than 2 seconds ago: finish the app.
5. Otherwise show the toast "Press back again to exit" and remember the time.

Every full screen header has a back button that calls exactly the same function as the system Back (`onBack()`), so they can never disagree.

What opens what

User action	State change
Swipe right on the editor (anywhere)	`sidebarOpen = true` (follows the finger first, then settles)
Tap a rail tool (Files, Search, GitHub, Terminal, Logcat, Problems)	`sidebarTool = that tool`. Sidebar stays open
Tap Settings in the rail	`push(Route.Settings)` (full screen). The sidebar stays open underneath
Tap a file in the Files panel	`openFile(file)`, `sidebarOpen = false`
Tap an action in a panel (Clone, Source control, History)	`push(Route.X)` (full screen)
Tap the project avatar	`sheet = Sheet.ProjectSwitcher`
Long-press a file	`sheet = Sheet.FileMenu(file)` (popup menu)
Tap Run in the Run card	`push(Route.Run)` and start the real build
Tap Back button in a header	`onBack()`

Where each tool shows its detail

Tool in the rail	Panel (half screen, in the sidebar)	Full screens it can open
Files	File tree, filter, new file, new folder, Run card	Editor (closes sidebar)
Search	Query, options, results, Run card	Editor at the line
GitHub	Account and repository actions	Clone a repository, Source control, History, Commit, Diff, GitHub sign in

Tool in the rail	Panel (half screen, in the sidebar)	Full screens it can open
Terminal	List of sessions, new session, Run card	Terminal session, New command (sheet)
Logcat	Live log with filter chips, Run card	none
Problems	Errors and warnings list, Run card	Editor at the line
Settings (rail bottom)	none	Settings (full screen) then each settings page

3. The swipe (exact behaviour)

- **The user can start the swipe anywhere on the editor, not only at the edge.** A drag counts as "open the sidebar" when it moves to the right, is mostly horizontal (sideways movement more than twice the up or down movement, after a 10 dp touch slop), AND the editor content under the finger cannot scroll further left (the code is not scrolled sideways, or the finger is on the tab row, the header or an empty area).
- If the code view IS scrolled sideways, a right swipe first scrolls the code back. A swipe that starts in the left 40 dp edge always opens the sidebar, even then.
- Vertical drags and upward or downward scrolling are never taken by the sidebar.
- A drag to the left while the sidebar is closed does nothing here (the editor keeps it).
- The editor card follows the finger. Progress goes 0.0 to 1.0 over 300 dp.
- While dragging: `translationX = 300 dp * progress`, the card's left corners get radius `26 dp * progress`, scrim (dark layer) `0.30 * progress`.
- On release: open if progress > 0.4 or the fling velocity to the right is above 800 dp/s. Close if progress < 0.4 or the fling goes left.
- Settle with `spring(dampingRatio = 0.85f, stiffness = 400f)`.
- While open, the card ignores taps on its content. A tap anywhere on the card closes the sidebar.
- The sidebar is NOT opened by a button. No hamburger icon.
- When the keyboard is up, the swipe is disabled.

4. Why each full screen exists (purpose, behaviour, back)

Screen	Why it is there	Behaviour	Back goes to
Welcome	First screen when there is no open project	Three buttons. Recent real projects below	Exit app
Start a project	Create a new project from a real template	Fill name and location, Create. Shows progress, then opens the editor	Welcome or editor
Clone a repository	Download a git repository	URL, optional branch, destination. Clone starts the progress screen	GitHub panel
Cloning progress	Shows the real progress of the clone	Cancel stops it and deletes the half-cloned folder	Clone screen
First-time setup	Downloads the tools the build needs	Real download progress per card. Finish enabled when all required cards are done	Welcome or editor
Source control	Stage, commit, pull and push	Real git status. Commit needs a message	GitHub panel

Screen	Why it is there	Behaviour	Back goes to
History	The commit list	Tap a commit to open it	Source control
Commit detail	Files that changed in one commit	Tap a file for the diff	History
Diff	Lines changed in one file	Read only. Colours from diff tokens	Commit detail or Source control
Terminal session	A real shell	Typing sends to the shell. Key row sends Tab, Ctrl, Esc, arrows	Terminal panel
Run and build	Shows the build steps	Real steps and percent. Cancel stops the build	Editor
Build finished	Result of the build	Ready: Open app, Share. Failed: real error text and a Problems link	Editor
Website preview	Shows the local web page	Address pill, reload, Console button	Run
Console	The page's console	Live lines	Preview
Compose previews	Shows the preview functions	List of real previews. Empty state when none	Run
Settings and pages	The app settings	Each toggle saves at once	Settings, then editor
Project settings	Name, colour, package, SDK	Save on change	Where it was opened from

5. Animations (what moves, how long, which curve)

What	Animation	Duration or spec
Sidebar open and close	Spring on the editor card offset	damping 0.85, stiffness 400
Tool switch in the panel	Cross-fade the panel content	120 ms
Rail selected pill	Height 8 to 30 and tile colour change	160 ms
Full screen push	Slide in from the right (full width) with the old screen moving 25% left	220 ms, FastOutSlowIn
Full screen pop	The reverse	200 ms
Bottom sheet	Slide up from the bottom, scrim fades to 0.30	250 ms ease-out
Popup menu	Fade and scale from 0.95, origin at the touch point	120 ms
Toggle	Knob slides, track colour fades	160 ms
Progress bar	Animate to each new value	200 ms linear
Toast	Fade in 150 ms, wait 2 s, fade out 200 ms	
Button press	Alpha 0.9	instant

Always check `LocalReduceMotion` / `Settings.Global.ANIMATOR_DURATION_SCALE`. If the scale is 0, use `snap()` instead of springs.

6. Data rules (real data, repeated because it matters)

- Every list is a `StateFlow` or `Flow` from the real database or the real file system. Screens only display. They never contain sample items.
- A project's colour and letter: stored in the project table (`colorHex`, derived from the name if not set). Letter is `name.first().uppercase()`.
- Times are formatted from real timestamps by one function `relativeTime(millis)`.
- If a list is empty, call `EmptyState(...)` (Part 2). If data is loading, call `LoadingList()`. If it failed, call `ErrorCard(message, onRetry)`.

7. What your developer should NOT do

- Do not add a menu button to open the sidebar.
- Do not show the rail on full screens.
- Do not keep a permanent text-labelled rail.
- Do not invent projects, files, commits or users.
- Do not change any logic. Only change how things look and how screens connect.

PART 2. THE CODE (Kotlin, Jetpack Compose)

Package names and file names are suggestions. Put each block in its own file. Every block compiles against Compose BOM 2024.09 or newer with Material 3. Nothing here contains sample data. Every screen takes its data as parameters from the real ViewModel.

2.1 How icons are built (the letters and numbers)

The letters and numbers your developer sees (for example `M15 18l-6-6 6-6`) are **SVG path data**. It is a tiny drawing language. Letters are commands, numbers are coordinates on a 24 by 24 grid:

Letter	Meaning	Example
M x y	Move the pen to the point (no line)	`M15 18`
L x y	Draw a line to the point	`L9 12`
l dx dy	Draw a line, relative to the current point	`l-6-6` means 6 left, 6 up
H x, h dx	Horizontal line	`H21`
V y, v dy	Vertical line	`v4`
A rx ry rot large sweep x y	Arc (curve of a circle)	`a2 2 0 0 1 2 2`
C, c	Curve	
Z	Close the shape	

So `M15 18l-6-6 6-6` means: go to (15,18), draw to the left-up, then draw right-up. That is the back chevron "<". The Kotlin code can read this text directly. You do not draw the icons by hand and you do not need a drawable file for each icon. Keep the path text in one list and build the vectors from it.

All icons: viewport 24 by 24, stroke only, stroke width 1.8 (2.2 chevrons, 2.4 ticks), round cap, round join, no fill (Play is filled).

File `ui/icons/CcIcons.kt`

```
package com.closecode.ui.icons

import androidx.compose.foundation.layout.size
import androidx.compose.material3.Icon
import androidx.compose.runtime.Composable
import androidx.compose.runtime.remember
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap
import androidx.compose.ui.graphics.StrokeJoin
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.PathParser
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.Dp
import androidx.compose.ui.unit.dp

/** Path data for every icon in the design. 24 x 24 grid. */
object CcIconPaths {
    val all: Map<String, String> = mapOf(
    "folder" to "M4 20h16a2 2 0 0 0 2-2V8a2 2 0 0 0-2-2h-7.9a2 2 0 0 1-1.69-.9L9.6 3.9A2 2 0 0 0 7.93 3H4a2 2 0 0 0-2 2v13c
0 1.1.9 2 2 2Z",
    "refresh" to "M3 12a9 9 0 0 1 9-9 9.75 9.75 0 0 1 6.74 2.74L21 8M21 3v5h-5M21 12a9 9 0 0 1-9 9 9.75 9.75 0 0 1-6.74-2.7
4L3 16M8 16H3v5",
    "branch" to "M6 3v12M18 9a3 3 0 1 0 0-6 3 3 0 0 0 0 6zM6 21a3 3 0 1 0 0-6 3 3 0 0 0 0 6zM18 9a9 9 0 0 1-9 9",
    "history" to "M3 12a9 9 0 1 0 3-6.7L3 8M3 3v5h5M12 7v5l4 2",
    "down" to "M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4M7 10l5 5 5-5M12 15V3",
    "up" to "M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4M17 8l-5-5-5 5M12 3v12",
    "more" to "M12 5v.01M12 12v.01M12 19v.01",
    "plus" to "M12 5v14M5 12h14",
    "check" to "M20 6L9 17l-5-5",
    "cd" to "M6 9l6 6 6 6-6",
    "cl" to "M15 18l-6-6 6-6",
    "cr" to "M9 18l6-6-6-6",
    "search" to "M11 19a8 8 0 1 0 0-16 8 8 0 0 0 0 16zM21 21l-4.3-4.3",
    "filter" to "M3 6h18M7 12h10M10 18h4",
    "copy" to "M9 9h11v11H9zM5 15H4V4h11v1",
    "file" to "M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8zM14 2v6h6",
    "undo" to "M9 14L4 9l5-5M4 9h10a6 6 0 0 1 0 12h-3",
    "redo" to "M15 14l5-5-5-5M20 9H10a6 6 0 0 0 0 12h3",
    "save" to "M19 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h11l5 5v11a2 2 0 0 1-2 2zM17 21v-8H7v8M7 3v5h8",
    "term" to "M4 17l6-6-6-6M12 19h8",
    "menu" to "M4 6h16M4 12h16M4 18h16",
    "x" to "M18 6L6 18M6 6l12 12",
    "bolt" to "M13 2L3 14h9l-1 8 10-12h-9z",
    "kbd" to "M4 5h16a2 2 0 0 1 2 2v10a2 2 0 0 1-2 2H4a2 2 0 0 1-2-2V7a2 2 0 0 1 2-2zM6 9h.01M10 9h.01M14 9h.01M18 9h.01M7
15h10",
    "code" to "M16 18l6-6-6-6M8 6l-6 6 6 6",
    "layers" to "M12 2l10 5-10 5L2 7zM2 17l10 5 10-5M2 12l10 5 10-5",
    "image" to "M5 3h14a2 2 0 0 1 2 2v14a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2zM9 8a1 1 0 1 0 0 .01M21 15l-5-5L5 2
1",
    "trash" to "M3 6h18M8 6V4h8v2M19 6l-1 14H6L5 6",
    "share" to "M4 12v8h16v-8M16 6l-4-4-4 4M12 2v13",
    "bug" to "M8 2l1.9 1.9M14.1 3.9L16 2M9 7.1V6a3 3 0 0 1 6 0v1.1M12 20a6 6 0 0 1-6-6v-3a4 4 0 0 1 4-4h4a4 4 0 0 1 4 4v3a6
6 0 0 1-6 6zM12 20v-9M6.5 13H2M17.5 13H22",
    "flask" to "M10 2v7.5L4 20a1 1 0 0 0 .9 1.5h14.2A1 1 0 0 0 20 20l-6-10.5V2M8.5 2h7",
    "alert" to "M12 9v4M12 17h.01M10.3 3.9L1.8 18a2 2 0 0 0 1.7 3h17a2 2 0 0 0 1.7-3L13.7 3.9a2 2 0 0 0-3.4 0z",
    "gh" to "M15 22v-4a4.8 4.8 0 0 0-1-3.5c3 0 6-2 6-5.5.08-1.25-.27-2.48-1-3.5.28-1.15.28-2.35 0-3.5 0 0-1 0-3 1.5-2.64-.5
-5.36-.5-8 0C6 2 5 2 5 2c-.3 1.15-.3 2.35 0 3.5A5.403 5.403 0 0 0 4 9c0 3.5 3 5.5 6 5.5-.39.49-.68 1.05-.85 1.65-.17.6-.22
1.23-.15 1.85v4M9 18c-4.51 2-5-2-7-2",
    "send" to "M22 2L11 13M22 2l-7 20-4-9-9-4z",
    "cog" to "M12 15a3 3 0 1 0 0-6 3 3 0 0 0 0 6zM19.4 15a1.65 1.65 0 0 0 .33 1.82l.06.06a2 2 0 1 1-2.83 2.83l-.06-.06a1.65
1.65 0 0 0-1.82-.33 1.65 1.65 0 0 0-1 1.51V21a2 2 0 1 1-4 0v-.09A1.65 1.65 0 0 0 9 19.4a1.65 1.65 0 0 0-1.82.33l-.06.06a2 2
0 1 1-2.83-2.83l.06-.06A1.65 1.65 0 0 0 4.68 15a1.65 1.65 0 0 0-1.51-1H3a2 2 0 1 1 0-4h.09A1.65 1.65 0 0 0 4.6 9a1.65 1.65
0 0 0-.33-1.82l-.06-.06a2 2 0 1 1 2.83-2.83l.06.06A1.65 1.65 0 0 0 9 4.68a1.65 1.65 0 0 0 1-1.51V3a2 2 0 1 1 4 0v.09a1.65
1.65 0 0 0 1 1.51 1.65 1.65 0 0 0 1.82-.33l.06-.06a2 2 0 1 1 2.83 2.83l-.06.06A1.65 1.65 0 0 0 19.4 9a1.65 1.65 0 0 0 1.51
1H21a2 2 0 1 1 0 4h-.09a1.65 1.65 0 0 0-1.51 1z",
    "ext" to "M18 13v6a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V8a2 2 0 0 1 2-2h6M15 3h6v6M10 14L21 3",
```

```
    "play" to "M7 4.5v15l13-7.5z",
    "user" to "M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2M12 11a4 4 0 1 0 0-8 4 4 0 0 0 0 8z",
    "panel" to "M5 3h14a2 2 0 0 1 2 2v14a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2zM9 3v18",
    "globe" to "M12 22a10 10 0 1 0 0-20 10 10 0 0 0 0 20zM2 12h20M12 2a15 15 0 0 1 0 20 15 15 0 0 1 0-20z",
    "flag" to "M4 22V4M4 4h13l-2 4 2 4H4",
    "moon" to "M21 12.8A9 9 0 1 1 11.2 3a7 7 0 0 0 9.8 9.8z",
    "sun2" to "M12 17a5 5 0 1 0 0-10 5 5 0 0 0 0 10zM12 1v2M12 21v2M4.2 4.2l1.4 1.4M18.4 18.4l1.4 1.4M1 12h2M21 12h2M4.2 19.8l1.4-1.4M18.4 5.6l1.4-1.4",
    "lock" to "M5 11h14v10H5zM8 11V7a4 4 0 0 1 8 0v4",
    "info" to "M12 22a10 10 0 1 0 0-20 10 10 0 0 0 0 20zM12 16v-4M12 8h.01",
    "db" to "M12 8c4.4 0 8-1.3 8-3s-3.6-3-8-3-8 1.3-8 3 3.6 3 8 3zM4 5v14c0 1.7 3.6 3 8 3s8-1.3 8-3V5M4 12c0 1.7 3.6 3 8 3s8-1.3 8-3",
    )
}

private val cache = HashMap<String, ImageVector>()

/** Builds (and caches) a stroked ImageVector from the path text. */
fun ccIconVector(name: String, strokeWidth: Float = 1.8f, filled: Boolean = false): ImageVector {
    val key = "$name|$strokeWidth|$filled"
    return cache.getOrPut(key) {
        val d = requireNotNull(CcIconPaths.all[name]) { "Unknown icon: $name" }
        ImageVector.Builder(
            name = name,
            defaultWidth = 24.dp, defaultHeight = 24.dp,
            viewportWidth = 24f, viewportHeight = 24f
        ).addPath(
            pathData = PathParser().parsePathString(d).toNodes(),
            fill = if (filled) SolidColor(Color.Black) else null,
            stroke = if (filled) null else SolidColor(Color.Black),
            strokeLineWidth = strokeWidth,
            strokeLineCap = StrokeCap.Round,
            strokeLineJoin = StrokeJoin.Round
        ).build()
    }
}

/** Use this everywhere. `tint` is a theme colour token. */
@Composable
fun CcIcon(
    name: String,
    tint: Color,
    size: Dp = 20.dp,
    strokeWidth: Float = 1.8f,
    modifier: Modifier = Modifier
) {
    val vector = remember(name, strokeWidth) {
        ccIconVector(name, strokeWidth, filled = name == "play")
    }
    Icon(imageVector = vector, contentDescription = null, tint = tint, modifier = modifier.size(size))
}
```

Usage: `CcIcon("back", CcTheme.colors.text2, 20.dp, 2.2f)`. The tint replaces the black placeholder. Content descriptions are set by the button that contains the icon.

2.2 Theme tokens

File `ui/theme/CcTheme.kt`

```
package com.closecode.ui.theme

import androidx.compose.runtime.Composable
import androidx.compose.runtime.CompositionLocalProvider
import androidx.compose.runtime.staticCompositionLocalOf
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.Font
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.lightColorScheme
```

```
import com.closecode.R

data class CcColors(
    val rail: Color, val panel: Color, val content: Color, val field: Color,
    val selectedRow: Color, val selectedFile: Color, val border: Color, val divider: Color,
    val toggleOff: Color, val text: Color, val text2: Color, val textSoft: Color,
    val muted: Color, val faint: Color, val accent: Color, val green: Color,
    val amber: Color, val red: Color, val blue: Color, val purple: Color,
    val yellow: Color, val teal: Color, val orange: Color, val pink: Color,
    val diffAddBg: Color, val diffAddText: Color, val diffDelBg: Color, val diffDelText: Color,
    val matchBg: Color, val matchText: Color, val dim: Color
)

val LightCcColors = CcColors(
    rail = Color(0xFFE3E5EA), panel = Color(0xFFF1F2F5), content = Color(0xFFFAFAFB),
    field = Color(0xFFE9EBEF), selectedRow = Color(0xFFDCDFE5), selectedFile = Color(0xFFDDE0FB),
    border = Color(0xFFD5D8DE), divider = Color(0xFFE3E5EA), toggleOff = Color(0xFFB8BCC5),
    text = Color(0xFF1F2024), text2 = Color(0xFF2E3036), textSoft = Color(0xFF4F535C),
    muted = Color(0xFF626772), faint = Color(0xFF858A95), accent = Color(0xFF5865F2),
    green = Color(0xFF248046), amber = Color(0xFFC98A0A), red = Color(0xFFDA373C),
    blue = Color(0xFF2F6FE0), purple = Color(0xFF7B5CE0), yellow = Color(0xFFD99A06),
    teal = Color(0xFF1F9D6B), orange = Color(0xFFD97706), pink = Color(0xFFDB4C9B),
    diffAddBg = Color(0xFFDDF3E4), diffAddText = Color(0xFF1A7F37),
    diffDelBg = Color(0xFFFBE4E5), diffDelText = Color(0xFFDA373C),
    matchBg = Color(0xFFFBE7A1), matchText = Color(0xFF6B4E00),
    dim = Color(0x4D1E2332)            // 30% of 1E2332
)

val Inter = FontFamily(
    Font(R.font.inter_regular, FontWeight.Normal),
    Font(R.font.inter_medium, FontWeight.Medium),
    Font(R.font.inter_semibold, FontWeight.SemiBold),
    Font(R.font.inter_bold, FontWeight.Bold),
    Font(R.font.inter_extrabold, FontWeight.ExtraBold)
)
val Mono = FontFamily(
    Font(R.font.jetbrains_mono_regular, FontWeight.Normal),
    Font(R.font.jetbrains_mono_semibold, FontWeight.SemiBold),
    Font(R.font.jetbrains_mono_bold, FontWeight.Bold)
)

object CcType {
    val hero = TextStyle(fontFamily = Inter, fontSize = 33.sp, fontWeight = FontWeight.ExtraBold, letterSpacing = (-1).sp)
    val setupTitle = TextStyle(fontFamily = Inter, fontSize = 27.sp, fontWeight = FontWeight.ExtraBold)
    val panelTitle = TextStyle(fontFamily = Inter, fontSize = 21.sp, fontWeight = FontWeight.ExtraBold)
    val sheetTitle = TextStyle(fontFamily = Inter, fontSize = 19.sp, fontWeight = FontWeight.ExtraBold)
    val screenTitle = TextStyle(fontFamily = Inter, fontSize = 16.5.sp, fontWeight = FontWeight.Bold)
    val itemTitle = TextStyle(fontFamily = Inter, fontSize = 14.5.sp, fontWeight = FontWeight.SemiBold)
    val body = TextStyle(fontFamily = Inter, fontSize = 14.sp, fontWeight = FontWeight.Medium)
    val small = TextStyle(fontFamily = Inter, fontSize = 13.sp)
    val caption = TextStyle(fontFamily = Inter, fontSize = 12.sp)
    val section = TextStyle(fontFamily = Inter, fontSize = 11.sp, fontWeight = FontWeight.Bold, letterSpacing = 0.8.sp)
    val code = TextStyle(fontFamily = Mono, fontSize = 11.5.sp, lineHeight = 21.sp)
    val log = TextStyle(fontFamily = Mono, fontSize = 11.sp, lineHeight = 16.sp)
}

object CcShape {
    val tile = RoundedCornerShape(13.dp)
    val tileLarge = RoundedCornerShape(15.dp)
    val button = RoundedCornerShape(15.dp)
    val card = RoundedCornerShape(18.dp)
    val chip = RoundedCornerShape(15.dp)
    val sheetTop = RoundedCornerShape(topStart = 26.dp, topEnd = 26.dp)
    val panelTopLeft = RoundedCornerShape(topStart = 26.dp)
    val menu = RoundedCornerShape(16.dp)
    val dialog = RoundedCornerShape(24.dp)
}

private val LocalCcColors = staticCompositionLocalOf { LightCcColors }

object CcTheme {
    val colors: CcColors @Composable get() = LocalCcColors.current
}

@Composable
fun CcTheme(content: @Composable () -> Unit) {
    CompositionLocalProvider(LocalCcColors provides LightCcColors) {
        MaterialTheme(
            colorScheme = lightColorScheme(
                primary = LightCcColors.accent, background = LightCcColors.content,
                surface = LightCcColors.content, onSurface = LightCcColors.text
            ),
            content = content
        )
```

```
    }
}

/** Tile tint = colour at 13% opacity. */
fun Color.tint() = this.copy(alpha = 0.13f)
```

Add the font files (Inter and JetBrains Mono) to `res/font/` with the names used above. Tell Android Studio to use the app's existing fonts if it already has them.

2.3 Components

File `ui/components/Basics.kt` (buttons, fields, toggle, chips, rows)

```
package com.closecode.ui.components

import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.animateDpAsState
import androidx.compose.animation.core.tween
import androidx.compose.foundation.*
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.BasicTextField
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.alpha
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.Dp
import androidx.compose.ui.unit.dp
import androidx.compose.material3.Text
import com.closecode.ui.icons.CcIcon
import com.closecode.ui.theme.*

/** Text that never wraps in headers, rows and buttons. */
@Composable
fun OneLine(text: String, style: androidx.compose.ui.text.TextStyle, color: Color, modifier: Modifier = Modifier) =
    Text(text, style = style, color = color, maxLines = 1, overflow = TextOverflow.Ellipsis, modifier = modifier)

@Composable
fun PrimaryButton(
    text: String, onClick: () -> Unit, modifier: Modifier = Modifier,
    icon: String? = null, enabled: Boolean = true, container: Color = CcTheme.colors.accent,
    height: Dp = 48.dp, radius: Dp = 15.dp
) {
    val c = CcTheme.colors
    val source = remember { MutableInteractionSource() }
    val pressed by source.collectIsPressedAsState()
    Row(
        modifier = modifier
            .height(height)
            .clip(RoundedCornerShape(radius))
            .background(if (enabled) container else c.field)
            .alpha(if (pressed) 0.9f else 1f)
            .clickable(interactionSource = source, indication = null, enabled = enabled, onClick = onClick)
            .padding(horizontal = 16.dp),
        horizontalArrangement = Arrangement.Center, verticalAlignment = Alignment.CenterVertically
    ) {
        if (icon != null) { CcIcon(icon, if (enabled) Color.White else c.muted, 17.dp, 2f); Spacer(Modifier.width(8.dp)) }
        OneLine(text, CcType.body.copy(fontSize = 15.sp, fontWeight = androidx.compose.ui.text.font.FontWeight.Bold),
            if (enabled) Color.White else c.muted)
    }
}

@Composable
fun SecondaryButton(text: String, onClick: () -> Unit, modifier: Modifier = Modifier,
                    icon: String? = null, destructive: Boolean = false, height: Dp = 48.dp) {
    val c = CcTheme.colors
    val color = if (destructive) c.red else c.textSoft
    Row(
```

```
        modifier.height(height).clip(RoundedCornerShape(15.dp)).background(c.field)
            .clickable(onClick = onClick).padding(horizontal = 16.dp),
        horizontalArrangement = Arrangement.Center, verticalAlignment = Alignment.CenterVertically
    ) {
        if (icon != null) { CcIcon(icon, color, 16.dp, 2f); Spacer(Modifier.width(7.dp)) }
        OneLine(text, CcType.body.copy(fontWeight = androidx.compose.ui.text.font.FontWeight.SemiBold), color)
    }
}

/** 42 x 25 toggle. Off: toggleOff track. On: accent track. Knob moves 160 ms. */
@Composable
fun CcToggle(checked: Boolean, onCheckedChange: (Boolean) -> Unit, modifier: Modifier = Modifier) {
    val c = CcTheme.colors
    val track by animateColorAsState(if (checked) c.accent else c.toggleOff, tween(160), label = "track")
    val knobX by animateDpAsState(if (checked) 20.dp else 3.dp, tween(160), label = "knob")
    Box(
        modifier.size(42.dp, 25.dp).clip(RoundedCornerShape(13.dp)).background(track)
            .clickable(interactionSource = remember { MutableInteractionSource() }, indication = null) {
                onCheckedChange(!checked)
            }
    ) {
        Box(Modifier.offset(x = knobX, y = 3.dp).size(19.dp).clip(CircleShape).background(Color.White))
    }
}

@Composable
fun CcChip(text: String, selected: Boolean, onClick: () -> Unit) {
    val c = CcTheme.colors
    Box(
        Modifier.height(30.dp).clip(CcShape.chip)
            .background(if (selected) c.accent else c.field).clickable(onClick = onClick)
            .padding(horizontal = 13.dp), contentAlignment = Alignment.Center
    ) { OneLine(text, CcType.caption.copy(fontWeight = androidx.compose.ui.text.font.FontWeight.SemiBold),
        if (selected) Color.White else c.textSoft) }
}

/** Labelled text field, 44 dp tall, focus ring 1.5 dp accent, error line in red. */
@Composable
fun CcTextField(
    value: String, onValueChange: (String) -> Unit, modifier: Modifier = Modifier,
    label: String? = null, hint: String = "", mono: Boolean = false, error: String? = null,
    leadingIcon: String? = null, singleLine: Boolean = true
) {
    val c = CcTheme.colors
    var focused by remember { mutableStateOf(false) }
    Column(modifier) {
        if (label != null) {
            Text(label, style = CcType.caption.copy(fontWeight = androidx.compose.ui.text.font.FontWeight.SemiBold), color
= c.textSoft)
            Spacer(Modifier.height(6.dp))
        }
        Row(
            Modifier.fillMaxWidth().height(44.dp).clip(RoundedCornerShape(13.dp)).background(c.field)
                .border(if (focused) 1.5.dp else 0.dp, if (focused) c.accent else Color.Transparent, RoundedCornerShape(13.
dp))
                .padding(horizontal = 14.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            if (leadingIcon != null) { CcIcon(leadingIcon, c.muted, 16.dp); Spacer(Modifier.width(8.dp)) }
            Box(Modifier.weight(1f)) {
                if (value.isEmpty()) Text(hint, style = CcType.small.copy(fontSize = 12.5.sp), color = c.muted)
                BasicTextField(
                    value = value, onValueChange = onValueChange, singleLine = singleLine,
                    textStyle = (if (mono) CcType.code.copy(fontSize = 12.5.sp, lineHeight = 18.sp) else CcType.small.copy
(fontSize = 12.5.sp)).copy(color = c.text),
                    cursorBrush = SolidColor(c.accent),
                    modifier = Modifier.fillMaxWidth().onFocusChanged { focused = it.isFocused }
                )
            }
        }
        if (error != null) { Spacer(Modifier.height(4.dp)); Text(error, style = CcType.caption.copy(fontSize = 11.5.sp), co
lor = c.red) }
    }
}

/** Row used in side panels and plain lists (tile 42, title, sub line, trailing). */
@Composable
fun PanelRow(
    icon: String, tint: Color, title: String, sub: String?, selected: Boolean = false,
    onClick: () -> Unit, trailing: @Composable (() -> Unit)? = null
) {
    val c = CcTheme.colors
    Row(
        Modifier.fillMaxWidth().padding(horizontal = if (selected) 8.dp else 0.dp)
```

```
            .clip(RoundedCornerShape(14.dp)).background(if (selected) c.selectedRow else Color.Transparent)
            .clickable(onClick = onClick).heightIn(min = 58.dp).padding(horizontal = 14.dp, vertical = 8.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Box(Modifier.size(42.dp).clip(CcShape.tile).background(tint.tint()), contentAlignment = Alignment.Center) {
            CcIcon(icon, tint, 21.dp)
        }
        Spacer(Modifier.width(12.dp))
        Column(Modifier.weight(1f)) {
            OneLine(title, CcType.itemTitle, c.text)
            if (sub != null) OneLine(sub, CcType.caption.copy(fontSize = 12.5.sp), c.muted)
        }
        if (trailing != null) trailing() else CcIcon("cr", c.faint, 16.dp, 2.2f)
    }
}

/** Grouped settings card. Each row is SettingRow. */
@Composable
fun SettingsGroup(label: String, content: @Composable ColumnScope.() -> Unit) {
    val c = CcTheme.colors
    Text(label.uppercase(), style = CcType.section, color = c.muted, modifier = Modifier.padding(start = 18.dp, top = 16.d
p, bottom = 6.dp))
    Column(Modifier.padding(horizontal = 12.dp).fillMaxWidth().clip(CcShape.card).background(c.field), content = content)
}

@Composable
fun SettingRow(
    icon: String, tint: Color, title: String, sub: String? = null,
    value: String? = null, onClick: (() -> Unit)? = null,
    trailing: @Composable (() -> Unit)? = null, showDivider: Boolean = true
) {
    val c = CcTheme.colors
    Column {
        Row(
            Modifier.fillMaxWidth().then(if (onClick != null) Modifier.clickable(onClick = onClick) else Modifier)
                .padding(horizontal = 14.dp, vertical = 12.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Box(Modifier.size(36.dp).clip(RoundedCornerShape(12.dp)).background(tint.tint()), contentAlignment = Alignment.
Center) {
                CcIcon(icon, tint, 18.dp)
            }
            Spacer(Modifier.width(12.dp))
            Column(Modifier.weight(1f)) {
                OneLine(title, CcType.itemTitle, c.text)
                if (sub != null) Text(sub, style = CcType.caption.copy(lineHeight = 17.sp), color = c.muted)
            }
            if (value != null) { OneLine(value, CcType.small, c.muted); Spacer(Modifier.width(4.dp)) }
            if (trailing != null) trailing() else if (onClick != null) CcIcon("cr", c.faint, 15.dp, 2.2f)
        }
        if (showDivider) Box(Modifier.padding(start = 62.dp).fillMaxWidth().height(1.dp).background(c.divider))
    }
}
```

(Add `import androidx.compose.ui.focus.onFocusChanged` and `import androidx.compose.ui.unit.sp`.)

File `ui/components/States.kt` (empty, loading, error)

```
@Composable
fun EmptyState(icon: String, title: String, message: String,
               actionText: String? = null, onAction: (() -> Unit)? = null, modifier: Modifier = Modifier) {
    val c = CcTheme.colors
    Column(modifier.fillMaxWidth().padding(32.dp), horizontalAlignment = Alignment.CenterHorizontally) {
        Box(Modifier.size(64.dp).clip(RoundedCornerShape(20.dp)).background(c.field), contentAlignment = Alignment.Center)
{
            CcIcon(icon, c.muted, 28.dp, 1.6f)
        }
        Spacer(Modifier.height(14.dp))
        Text(title, style = CcType.itemTitle.copy(fontSize = 16.sp, fontWeight = FontWeight.Bold), color = c.text)
        Spacer(Modifier.height(4.dp))
        Text(message, style = CcType.small, color = c.muted, textAlign = TextAlign.Center)
        if (actionText != null && onAction != null) { Spacer(Modifier.height(16.dp)); PrimaryButton(actionText, onAction) }
    }
}

@Composable
fun LoadingList(rows: Int = 4) {
    val c = CcTheme.colors
    Column(Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(10.dp)) {
```

```
        repeat(rows) { Box(Modifier.fillMaxWidth().height(42.dp).clip(RoundedCornerShape(13.dp)).background(c.field)) }
    }
}

@Composable
fun ErrorCard(message: String, onRetry: (() -> Unit)?) {
    val c = CcTheme.colors
    Column(Modifier.padding(14.dp).fillMaxWidth().clip(RoundedCornerShape(14.dp)).background(c.diffDelBg).padding(12.dp)) {
        Text(message, style = CcType.small, color = c.red)   // the REAL error text from the system
        if (onRetry != null) { Spacer(Modifier.height(10.dp)); SecondaryButton("Retry", onRetry, icon = "refresh", height =
40.dp) }
    }
}

@Composable
fun ProgressLine(label: String, percent: Float, right: String? = null) {
    val c = CcTheme.colors
    val p by animateFloatAsState(percent.coerceIn(0f, 1f), tween(200, easing = LinearEasing), label = "p")
    Column {
        Row { Text(label, style = CcType.caption.copy(fontWeight = FontWeight.Bold), color = c.accent, modifier = Modifier.
weight(1f))
              if (right != null) Text(right, style = CcType.caption, color = c.muted) }
        Spacer(Modifier.height(6.dp))
        Box(Modifier.fillMaxWidth().height(6.dp).clip(RoundedCornerShape(4.dp)).background(c.rail)) {
            Box(Modifier.fillMaxHeight().fillMaxWidth(p).background(c.accent))
        }
    }
}
```

File `ui/components/Header.kt` (header with back button and origin)

```
/** Every full screen uses this. `origin` is the screen the user came from (bold), `title` is this screen (muted). */
@Composable
fun ScreenHeader(
    origin: String?, title: String, onBack: () -> Unit,
    actions: @Composable RowScope.() -> Unit = {}
) {
    val c = CcTheme.colors
    Column(Modifier.background(c.content)) {
        Row(Modifier.fillMaxWidth().height(56.dp).padding(start = 12.dp, end = 14.dp), verticalAlignment = Alignment.Center
Vertically) {
            Box(
                Modifier.size(36.dp).clip(RoundedCornerShape(12.dp)).background(c.field)
                    .clickable(onClick = onBack).semantics { contentDescription = "Back" },
                contentAlignment = Alignment.Center
            ) { CcIcon("back", c.text2, 20.dp, 2.2f) }
            Spacer(Modifier.width(10.dp))
            Row(Modifier.weight(1f), verticalAlignment = Alignment.CenterVertically) {
                if (origin != null) {
                    OneLine(origin, CcType.screenTitle, c.text, Modifier.weight(1f, fill = false))
                    Spacer(Modifier.width(7.dp))
                    OneLine(title, CcType.caption.copy(fontWeight = FontWeight.Medium), c.muted)
                } else OneLine(title, CcType.screenTitle, c.text)
            }
            Row(horizontalArrangement = Arrangement.spacedBy(10.dp), verticalAlignment = Alignment.CenterVertically, conten
t = actions)
        }
        Box(Modifier.fillMaxWidth().height(1.dp).background(c.divider))
    }
}

@Composable
fun HeaderAction(icon: String, description: String, enabled: Boolean = true, onClick: () -> Unit) {
    val c = CcTheme.colors
    Box(Modifier.size(38.dp).clip(RoundedCornerShape(12.dp)).background(c.field)
        .clickable(enabled = enabled, onClick = onClick).semantics { contentDescription = description },
        contentAlignment = Alignment.Center) { CcIcon(icon, if (enabled) c.text2 else c.faint, 18.dp, 1.9f) }
}
```

File `ui/components/Sheets.kt` (bottom sheet, popup menu, dialog, toast)

```
/** Use Material3's ModalBottomSheet with our colours and handle. */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CcBottomSheet(title: String, onDismiss: () -> Unit, content: @Composable ColumnScope.() -> Unit) {
    val c = CcTheme.colors
```

```
    ModalBottomSheet(
        onDismissRequest = onDismiss, containerColor = c.panel, scrimColor = c.dim,
        shape = CcShape.sheetTop,
        dragHandle = { Box(Modifier.padding(top = 12.dp, bottom = 8.dp).size(40.dp, 5.dp).clip(RoundedCornerShape(3.dp)).ba
ckground(c.toggleOff)) }
    ) {
        Column(Modifier.padding(start = 16.dp, end = 16.dp, bottom = 24.dp)) {
            Text(title, style = CcType.sheetTitle, color = c.text)
            Spacer(Modifier.height(12.dp))
            content()
        }
    }
}

data class MenuItem(val icon: String, val text: String, val destructive: Boolean = false, val onClick: () -> Unit)

@Composable
fun CcPopupMenu(expanded: Boolean, onDismiss: () -> Unit, items: List<MenuItem>) {
    val c = CcTheme.colors
    DropdownMenu(
        expanded = expanded, onDismissRequest = onDismiss,
        modifier = Modifier.width(210.dp).background(c.panel, CcShape.menu).border(1.dp, c.border, CcShape.menu),
        shape = CcShape.menu, containerColor = c.panel, shadowElevation = 12.dp
    ) {
        items.forEach { m ->
            Row(Modifier.fillMaxWidth().height(42.dp).clickable { onDismiss(); m.onClick() }.padding(horizontal = 14.dp),
                verticalAlignment = Alignment.CenterVertically) {
                CcIcon(m.icon, if (m.destructive) c.red else c.text2, 17.dp)
                Spacer(Modifier.width(11.dp))
                OneLine(m.text, CcType.body, if (m.destructive) c.red else c.text)
            }
        }
    }
}

@Composable
fun CcConfirmDialog(title: String, text: String, confirmText: String, destructive: Boolean,
                    onConfirm: () -> Unit, onDismiss: () -> Unit) {
    val c = CcTheme.colors
    Dialog(onDismissRequest = onDismiss) {
        Column(Modifier.width(300.dp).clip(CcShape.dialog).background(c.panel).padding(20.dp)) {
            Text(title, style = CcType.itemTitle.copy(fontSize = 17.sp, fontWeight = FontWeight.ExtraBold), color = c.text)
            Spacer(Modifier.height(8.dp))
            Text(text, style = CcType.small.copy(fontSize = 13.5.sp), color = c.textSoft)
            Spacer(Modifier.height(18.dp))
            Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
                SecondaryButton("Cancel", onDismiss, Modifier.weight(1f), height = 44.dp)
                if (destructive) SecondaryButton(confirmText, onConfirm, Modifier.weight(1f), destructive = true, height =
44.dp)
                else PrimaryButton(confirmText, onConfirm, Modifier.weight(1f), height = 44.dp)
            }
        }
    }
}

/** Simple toast host. Show with toast.show("Press back again to exit"). */
class ToastState { var message by mutableStateOf<String?>(null)
    suspend fun show(text: String) { message = text; delay(2000); message = null } }

@Composable
fun ToastHost(state: ToastState, modifier: Modifier = Modifier) {
    val c = CcTheme.colors
    AnimatedVisibility(state.message != null, modifier, enter = fadeIn(tween(150)), exit = fadeOut(tween(200))) {
        Box(Modifier.padding(bottom = 28.dp).clip(RoundedCornerShape(20.dp)).background(c.text2).padding(horizontal = 16.d
p, vertical = 10.dp)) {
            Text(state.message ?: "", style = CcType.small, color = Color.White)
        }
    }
}
```

2.4 The shell: state, navigation, sidebar and swipe

File `app/AppState.kt` (one source of truth)

```
enum class Tool(val icon: String, val label: String) {
    Files("folder", "Files"), Search("search", "Search"), GitHub("gh", "GitHub"),
```

```
    Terminal("term", "Terminal"), Logcat("bug", "Logcat"), Problems("alert", "Problems")
}

/** Full screens (layer 3). Each knows where it came from through the stack. */
sealed interface Route {
    val title: String
    data object Settings : Route { override val title = "Settings" }
    data class SettingsPage(val page: String) : Route { override val title get() = page }
    data object StartProject : Route { override val title = "Start a project" }
    data object Clone : Route { override val title = "Clone a repository" }
    data object Cloning : Route { override val title = "Cloning" }
    data object FirstSetup : Route { override val title = "Set up" }
    data object SourceControl : Route { override val title = "Source control" }
    data object History : Route { override val title = "History" }
    data class Commit(val hash: String) : Route { override val title get() = hash.take(7) }
    data class Diff(val path: String) : Route { override val title get() = path.substringAfterLast('/') }
    data class TerminalSession(val id: String, val name: String) : Route { override val title get() = name }
    data object Run : Route { override val title = "Run" }
    data object BuildFinished : Route { override val title = "Build finished" }
    data object Preview : Route { override val title = "Preview" }
    data object Console : Route { override val title = "Console" }
    data object ComposePreviews : Route { override val title = "Previews" }
    data object GitHubSignIn : Route { override val title = "Sign in" }
    data object ProjectSettings : Route { override val title = "Project settings" }
}

sealed interface Sheet {
    data object ProjectSwitcher : Sheet
    data class FileMenu(val path: String) : Sheet
    data object NewCommand : Sheet
}

class AppNav : ViewModel() {
    var sidebarOpen by mutableStateOf(false)
    var sidebarTool by mutableStateOf(Tool.Files)
    val stack = mutableStateListOf<Route>()
    var sheet by mutableStateOf<Sheet?>(null)
    private var lastBackAtEditor = 0L
    val toast = ToastState()

    fun push(r: Route) { stack.add(r) }
    fun closeSidebar() { sidebarOpen = false }

    /** The origin shown in a header = the screen under it in the stack, or the open tool, or the project name. */
    fun originOf(index: Int, projectName: String?): String? = when {
        index > 0 -> stack[index - 1].title
        stack[index] is Route.SettingsPage -> "Settings"
        else -> when (stack[index]) {
            Route.Clone, Route.SourceControl, Route.GitHubSignIn -> "GitHub"
            is Route.TerminalSession -> "Terminal"
            Route.ProjectSettings -> projectName
            else -> null
        }
    }

    /** Returns true when the app should finish. Same function for the header back button and system Back. */
    fun onBack(nowMillis: Long): BackResult {
        return when {
            sheet != null -> { sheet = null; BackResult.Handled }
            stack.isNotEmpty() -> { stack.removeAt(stack.lastIndex); BackResult.Handled }
            sidebarOpen -> { sidebarOpen = false; BackResult.Handled }
            nowMillis - lastBackAtEditor < 2000 -> BackResult.Exit
            else -> { lastBackAtEditor = nowMillis; BackResult.AskAgain }
        }
    }
}
enum class BackResult { Handled, AskAgain, Exit }
```

In the Activity: `BackHandler(enabled = true) { when (nav.onBack(SystemClock.elapsedRealtime())) { BackResult.Exit -> finish(); BackResult.AskAgain -> scope.launch { nav.toast.show("Press back again to exit") }; else -> {} } }`. Header back buttons call the same code through a helper `onBack()`.

File `app/Sidebar.kt` (rail + panel + the swipe)

```
private val SidebarWidth = 300.dp
private val RailWidth = 66.dp
private val EdgeZone = 40.dp
```

```
/** Wraps the editor. `sidebar` is the rail+panel content, drawn BEHIND the editor card. */
@Composable
fun SidebarHost(
    open: Boolean, onOpenChange: (Boolean) -> Unit, keyboardVisible: Boolean,
    canScrollLeft: () -> Boolean,          // true while the code view is scrolled sideways (editor gives this)
    sidebar: @Composable () -> Unit, editor: @Composable () -> Unit
) {
    val c = CcTheme.colors
    val density = LocalDensity.current
    val widthPx = with(density) { SidebarWidth.toPx() }
    val edgePx = with(density) { EdgeZone.toPx() }
    val offset = remember { Animatable(0f) }        // 0 .. widthPx
    val scope = rememberCoroutineScope()
    val spec = spring<Float>(dampingRatio = 0.85f, stiffness = 400f)

    // keep the animation in step with the state (Back button, tool taps...)
    LaunchedEffect(open) { offset.animateTo(if (open) widthPx else 0f, spec) }

    val progress = (offset.value / widthPx).coerceIn(0f, 1f)

    Box(Modifier.fillMaxSize().background(c.rail)) {
        // layer 2: the two panels (rail + panel) sit under the card
        Box(Modifier.fillMaxHeight().width(SidebarWidth)) { sidebar() }

        // layer 1: the editor card. It slides right 300 dp and keeps its left corners rounded (radius 26).
        Box(
            Modifier.fillMaxSize()
                .graphicsLayer {
                    translationX = offset.value
                    shape = RoundedCornerShape(topStart = (26f * progress).dp, bottomStart = (26f * progress).dp)
                    clip = true
                    shadowElevation = 24f * progress
                }
                .pointerInput(open, keyboardVisible, canScrollLeft) {
                    if (keyboardVisible) return@pointerInput
                    awaitEachGesture {
                        // Initial pass: we see the touch before the editor does, but we only take it when it is our swipe.
                        val down = awaitFirstDown(requireUnconsumed = false, pass = PointerEventPass.Initial)
                        val inEdge = down.position.x <= edgePx                        // 40 dp edge zone
                        val slop = awaitHorizontalTouchSlopOrCancellation(down.id) { change, over ->
                            val dy = abs(change.position.y - down.position.y)
                            val goingRight = over > 0f
                            val mostlyHorizontal = abs(over) > 2f * dy
                            val allowed = open || inEdge || !canScrollLeft()          // the code cannot scroll left any mo
re
                            if ((open || goingRight) && mostlyHorizontal && allowed) change.consume()
                        } ?: return@awaitEachGesture
                        val tracker = VelocityTracker()
                        horizontalDrag(slop.id) { change ->
                            val dx = change.positionChange().x
                            tracker.addPosition(change.uptimeMillis, change.position)
                            change.consume()
                            scope.launch { offset.snapTo((offset.value + dx).coerceIn(0f, widthPx)) }
                        }
                        val v = tracker.calculateVelocity().x
                        val fast = 800f.dpToPx(density)
                        val target = when { v > fast -> true; v < -fast -> false; else -> offset.value / widthPx > 0.4f }
                        onOpenChange(target)
                        scope.launch { offset.animateTo(if (target) widthPx else 0f, spec) }
                    }
                }
        ) {
            editor()
            // dim and block taps on the editor while it is open
            if (progress > 0f) Box(Modifier.fillMaxSize().background(c.dim.copy(alpha = 0.30f * progress))
                .pointerInput(Unit) { detectTapGestures { onOpenChange(false) } })
        }
    }
}
private fun Float.dpToPx(d: Density) = with(d) { this@dpToPx.dp.toPx() }
```

Note: the editor passes `canScrollLeft = { codeScrollState.value > 0 }`. When the code is at its left end, a right swipe anywhere opens the sidebar. When it is scrolled sideways, the swipe scrolls the code back first, and a swipe from the 40 dp left edge still opens the sidebar. Pass `canScrollLeft = { false }` on screens without sideways scrolling.

File `app/Rail.kt`

```
@Composable
fun ToolRail(
    project: Project?, selected: Tool, badges: Map<Tool, Int>,
    onSelect: (Tool) -> Unit, onSettings: () -> Unit, onProjectClick: () -> Unit
) {
    val c = CcTheme.colors
    Column(Modifier.fillMaxHeight().width(66.dp).background(c.rail)
        .statusBarsPadding().padding(start = 14.dp, top = 12.dp, bottom = 16.dp)) {
        // project avatar from the REAL project; null project -> a plain "plus" tile
        Box(Modifier.size(46.dp).clickable(onClick = onProjectClick)) {
            Box(Modifier.fillMaxSize().clip(CcShape.tileLarge).background(project?.color ?: c.field), contentAlignment = Al
ignment.Center) {
                if (project != null) Text(project.letter, style = CcType.itemTitle.copy(fontSize = 18.sp, fontWeight = Font
Weight.ExtraBold), color = Color.White)
                else CcIcon("plus", c.text2, 20.dp)
            }
            Box(Modifier.align(Alignment.BottomEnd).size(16.dp).clip(CircleShape).background(c.rail), contentAlignment = Al
ignment.Center) {
                CcIcon("cd", c.text2, 10.dp, 2.6f)
            }
        }
        Spacer(Modifier.height(10.dp))
        Box(Modifier.padding(start = 8.dp).size(30.dp, 2.dp).background(c.border))
        Spacer(Modifier.height(10.dp))
        Tool.entries.forEach { tool ->
            RailItem(tool.icon, tool.label, toolColor(tool), tool == selected, badges[tool] ?: 0) { onSelect(tool) }
            Spacer(Modifier.height(11.dp))
        }
        Spacer(Modifier.weight(1f))
        RailItem("cog", "Settings", c.textSoft, false, 0, onSettings)
    }
}

@Composable
private fun toolColor(t: Tool): Color = with(CcTheme.colors) {
    when (t) { Tool.Files -> yellow; Tool.Search -> blue; Tool.GitHub -> text2
        Tool.Terminal -> blue; Tool.Logcat -> green; Tool.Problems -> orange }
}

@Composable
private fun RailItem(icon: String, description: String, tint: Color, selected: Boolean, badge: Int, onClick: () -> Unit) {
    val c = CcTheme.colors
    val radius by animateDpAsState(if (selected) 15.dp else 23.dp, tween(160), label = "r")
    val pill by animateDpAsState(if (selected) 30.dp else 0.dp, tween(160), label = "pill")
    Box(Modifier.size(46.dp)) {
        // the white pill on the left edge, 10 dp outside the tile
        Box(Modifier.align(Alignment.CenterStart).offset(x = (-10).dp).size(5.dp, pill)
            .clip(RoundedCornerShape(topEnd = 4.dp, bottomEnd = 4.dp)).background(Color.White))
        Box(Modifier.fillMaxSize().clip(RoundedCornerShape(radius)).background(if (selected) c.accent else c.panel)
            .clickable(onClick = onClick).semantics { contentDescription = description },
            contentAlignment = Alignment.Center) { CcIcon(icon, if (selected) Color.White else tint, 22.dp) }
        if (badge > 0) Box(Modifier.align(Alignment.TopEnd).offset(x = 4.dp, y = (-4).dp).defaultMinSize(19.dp, 19.dp)
            .clip(CircleShape).background(c.red).border(3.dp, c.rail, CircleShape).padding(horizontal = 4.dp),
            contentAlignment = Alignment.Center) {
            Text(if (badge > 99) "99+" else "$badge", style = CcType.caption.copy(fontSize = 10.sp, fontWeight = FontWeigh
t.Bold), color = Color.White)
        }
    }
}
```

File `app/Panel.kt` (side panel frame and Run card)

```
/** The panel is 234 dp wide (300 - 66). Every tool draws its content inside this frame. */
@Composable
fun SidePanel(title: String, subtitle: String?, actions: @Composable RowScope.() -> Unit = {},
              runCard: (@Composable () -> Unit)? = null, body: @Composable ColumnScope.() -> Unit) {
    val c = CcTheme.colors
    Box(Modifier.fillMaxHeight().width(234.dp).clip(CcShape.panelTop).background(c.panel)) {
        Column(Modifier.fillMaxSize()) {
            Row(Modifier.statusBarsPadding().padding(start = 16.dp, top = 16.dp, end = 12.dp), verticalAlignment = Alignmen
t.CenterVertically) {
                Column(Modifier.weight(1f)) {
                    OneLine(title, CcType.panelTitle, c.text)
                    if (subtitle != null) OneLine(subtitle, CcType.caption, c.muted)
                }
                Row(horizontalArrangement = Arrangement.spacedBy(6.dp), content = actions)
            }
            Column(Modifier.weight(1f).verticalScroll(rememberScrollState()), content = body)
            Spacer(Modifier.height(if (runCard != null) 84.dp else 0.dp))
```

```
        }
        if (runCard != null) Box(Modifier.align(Alignment.BottomCenter).padding(start = 10.dp, end = 10.dp, bottom = 12.d
p)) { runCard() }
    }
}

@Composable
fun RunCard(project: Project, branch: String?, state: RunState, onRun: () -> Unit) {
    val c = CcTheme.colors
    Row(Modifier.fillMaxWidth().height(64.dp).clip(RoundedCornerShape(22.dp)).background(c.content)
        .border(1.dp, c.border, RoundedCornerShape(22.dp)).padding(12.dp),
        verticalAlignment = Alignment.CenterVertically) {
        Box(Modifier.size(36.dp).clip(RoundedCornerShape(12.dp)).background(project.color), contentAlignment = Alignment.Ce
nter) {
            Text(project.letter, style = CcType.itemTitle.copy(fontSize = 15.sp, fontWeight = FontWeight.ExtraBold), color
= Color.White)
        }
        Spacer(Modifier.width(8.dp))
        Column(Modifier.weight(1f)) {                                  // takes what is left, so it can never overlap the b
utton
            OneLine(project.name, CcType.itemTitle.copy(fontSize = 14.sp, fontWeight = FontWeight.Bold), c.text)
            Row(verticalAlignment = Alignment.CenterVertically) {
                CcIcon("branch", c.muted, 11.dp)
                Spacer(Modifier.width(3.dp))
                OneLine(listOfNotNull(branch, state.label).joinToString(" - "), CcType.caption.copy(fontSize = 11.5.sp),
                    when (state) { RunState.Failed -> c.red; RunState.Ready -> c.green; else -> c.muted })
            }
        }
        Spacer(Modifier.width(8.dp))
        Box(Modifier.size(40.dp).clip(RoundedCornerShape(14.dp)).background(c.green).clickable(onClick = onRun)
            .semantics { contentDescription = "Run" }, contentAlignment = Alignment.Center) {
            CcIcon("play", Color.White, 17.dp)
        }
    }
}
```

The panel is 234 dp wide, but the mockups use an icon-only Run button, so the Run button is icon only and the name column g
ets the remaining width with an ellipsis. Never let the project name or the state wrap or overlap the button.

File `app/MainScreen.kt` (puts everything together)

```kotlin
@Composable
fun MainScreen(vm: MainViewModel, nav: AppNav) {
    val project by vm.project.collectAsState()              // REAL project or null
    val c = CcTheme.colors
    val imeVisible = WindowInsets.isImeVisible

    Box(Modifier.fillMaxSize().background(c.rail)) {
        SidebarHost(
            open = nav.sidebarOpen, onOpenChange = { nav.sidebarOpen = it }, keyboardVisible = imeVisible,
            sidebar = {
                Row {
                    ToolRail(project, nav.sidebarTool, vm.badges.collectAsState().value,
                        onSelect = { nav.sidebarTool = it },
                        onSettings = { nav.push(Route.Settings) },
                        onProjectClick = { nav.sheet = Sheet.ProjectSwitcher })
                    Crossfade(nav.sidebarTool, animationSpec = tween(120), label = "tool") { tool ->
                        when (tool) {
                            Tool.Files -> FilesPanel(vm, nav)
                            Tool.Search -> SearchPanel(vm, nav)
                            Tool.GitHub -> GitHubPanel(vm, nav)
                            Tool.Terminal -> TerminalPanel(vm, nav)
                            Tool.Logcat -> LogcatPanel(vm, nav)
                            Tool.Problems -> ProblemsPanel(vm, nav)
                        }
                    }
                }
            },
            canScrollLeft = { vm.codeScrollX > 0 },
            editor = { if (project == null) WelcomeScreen(vm, nav) else EditorScreen(vm, nav) }
        )

        // layer 3: full screens slide over everything, including the sidebar
        nav.stack.forEachIndexed { index, route ->
            val isTop = index == nav.stack.lastIndex
            AnimatedVisibility(
                visible = true,
                enter = slideInHorizontally(tween(220, easing = FastOutSlowInEasing)) { it },
            ) {
                Box(Modifier.fillMaxSize().background(c.content)
                    .pointerInput(Unit) {}) {     // blocks touches reaching what is underneath
                    RouteScreen(route, origin = nav.originOf(index, project?.name), vm = vm, nav = nav)
                }
```

```
            }
        }

        // layer 4: sheets
        when (val s = nav.sheet) {
            Sheet.ProjectSwitcher -> ProjectSwitcherSheet(vm, nav)
            is Sheet.FileMenu -> FileMenu(s.path, vm, nav)
            Sheet.NewCommand -> NewCommandSheet(vm, nav)
            null -> {}
        }
        ToastHost(nav.toast, Modifier.align(Alignment.BottomCenter))
    }
}
```

`RouteScreen` is a `when (route)` that calls each full screen in 2.5. `Project` is the app's real project model: `name`, `letter = name.first().uppercase()`, `color` (stored or derived).

2.5 Every screen: plain description first, then the code

Layout of each entry: **What it is and why it is there**, **Behaviour**, **Data (real)**, then the **code**. Screen numbers match the PNG files. In the code, `vm` is the real ViewModel that exposes `StateFlow`s. Nothing is typed in.

01 Welcome (full screen, layer 3 root when no project is open)

Why: the first thing a user sees when no project is open. **Behaviour:** three buttons start the three ways to get a project. Under them are the real recent projects (hidden when there are none). Back exits the app. **Data:** `vm.recentProjects`. Layout: top padding 72, logo tile 64 (radius 20, accent), hero text 33 sp, one line 14 sp muted, buttons 52 tall 12 apart, side padding 22.

```
@Composable
fun WelcomeScreen(vm: MainViewModel, nav: AppNav) {
    val c = CcTheme.colors
    val recent by vm.recentProjects.collectAsState()
    Column(Modifier.fillMaxSize().background(c.content).statusBarsPadding().padding(horizontal = 22.dp)) {
        Spacer(Modifier.height(72.dp))
        Box(Modifier.size(64.dp).clip(RoundedCornerShape(20.dp)).background(c.accent), contentAlignment = Alignment.Center)
{
            CcIcon("code", Color.White, 30.dp, 2f)
        }
        Spacer(Modifier.height(22.dp))
        Text("CloseCode", style = CcType.hero, color = c.text)
        Spacer(Modifier.height(8.dp))
        Text(stringResource(R.string.welcome_line), style = CcType.body.copy(fontWeight = FontWeight.Normal), color = c.mut
ed)
        Spacer(Modifier.height(28.dp))
        PrimaryButton("Start a project", { nav.push(Route.StartProject) }, Modifier.fillMaxWidth(), icon = "plus", height =
52.dp, radius = 16.dp)
        Spacer(Modifier.height(12.dp))
        SecondaryButton("Clone a repository", { nav.push(Route.Clone) }, Modifier.fillMaxWidth(), icon = "branch", height =
52.dp)
        Spacer(Modifier.height(12.dp))
        SecondaryButton("Open a folder", vm::pickFolder, Modifier.fillMaxWidth(), icon = "folder", height = 52.dp)
        if (recent.isNotEmpty()) {
            Text("RECENT", style = CcType.section, color = c.muted, modifier = Modifier.padding(top = 28.dp, bottom = 6.d
p))
            LazyColumn { items(recent, key = { it.id }) { p ->
                PanelRow("folder", p.color, p.name, relativeTime(p.lastOpened), onClick = { vm.open(p) })
            } }
        }
    }
}
```

02 Start a project (full screen)

Why: create a project from a template the app really ships. **Behaviour:** pick a template card (selected card gets a 2 dp accent outline), type a name (validated: not empty, not already used), choose the location, press Create. A progress bar shows while files are written, then the editor opens. **Data:** `vm.templates`, `vm.storageRoot`.

```
@Composable
fun StartProjectScreen(origin: String?, vm: MainViewModel, onBack: () -> Unit) {
    val c = CcTheme.colors
    val templates by vm.templates.collectAsState()
    var selected by remember { mutableStateOf(templates.firstOrNull()?.id) }
    var name by remember { mutableStateOf("") }
    val error = vm.nameError(name)
    Column(Modifier.fillMaxSize().background(c.content)) {
        ScreenHeader(origin, "Start a project", onBack)
        LazyVerticalGrid(GridCells.Fixed(2), Modifier.weight(1f), contentPadding = PaddingValues(16.dp),
            horizontalArrangement = Arrangement.spacedBy(10.dp), verticalArrangement = Arrangement.spacedBy(10.dp)) {
            items(templates, key = { it.id }) { t ->
                val sel = t.id == selected
                Column(Modifier.clip(CcShape.card).background(c.field)
                    .border(if (sel) 2.dp else 0.dp, if (sel) c.accent else Color.Transparent, CcShape.card)
                    .clickable { selected = t.id }.padding(14.dp)) {
                    Box(Modifier.size(42.dp).clip(CcShape.tile).background(t.tint.tint()), contentAlignment = Alignment.Cen
ter) { CcIcon(t.icon, t.tint, 21.dp) }
                    Spacer(Modifier.height(10.dp))
                    OneLine(t.name, CcType.itemTitle, c.text)
                    OneLine(t.description, CcType.caption, c.muted)
                }
            }
        }
        Column(Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
            CcTextField(name, { name = it }, Modifier.fillMaxWidth(), label = "Project name", hint = "my-app", error = if
(name.isNotEmpty()) error else null)
            PrimaryButton("Create", { vm.create(selected!!, name) }, Modifier.fillMaxWidth(), enabled = selected != null &&
name.isNotBlank() && error == null)
        }
    }
}
```

03 Switch project (bottom sheet, layer 4)

Why: change the open project quickly. **Behaviour:** opens when the avatar at the top of the rail is tapped. Lists the real projects, current one has a check. Buttons below start, clone or open. No projects: the empty state. **Data:** `vm.projects`.

```
@Composable
fun ProjectSwitcherSheet(vm: MainViewModel, nav: AppNav) {
    val projects by vm.projects.collectAsState(); val current by vm.project.collectAsState()
    CcBottomSheet("Projects", onDismiss = { nav.sheet = null }) {
        if (projects.isEmpty()) EmptyState("folder", "No projects yet", "Start a project, clone one or open a folder.")
        else projects.forEach { p ->
            PanelRow("folder", p.color, p.name, relativeTime(p.lastOpened), selected = p.id == current?.id,
                onClick = { vm.open(p); nav.sheet = null; nav.closeSidebar() },
                trailing = if (p.id == current?.id) ({ CcIcon("check", CcTheme.colors.accent, 18.dp, 2.4f) }) else null)
        }
        Spacer(Modifier.height(12.dp))
        PrimaryButton("New project", { nav.sheet = null; nav.push(Route.StartProject) }, Modifier.fillMaxWidth(), icon = "p
lus")
        Spacer(Modifier.height(8.dp))
        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            SecondaryButton("Open folder", { nav.sheet = null; vm.pickFolder() }, Modifier.weight(1f), icon = "folder")
            SecondaryButton("Clone", { nav.sheet = null; nav.push(Route.Clone) }, Modifier.weight(1f), icon = "branch")
        }
    }
}
```

04 Clone a repository, 05 Cloning progress (full screens)

Why: get a project from git. **Behaviour:** URL (mono) and optional branch. Clone starts a real `git clone` and goes to the progress screen. Progress shows the real step text (Counting, Receiving, Resolving)

and percent. Cancel stops the process and deletes the half-cloned folder. On success the project opens in the editor and the stack is cleared. On error the real message shows in an `ErrorCard` with Retry. **Data:** `vm.cloneState`.

```
@Composable
fun CloneScreen(origin: String?, vm: MainViewModel, onBack: () -> Unit, onStarted: () -> Unit) {
    val c = CcTheme.colors
    var url by remember { mutableStateOf("") }; var branch by remember { mutableStateOf("") }
    val signedIn by vm.github.signedIn.collectAsState()
    Column(Modifier.fillMaxSize().background(c.content)) {
        ScreenHeader(origin, "Clone a repository", onBack)
        Column(Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(14.dp)) {
            CcTextField(url, { url = it }, Modifier.fillMaxWidth(), label = "Repository link", hint = "https://github.com/u
ser/repo.git", mono = true, leadingIcon = "globe")
            CcTextField(branch, { branch = it }, Modifier.fillMaxWidth(), label = "Branch (optional)", hint = "main", mono
= true, leadingIcon = "branch")
            if (!signedIn) Text("Private repositories need GitHub sign in.", style = CcType.caption, color = c.muted)
            PrimaryButton("Clone", { vm.startClone(url.trim(), branch.trim().ifEmpty { null }); onStarted() },
                Modifier.fillMaxWidth(), icon = "down", enabled = url.isNotBlank())
        }
    }
}

@Composable
fun CloningScreen(origin: String?, vm: MainViewModel, onBack: () -> Unit) {
    val s by vm.cloneState.collectAsState()
    Column(Modifier.fillMaxSize().background(CcTheme.colors.content)) {
        ScreenHeader(origin, "Cloning", onBack)
        Column(Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(14.dp)) {
            when (val st = s) {
                is CloneState.Running -> { ProgressLine(st.step, st.percent / 100f, "${st.percent}%"); SecondaryButton("Can
cel", vm::cancelClone, Modifier.fillMaxWidth(), destructive = true) }
                is CloneState.Failed -> ErrorCard(st.message, onRetry = vm::retryClone)
                else -> {}
            }
        }
    }
}
```

06 First-time setup (full screen)

Why: downloads build tools the app needs. **Behaviour:** each card is a tool the app really needs, with real download progress, a tick when done and a Skip for optional ones. Finish is disabled (grey) until the required cards are done. **Data:** `vm.toolchain` (list of tools with state Missing, Downloading(percent), Ready).

```
@Composable
fun FirstSetupScreen(origin: String?, vm: MainViewModel, onBack: () -> Unit, onFinish: () -> Unit) {
    val c = CcTheme.colors
    val tools by vm.toolchain.collectAsState()
    val canFinish = tools.filter { it.required }.all { it.state is ToolState.Ready }
    Column(Modifier.fillMaxSize().background(c.content)) {
        ScreenHeader(origin, "Set up", onBack)
        Column(Modifier.weight(1f).verticalScroll(rememberScrollState()).padding(horizontal = 22.dp, vertical = 20.dp)) {
            Text("CLOSECODE", style = CcType.section.copy(fontSize = 13.sp), color = c.accent)
            Text("Set up your build environment", style = CcType.setupTitle, color = c.text, modifier = Modifier.padding(to
p = 10.dp))
            Text("One-time downloads. You can skip any and add it later in Settings.", style = CcType.small.copy(fontSize =
13.5.sp), color = c.muted, modifier = Modifier.padding(top = 8.dp, bottom = 18.dp))
            tools.forEach { t -> ToolCard(t, onDownload = { vm.download(t.id) }, onSkip = { vm.skip(t.id) }) }
        }
        PrimaryButton("Finish setup", onFinish, Modifier.padding(16.dp).fillMaxWidth(), enabled = canFinish, height = 52.d
p, radius = 16.dp)
    }
}
```

07 Editor, 08 Editor with keyboard (layer 1)

Why: the app's home. **Behaviour:** the header has the file name, tabs for opened files and undo and redo only (no Save, no More; it saves automatically). Code area with line numbers, current line highlight, git change bars in the gutter. When the keyboard is up, a key row sits above it with Tab, brackets, quotes, arrows. The sidebar can only be opened by swiping right, and the swipe works from anywhere on the editor. **Data:** the open file from disk, tabs from `vm.openFiles`.

```
@Composable
fun EditorScreen(vm: MainViewModel, nav: AppNav) {
    val c = CcTheme.colors
    val tabs by vm.openFiles.collectAsState(); val active by vm.activeFile.collectAsState()
    Column(Modifier.fillMaxSize().background(c.content).statusBarsPadding().imePadding()) {
        Row(Modifier.fillMaxWidth().height(56.dp).padding(start = 18.dp, end = 14.dp), verticalAlignment = Alignment.Center
Vertically) {
            OneLine(active?.name ?: "No file open", CcType.screenTitle, c.text, Modifier.weight(1f))
            HeaderAction("undo", "Undo", enabled = vm.canUndo.collectAsState().value, onClick = vm::undo)
            Spacer(Modifier.width(10.dp))
            HeaderAction("redo", "Redo", enabled = vm.canRedo.collectAsState().value, onClick = vm::redo)
        }
        LazyRow(Modifier.fillMaxWidth().height(40.dp), contentPadding = PaddingValues(horizontal = 12.dp), horizontalArrang
ement = Arrangement.spacedBy(6.dp)) {
            items(tabs, key = { it.path }) { f -> CcChip(f.name, f.path == active?.path) { vm.activate(f) } }
        }
        Box(Modifier.weight(1f)) {
            if (active == null) EmptyState("file", "No file open", "Swipe right and pick a file.")
            else CodeArea(active!!, vm)
        }
        if (WindowInsets.isImeVisible) KeyRow(onKey = vm::insertText)
    }
}

@Composable
fun KeyRow(onKey: (String) -> Unit) {
    val c = CcTheme.colors
    LazyRow(Modifier.fillMaxWidth().height(40.dp).background(c.panel), contentPadding = PaddingValues(horizontal = 8.dp, ve
rtical = 4.dp),
        horizontalArrangement = Arrangement.spacedBy(6.dp)) {
        items(listOf("Tab" to "\t", "{" to "{", "}" to "}", "(" to "(", ")" to ")", "[" to "[", "]" to "]", "\"" to "\"",
";" to ";", "=" to "=", "<" to "<", ">" to ">")) { (label, text) ->
            Box(Modifier.height(32.dp).defaultMinSize(minWidth = 36.dp).clip(RoundedCornerShape(9.dp)).background(c.field)
                .clickable { onKey(text) }.padding(horizontal = 10.dp), contentAlignment = Alignment.Center) {
                Text(label, style = CcType.code.copy(fontSize = 12.sp, fontWeight = FontWeight.SemiBold), color = c.text2)
            }
        }
    }
}
```

`CodeArea` is the app's existing editor view. Keep its logic. Only restyle: mono 11.5 sp, line height 21 sp, line number column 34 dp wide in `faint`, current line `field`, syntax colours from the table, git gutter bars 3 dp wide.

09 Swipe and 10 Files panel, 11 Files menu

Why: browse and manage project files. **Behaviour:** title "Files" with the project name under it, buttons New file and New folder, a filter field, the tree (folders expand with a chevron that turns 90 degrees, 120 ms), git letters on the right. Tap file: opens it and closes the sidebar. Long-press: popup menu. The Run card sits at the bottom. **Data:** `vm.tree` (real folder), `vm.gitStatus`.

```
@Composable
fun FilesPanel(vm: MainViewModel, nav: AppNav) {
    val c = CcTheme.colors
    val project by vm.project.collectAsState(); val rows by vm.visibleTree.collectAsState()
    var filter by remember { mutableStateOf("") }
    SidePanel("Files", project?.name,
        actions = { HeaderAction("plus", "New file") { vm.newFile() }; HeaderAction("folder", "New folder") { vm.newFolder
() } },
        runCard = project?.let { p -> { RunCard(p, vm.branch.collectAsState().value, vm.runState.collectAsState().value) {
nav.push(Route.Run); vm.run() } } }
    ) {
        CcTextField(filter, { filter = it; vm.setFilter(it) }, Modifier.padding(horizontal = 12.dp, vertical = 10.dp), hint
= "Filter files", leadingIcon = "search")
```

```
        if (rows.isEmpty()) EmptyState("folder", "This folder is empty", "Create a file or a folder.", "New file", vm::newF
ile)
        rows.forEach { n ->
            TreeRow(n, onClick = { if (n.isDir) vm.toggle(n) else { vm.openFile(n); nav.closeSidebar() } },
                    onLongClick = { nav.sheet = Sheet.FileMenu(n.path) })
        }
    }
}

@OptIn(ExperimentalFoundationApi::class)
@Composable
fun TreeRow(n: TreeNode, onClick: () -> Unit, onLongClick: () -> Unit) {
    val c = CcTheme.colors
    val rot by animateFloatAsState(if (n.expanded) 90f else 0f, tween(120), label = "chev")
    val (icon, tint) = fileIconFor(n)          // by REAL extension, see the Icons section
    Row(Modifier.fillMaxWidth().padding(horizontal = 8.dp, vertical = 1.dp).height(38.dp).clip(RoundedCornerShape(11.dp))
        .background(if (n.selected) c.selectedFile else Color.Transparent)
        .combinedClickable(onClick = onClick, onLongClick = onLongClick).padding(start = (8 + 16 * n.depth).dp, end = 8.d
p),
        verticalAlignment = Alignment.CenterVertically) {
        if (n.isDir) CcIcon("cr", c.faint, 13.dp, 2.2f, Modifier.rotate(rot)) else Spacer(Modifier.width(13.dp))
        Spacer(Modifier.width(6.dp)); CcIcon(icon, tint, 17.dp, 1.7f); Spacer(Modifier.width(8.dp))
        OneLine(n.name, CcType.small.copy(fontSize = 13.5.sp), c.text2, Modifier.weight(1f))
        n.gitLetter?.let { Text(it, style = CcType.code.copy(fontSize = 11.sp, fontWeight = FontWeight.Bold), color = when
(it) { "M" -> c.amber; "A" -> c.green; else -> c.red }) }
    }
}

fun fileIconFor(n: TreeNode): Pair<String, Color> = when {
    n.isDir -> "folder" to LightCcColors.yellow
    n.ext in setOf("kt", "java", "ts", "js", "py", "html", "css", "xml", "json", "gradle", "kts") -> "code" to LightCcColor
s.purple
    else -> "file" to LightCcColors.blue
}
```

File menu items (show only the ones the app supports): New file, New folder, Rename, Duplicate, Delete (red, goes through `CcConfirmDialog`), Copy path. Use `CcPopupMenu` anchored at the row.

12 Search and 13 Search options (panel, layer 2)

Why: find text in the project. **Behaviour:** type to search after a 300 ms pause. Chips for match case, whole word, regex; include and exclude path fields. Results are grouped by file; tap a result: open the file at that line and close the sidebar. Replace is a toggle that shows a second field and a Replace all button (asks to confirm). **Data:** `vm.searchResults` (real).

```
@Composable
fun SearchPanel(vm: MainViewModel, nav: AppNav) {
    val c = CcTheme.colors
    val q by vm.query.collectAsState(); val opts by vm.searchOptions.collectAsState(); val res by vm.searchResults.collectA
sState()
    val project by vm.project.collectAsState()
    SidePanel("Search", project?.name, runCard = project?.let { p -> { RunCard(p, vm.branch.collectAsState().value, vm.runS
tate.collectAsState().value) { nav.push(Route.Run); vm.run() } } }) {
        CcTextField(q, vm::setQuery, Modifier.padding(horizontal = 12.dp, vertical = 10.dp), hint = "Search", leadingIcon =
"search")
        LazyRow(contentPadding = PaddingValues(horizontal = 12.dp), horizontalArrangement = Arrangement.spacedBy(6.dp)) {
            item { CcChip("Aa", opts.matchCase) { vm.toggleCase() } }
            item { CcChip("Word", opts.wholeWord) { vm.toggleWord() } }
            item { CcChip(".*", opts.regex) { vm.toggleRegex() } }
        }
        when (val r = res) {
            SearchState.Idle -> {}
            SearchState.Running -> LoadingList(3)
            is SearchState.Done -> if (r.groups.isEmpty()) EmptyState("search", "No results", "Try different words or optio
ns.")
                else r.groups.forEach { g ->
                    Text(g.path, style = CcType.section.copy(fontSize = 11.sp), color = c.muted, modifier = Modifier.paddin
g(start = 16.dp, top = 12.dp, bottom = 4.dp), maxLines = 1, overflow = TextOverflow.Ellipsis)
                    g.hits.forEach { h -> ResultCard(h) { vm.openAt(g.path, h.line); nav.closeSidebar() } }
                }
            is SearchState.Failed -> ErrorCard(r.message, onRetry = vm::search)
        }
    }
}
```

```
@Composable
fun ResultCard(h: SearchHit, onClick: () -> Unit) {
    val c = CcTheme.colors
    val text = buildAnnotatedString {
        append(h.before)
        withStyle(SpanStyle(background = c.matchBg, color = c.matchText)) { append(h.match) }
        append(h.after)
    }
    Row(Modifier.padding(horizontal = 14.dp, vertical = 2.dp).fillMaxWidth().clip(RoundedCornerShape(9.dp)).background(c.co
ntent)
        .clickable(onClick = onClick).padding(horizontal = 9.dp, vertical = 6.dp)) {
        Text("${h.line}", style = CcType.code.copy(fontSize = 11.sp), color = c.faint, modifier = Modifier.width(26.dp))
        Text(text, style = CcType.code.copy(fontSize = 11.sp, lineHeight = 15.sp), color = c.text2, maxLines = 1, overflow
= TextOverflow.Ellipsis)
    }
}
```

14 GitHub (panel), 15 Not connected, 16 Sign in (full screen)

Why: use GitHub from the phone. **Behaviour:** if no account is stored the panel shows the empty state with a Sign in button, which opens the sign-in screen. Sign in shows the real steps of the app's sign-in flow (for example the device code and the link). When signed in the panel shows the account and actions (Clone, Source control, History, Pull, Push). **Data:** `vm.github.account`.

```
@Composable
fun GitHubPanel(vm: MainViewModel, nav: AppNav) {
    val acc by vm.github.account.collectAsState()
    val c = CcTheme.colors
    SidePanel("GitHub", acc?.login) {
        if (acc == null) EmptyState("gh", "Not connected", "Sign in to clone, pull and push.", "Sign in") { nav.push(Route.
GitHubSignIn) }
        else {
            PanelRow("gh", c.text2, acc!!.login, acc!!.name ?: "Signed in") {}
            PanelRow("branch", c.blue, "Clone a repository", null, onClick = { nav.push(Route.Clone) })
            PanelRow("refresh", c.green, "Source control", vm.changeCount.collectAsState().value.let { if (it == 0) "No cha
nges" else "$it changes" }, onClick = { nav.push(Route.SourceControl) })
            PanelRow("history", c.purple, "History", null, onClick = { nav.push(Route.History) })
        }
    }
}
```

17 Source control, 18 More, 19 History, 20 Commit, 21 Diff (full screens)

Why: the git workflow. **Behaviour:** Source control shows the branch, Pull and Push small buttons, then Staged and Changes lists (tap a row to open its diff, tap the tick to stage or unstage), a message field and Commit (disabled while the message is empty or nothing is staged). The three-dot icon in this screen is not allowed (no More button): the extra git actions live in the branch row, which opens a popup menu. History lists the real commits. Commit shows its files. Diff shows green added lines and red removed lines using the diff tokens. **Data:** `vm.git.*` (status, log, show, diff).

```
@Composable
fun SourceControlScreen(origin: String?, vm: MainViewModel, nav: AppNav, onBack: () -> Unit) {
    val c = CcTheme.colors
    val st by vm.git.status.collectAsState(); var msg by remember { mutableStateOf("") }
    var menu by remember { mutableStateOf(false) }
    Column(Modifier.fillMaxSize().background(c.content)) {
        ScreenHeader(origin, "Source control", onBack)
        Row(Modifier.padding(14.dp), verticalAlignment = Alignment.CenterVertically) {
            Box { Row(Modifier.clip(RoundedCornerShape(12.dp)).background(c.field).clickable { menu = true }.padding(horizo
ntal = 12.dp, vertical = 9.dp), verticalAlignment = Alignment.CenterVertically) {
                CcIcon("branch", c.muted, 14.dp); Spacer(Modifier.width(6.dp)); OneLine(st.branch, CcType.small, c.text); S
pacer(Modifier.width(4.dp)); CcIcon("cd", c.faint, 14.dp, 2.2f) }
                CcPopupMenu(menu, { menu = false }, listOf(MenuItem("history", "History") { nav.push(Route.History) }, Menu
Item("refresh", "Refresh") { vm.git.refresh() })) }
            Spacer(Modifier.weight(1f))
            SecondaryButton("Pull", vm.git::pull, icon = "down", height = 40.dp); Spacer(Modifier.width(8.dp))
            SecondaryButton("Push", vm.git::push, icon = "up", height = 40.dp)
        }
```

```
        LazyColumn(Modifier.weight(1f)) {
            if (st.isClean) item { EmptyState("check", "No changes", "Your working tree is clean.") }
            if (st.staged.isNotEmpty()) { item { SectionLabel("Staged · ${st.staged.size}") }; items(st.staged, key = { "s"
+ it.path }) { ChangeRow(it, true, { nav.push(Route.Diff(it.path)) }) { vm.git.unstage(it.path) } } }
            if (st.unstaged.isNotEmpty()) { item { SectionLabel("Changes · ${st.unstaged.size}") }; items(st.unstaged, key
= { "u" + it.path }) { ChangeRow(it, false, { nav.push(Route.Diff(it.path)) }) { vm.git.stage(it.path) } } }
        }
        Column(Modifier.padding(14.dp), verticalArrangement = Arrangement.spacedBy(10.dp)) {
            CcTextField(msg, { msg = it }, Modifier.fillMaxWidth(), hint = "Commit message")
            PrimaryButton("Commit", { vm.git.commit(msg); msg = "" }, Modifier.fillMaxWidth(), icon = "check", enabled = ms
g.isNotBlank() && st.staged.isNotEmpty())
        }
    }
}

@Composable
fun DiffLine(kind: Char, number: Int?, text: String) {
    val c = CcTheme.colors
    val bg = when (kind) { '+' -> c.diffAddBg; '-' -> c.diffDelBg; else -> Color.Transparent }
    val fg = when (kind) { '+' -> c.diffAddText; '-' -> c.diffDelText; else -> c.text2 }
    Row(Modifier.fillMaxWidth().background(bg).padding(horizontal = 10.dp)) {
        Text(number?.toString() ?: "", style = CcType.code.copy(fontSize = 10.5.sp), color = c.faint, modifier = Modifier.w
idth(32.dp))
        Text("$kind $text", style = CcType.code.copy(fontSize = 11.sp), color = fg, softWrap = false)
    }
}
```

22 Terminal (panel), 23 Terminal session (full screen), 24 New command (sheet)

Why: run shell commands. **Behaviour:** the panel lists the real sessions with a green dot for running; tapping one opens the full screen session. The session is a real shell: output area (mono 12, scrolls to the end), input row, key row with Tab, Ctrl, Esc and arrows. The New command sheet has a field and the user's recent and saved commands; Run sends it to the shell. **Data:** `vm.terminal.sessions`, `vm.termina l.output(id)`, `vm.savedCommands`.

```
@Composable
fun TerminalSessionScreen(origin: String?, id: String, name: String, vm: MainViewModel, onBack: () -> Unit) {
    val c = CcTheme.colors
    val lines by vm.terminal.output(id).collectAsState(emptyList()); var input by remember { mutableStateOf("") }
    val listState = rememberLazyListState()
    LaunchedEffect(lines.size) { if (lines.isNotEmpty()) listState.animateScrollToItem(lines.lastIndex) }
    Column(Modifier.fillMaxSize().background(c.content).imePadding()) {
        ScreenHeader(origin, name, onBack, actions = { HeaderAction("trash", "Clear") { vm.terminal.clear(id) } })
        LazyColumn(Modifier.weight(1f).background(c.field).padding(10.dp), state = listState) {
            items(lines) { Text(it.text, style = CcType.log.copy(fontSize = 12.sp), color = if (it.isError) c.red else c.te
xt2) }
        }
        LazyRow(Modifier.height(40.dp).padding(horizontal = 8.dp, vertical = 4.dp), horizontalArrangement = Arrangement.spa
cedBy(6.dp)) {
            items(listOf("Tab" to "\t", "Esc" to "\u001B", "Ctrl" to "", "↑" to "\u001B[A", "↓" to "\u001B[B", "←" to "\u00
1B[D", "→" to "\u001B[C")) { (l, s) ->
                Box(Modifier.height(32.dp).clip(RoundedCornerShape(9.dp)).background(c.field).clickable { vm.terminal.send
(id, s) }.padding(horizontal = 12.dp), contentAlignment = Alignment.Center) { Text(l, style = CcType.code.copy(fontSize = 1
2.sp, fontWeight = FontWeight.SemiBold), color = c.text2) } }
        }
        Row(Modifier.padding(10.dp), verticalAlignment = Alignment.CenterVertically) {
            CcTextField(input, { input = it }, Modifier.weight(1f), hint = "Type a command", mono = true)
            Spacer(Modifier.width(8.dp))
            Box(Modifier.size(44.dp).clip(RoundedCornerShape(13.dp)).background(c.accent).clickable { vm.terminal.send(id,
input + "\n"); input = "" }, contentAlignment = Alignment.Center) { CcIcon("send", Color.White, 18.dp, 2f) }
        }
    }
}
```

25 Run and build, 26 Build finished (full screens)

Why: build and install the app. **Behaviour:** Run opens the screen at once and starts the real build. Steps appear as the build reports them; each step shows a spinner, then a green tick or a red cross. A progress bar shows the real percent. Cancel stops the build. When it ends: Ready shows a green

badge, time taken, Open app (green) and Share; Failed shows a red badge with the real error text and a Problems button. **Data:** `vm.build.state`.

```
@Composable
fun RunScreen(origin: String?, vm: MainViewModel, nav: AppNav, onBack: () -> Unit) {
    val c = CcTheme.colors
    val s by vm.build.state.collectAsState()
    LaunchedEffect(s) { if (s is BuildState.Done) { nav.stack.removeLastOrNull(); nav.push(Route.BuildFinished) } }
    Column(Modifier.fillMaxSize().background(c.content)) {
        ScreenHeader(origin, "Run", onBack)
        (s as? BuildState.Running)?.let { r ->
            Column(Modifier.padding(16.dp)) {
                ProgressLine(r.currentStep, r.percent / 100f, "${r.percent}%")
                Spacer(Modifier.height(12.dp))
                r.steps.forEach { st -> Row(Modifier.padding(vertical = 4.dp), verticalAlignment = Alignment.CenterVertical
ly) {
                    when (st.state) { StepState.Done -> CcIcon("check", c.green, 16.dp, 2.4f); StepState.Failed -> CcIcon
("x", c.red, 16.dp, 2.4f); else -> CircularProgressIndicator(Modifier.size(14.dp), color = c.accent, strokeWidth = 2.dp) }
                    Spacer(Modifier.width(10.dp)); Text(st.name, style = CcType.small, color = c.text2) } }
                Spacer(Modifier.height(12.dp)); SecondaryButton("Cancel", vm.build::cancel, Modifier.fillMaxWidth(), destru
ctive = true)
            }
        }
    }
}
```

27 Website preview, 28 Console, 29 Compose previews (full screens)

Why: see the result without leaving the app. Preview: a `WebView` of the local page with an address pill and a reload icon. Console: the page's `console.*` lines coloured by level (error red, warn amber, others text2). Compose previews: a list of the `@Preview` functions the project really has; empty state when none.

30 Logcat, 31 Problems (panels)

Why: see the app's logs and the code's errors. **Behaviour:** Logcat streams the real device log; chips filter by level; each line is time, level letter (coloured), tag, message, in mono 10.5 sp. Problems lists real errors and warnings grouped by file; tap a row: opens the file at the line and closes the sidebar. **Data:** `vm.logcat`, `vm.problems`.

32 Settings and 33 to 44 Settings pages (full screens)

Why: control the app. **Behaviour:** opening Settings (rail, bottom) pushes the categories screen. Tapping a category pushes its page and the header reads "Settings General" with the back button. Every toggle and value writes to the real settings store at once (`DataStore`). Show only rows that match features the app has. The pages are General, Editor, Code intelligence, Build and toolchain, Terminal, GitHub, Appearance, Storage, Security, About, Developer. Rows that run an action (Clear cache, Delete build tools, Crash test) ask to confirm first with `CcConfirmDialog`.

```
@Composable
fun SettingsScreen(origin: String?, nav: AppNav, onBack: () -> Unit) {
    val c = CcTheme.colors
    Column(Modifier.fillMaxSize().background(c.content)) {
        ScreenHeader(origin, "Settings", onBack)
        Column(Modifier.verticalScroll(rememberScrollState()).padding(bottom = 24.dp)) {
            SettingsGroup("App") {
                SettingRow("cog", c.blue, "General", onClick = { nav.push(Route.SettingsPage("General")) })
                SettingRow("code", c.purple, "Editor", onClick = { nav.push(Route.SettingsPage("Editor")) })
                SettingRow("flask", c.yellow, "Code intelligence", onClick = { nav.push(Route.SettingsPage("Code intelligen
ce")) })
                SettingRow("play", c.green, "Build and toolchain", onClick = { nav.push(Route.SettingsPage("Build and toolc
hain")) })
                SettingRow("term", c.blue, "Terminal", onClick = { nav.push(Route.SettingsPage("Terminal")) })
```

```
                SettingRow("gh", c.text2, "Git and GitHub", onClick = { nav.push(Route.SettingsPage("Git and GitHub")) })
                SettingRow("moon", c.purple, "Appearance", onClick = { nav.push(Route.SettingsPage("Appearance")) })
                SettingRow("db", c.orange, "Storage", onClick = { nav.push(Route.SettingsPage("Storage")) })
                SettingRow("lock", c.teal, "Security", onClick = { nav.push(Route.SettingsPage("Security")) })
                SettingRow("info", c.faint, "About", onClick = { nav.push(Route.SettingsPage("About")) }, showDivider = fal
se)
            }
        }
    }
}

/** Example of one page: every value is read from and written to the real settings store. */
@Composable
fun EditorSettingsPage(origin: String?, vm: SettingsViewModel, onBack: () -> Unit) {
    val c = CcTheme.colors
    val s by vm.editor.collectAsState()
    Column(Modifier.fillMaxSize().background(c.content)) {
        ScreenHeader(origin, "Editor", onBack)
        Column(Modifier.verticalScroll(rememberScrollState())) {
            SettingsGroup("Text") {
                SettingRow("code", c.purple, "Font size", value = "${s.fontSize}", onClick = vm::pickFontSize)
                SettingRow("menu", c.blue, "Word wrap", "Wrap long lines", trailing = { CcToggle(s.wordWrap, vm::setWordWra
p) })
                SettingRow("check", c.green, "Show line numbers", trailing = { CcToggle(s.lineNumbers, vm::setLineNumbers)
}, showDivider = false)
            }
        }
    }
}
```

All the other pages are built exactly the same way: `ScreenHeader`, then one or more `SettingsGroup`s with `SettingRow`s, values read from the store.

45 Project settings (full screen)

Why: edit the project's name, colour, package id, SDK versions and build variants. **Behaviour:** the header shows the project name in bold and "Project settings" muted. A colour picker row lets the user choose a project colour from the six tile colours; the letter updates from the name. Edits save at once. **Data:** the real project and its Gradle files.

Screens that are not in this document

The app has more screens than are written here. Build each one with the same components from 2.3: `ScreenHeader` (with back and origin), `SettingsGroup` and `SettingRow` for option screens, `PanelRow` for lists, `CcTextField` for input, `PrimaryButton` and `SecondaryButton` for actions, `CcBottomSheet`, `CcPopupMenu` and `CcConfirmDialog` for choices, and `EmptyState`, `LoadingList`, `ErrorCard` for the three states. Keep the existing logic. Only change the look and the way the screen opens and closes. If a screen needs an icon that is not in `CcIconPaths`, add its path text to the map using the same 24 by 24 rules.

2.7 Project settings: App icon, Signing keys and Self-test (screens 45 to 51)

These screens exist in the app today as rows (App icon, Debug key, Release key, Self-test). This section says what happens when each row is opened. Every value in the pictures is an example. Everything shown must be read from the real project and the real device.

45 Project settings (the screen that opens them)

The App group has App name, Package name and **App icon** (a small preview of the real icon on the right, then a chevron). The Signing group has **Debug key** (green "Ready") and **Release key** (amber "Not set", or green "Ready" when a key exists). Tapping App icon opens screen 46. Tapping either key row opens screen 47 (or 48 when a release key already exists).

46 App icon

What it is: the place where the user gives the app its launcher icon. The user does not choose from ready-made icon types. The user picks **their own picture** and the app makes every size the project needs.

How it opens: a full screen, header "Project settings" (bold) and "App icon" (muted). At the top a card shows the **current icon of this project** (read from the project's `res/mipmap` folder; if the project has none, show the default Android icon and the text "No icon yet") and three small previews of the same icon as a circle, a rounded square and a square, because phones crop icons differently.

What the user can do: 1. Tap **Choose an image**. The Android file picker opens (`ActivityResultContracts.PickVisualMedia` for the gallery or `OpenDocument` for Files). The user picks a square PNG or JPG. 1024 by 1024 is best; 512 by 512 is the smallest. If it is smaller or not square the screen shows a red line with the real reason and does not apply it. (A non-square picture can be cropped to the centre square.) 2. Choose a **background colour** from the six project colours. It fills behind a picture that has transparent edges. 3. The three preview icons update at once. 4. Tap **Apply to project**. The app writes the icon files into the project (see code). The row in Project settings shows the new icon. Tap **Reset** to go back to what the project had.

Where the files go: `res/mipmap-mdpi` (48 px), `hdpi` (72), `xhdpi` (96), `xxhdpi` (144), `xxxhdpi` (192): `ic_launcher.png` and `ic_launcher_round.png` in each. For adaptive icons also `res/mipmap-anydpi-v26/ic_launcher.xml` with the foreground picture and `res/values/ic_launcher_background.xml` with the colour.

```
suspend fun applyIcon(project: Project, source: Uri, background: Int, ctx: Context) = withContext(Dispatchers.IO) {
    val bmp = ctx.contentResolver.openInputStream(source).use { BitmapFactory.decodeStream(it) }
        ?: error("This file is not a readable picture")
    require(minOf(bmp.width, bmp.height) >= 512) { "The picture is too small. Use at least 512 by 512." }
    val side = minOf(bmp.width, bmp.height)
    val square = Bitmap.createBitmap(bmp, (bmp.width - side) / 2, (bmp.height - side) / 2, side, side)
    val sizes = mapOf("mdpi" to 48, "hdpi" to 72, "xhdpi" to 96, "xxhdpi" to 144, "xxxhdpi" to 192)
    sizes.forEach { (density, px) ->
        val dir = File(project.dir, "app/src/main/res/mipmap-$density").apply { mkdirs() }
        val scaled = Bitmap.createScaledBitmap(square, px, px, true)
        File(dir, "ic_launcher.png").outputStream().use { scaled.compress(Bitmap.CompressFormat.PNG, 100, it) }
        File(dir, "ic_launcher_round.png").outputStream().use { roundOf(scaled).compress(Bitmap.CompressFormat.PNG, 100, i
t) }
    }
    File(project.dir, "app/src/main/res/values/ic_launcher_background.xml").writeText(
        """<resources><color name="ic_launcher_background">#%06X</color></resources>""".format(background and 0xFFFFFF))
}
```

Empty and error states: no icon yet (default icon and text), picture too small or unreadable (red line, nothing is written), write failed (real error with Retry).

47 Signing keys (Debug key and Release key)

First the idea, in plain words, because it is easy to mix up:

	Debug key	Release key
What it is	A key the app **makes by itself** the first time it builds	The **real key of the app**, made by the user (or imported)
Used for	Installing test builds on the user's own phone	Publishing to Google Play and every later update
Can publish to Play	**No**	**Yes**
Who creates it	Nobody, it is automatic	The user, once. The app can generate it for the user (button "Create a new key") or the user can import one made before
If lost	Nothing happens, a new one is made	The app can never be updated on Google Play under the same listing. **Back it up**

How it opens: two cards. The Debug key card says "Made by the app, automatically", shows a green Ready pill and its fingerprint, and has no buttons. The Release key card says "Needed to publish" and has an amber "Not set" pill with two buttons: **Create a new key** and **Import a key file**. Under the cards a small text says keys stay on this phone in the app's private storage and are never uploaded.

Can the key be imported? Yes. Android signing keys are normal files: `.jks`, `.keystore` or `.p12` (PKCS12). Many developers already have one. **Can the app make it automatically?** Yes. **Create a new key** makes the key on the phone with a strong random password and keeps it in the app's private storage. The user can still change the password. Either way, the app tells the user to back it up.

Where it is kept: the key file is copied into the app's private folder (`filesDir/keys/<project id>/release.jks`). Passwords are not saved as plain text. They are encrypted with a key held by the Android Keystore system (`EncryptedSharedPreferences` or Tink) and only when "Remember the password on this phone" is on. Never write the password to a log.

48 Release key (ready)

Opens when a release key exists. Shows the real alias, the date it is valid until, the owner (CN), the SHA-256 fingerprint and the key file name. Buttons: **Back up** (opens the Android share sheet or save dialog with the `.jks` file so the user can store it safely), **Replace** (asks to confirm, then goes to Create or Import), **Remove from this phone** (red, confirm dialog that repeats the warning that the key cannot be recovered unless it was backed up). An amber warning card keeps telling the user to back it up.

49 Create a key

Fields: Key alias (default `release`), Your name, Country code (two letters), Valid for (25, 30 or 50 years, 25 is selected), Password (made by the app, can be changed), toggle "Remember the password on this phone" (on by default). **Create key** is disabled until the alias, name and country code are valid and the password has at least 6 characters. While it works it shows a progress line. On success it opens screen 48. On failure it shows the real error.

```
suspend fun createReleaseKey(p: Project, alias: String, name: String, country: String, years: Int, password: CharArray): Ke
yInfo =
    withContext(Dispatchers.IO) {
        val file = File(ctx.filesDir, "keys/${p.id}/release.jks").apply { parentFile!!.mkdirs() }
        // uses the keytool of the build tools the app already has
        val cmd = listOf(keytool, "-genkeypair", "-v", "-keystore", file.path, "-alias", alias,
            "-keyalg", "RSA", "-keysize", "2048", "-validity", (years * 365).toString(),
            "-storepass", String(password), "-keypass", String(password),
            "-dname", "CN=$name, C=$country")
```

```
        val r = runProcess(cmd)
        if (r.exitCode != 0) { file.delete(); error(r.stderr.lastLine()) }
        secrets.save(p.id, alias, password)          // only if the toggle is on
        readKeyInfo(file, password, alias)
    }
```

50 Import a key

Step 1: **Choose key file** opens the file picker (`OpenDocument` with `.jks`, `.keystore`, `.p12`). The picked name and size show in a card. Step 2: **Store password**. When the user types it and leaves the field, the app tries to open the file and lists the keys inside (alias and fingerprint) as radio rows. A wrong password shows "Wrong password" in red under the field. Step 3: **Key password** (optional, empty means the same as the store password). **Import key** copies the file into the private folder and opens screen 48.

```
fun openKeystore(file: File, storePass: CharArray): KeyStore {
    for (type in listOf("PKCS12", "JKS")) {
        try { return KeyStore.getInstance(type).apply { file.inputStream().use { load(it, storePass) } } } catch (_: Except
ion) {}
    }
    error("Wrong password or not a key file")
}
fun aliasesOf(ks: KeyStore) = ks.aliases().toList().filter { ks.isKeyEntry(it) }
```

51 Self-test

What it is: a quick health check of the app on this phone, found in Settings, Developer, Self-test. It exists so the user (or the developer) can see why a build or a feature fails without reading logs.

How it opens: a full screen, header "Developer" (bold) and "Self-test" (muted). It runs the checks at once. A summary card at the top says "15 passed · 1 to check" and the time it took, with a **Run again** button. Under it is the list, one row per check, each with a green tick, an amber triangle (needs attention but not fatal, for example "Shell tools: not installed, optional") or a red cross (fails), a short title and **one sentence** that says the result in plain words. At the bottom **Share report** shares the results as text (version, self-test and the last 200 log lines).

The 16 checks are the real ones the app already has. The rows in the picture are examples. Do not add or invent checks. Each check returns `Pass`, `Warn(message)` or `Fail(message)`; the list shows them in the order they ran.

```
sealed interface CheckResult { data object Pass : CheckResult; data class Warn(val msg: String) : CheckResult; data class F
ail(val msg: String) : CheckResult }
data class Check(val title: String, val passText: String, val run: suspend () -> CheckResult)
```

2.8 Icon manager (NEW FEATURE, screens 52 to 57)

This is a new feature. It is not in the app today. Build it only after the restyle is finished, and do not let it change anything that already works. It follows the same rules as everything else: build exactly what the pictures show and invent nothing.

What it is for: to manage the icons of the open project in one place: the app logo (screen 46), the project's own icons, and ready-made icon libraries the user can download and use without internet.

How it opens: Project settings has a new row **Icon manager** (App group, under App icon). It opens a full screen with the header "Project settings" (bold) and "Icon manager" (muted) and three tabs: **Project**, **Libraries**, **Compose**. Back goes to Project settings. The tab row is 44 tall: each tab is equal width, text 14 sp / 600, selected tab text in accent with a 2 dp accent line under it, a 1 dp divider under the row. Below the tabs is a search pill (42 tall, radius 21) on every tab.

52 and 53 Project tab

- **App icon card** at the top: the project's real launcher icon on the left (48 by 48, radius 15), title "App icon", line "Preview and replace the launcher icon", chevron. Tap it: screen 46 (the app logo screen). If the project has no icon, the tile is grey with an image icon and the text reads "No icon yet. Choose your logo."
- **Import an SVG** button (40 tall pill, centred): opens the file picker for `.svg`. The app converts the SVG to an Android vector drawable and saves it in `res/drawable`. If the SVG cannot be converted (for example it uses unsupported features) show the real reason in a red line.
- **In this project · N:** a 3-column grid of the project's real drawable and mipmap files (icon, name). The count N is the real count. Tap a tile: a sheet to rename, copy as Compose code, or delete (delete asks to confirm). Screen 53 is the empty state: "No icons in this project. Import an SVG or add one from a library."

54 Libraries tab, before and during download

The user has not downloaded any pack yet. A short line says a pack is downloaded once and then works without internet. One card per pack the app offers, each with name, number of icons, download size and a **Download** button. While downloading the card shows a progress bar with the real percent and size, and the button becomes **Cancel**. When it finishes the card is replaced by the icon grid (screen 55). If the download fails, show the real message and a Retry button.

Packs the app offers come from one small index file that the app downloads (`packs.json`: id, name, icon count, size, url). Do not type the pack list into the app. Example entries: Material Symbols (300 popular icons) and Material Symbols (all, 4,304 icons). Material Symbols is free to use (Apache 2.0 licence); keep the licence text in About.

55 Libraries tab, after download

Search field, then one chip per downloaded pack (selected chip is accent, shows the real count). Under it a 3-column grid of tiles: icon 26 dp and its name 12 sp, tile radius 18, fill field. Search filters by name as the user types (after 150 ms). The grid is a `LazyVerticalGrid` so 4,304 icons stay smooth. Tap a tile: screen 56.

56 Add to project (bottom sheet)

Shows the icon large with its name and pack, then **Colour** (six swatches, selected has an accent outline), **Size** (24, 32 or 48 dp chips), **File name** (default `ic_<name>`, must be lower case letters, numbers and underscores; shows a red line if the name already exists), and two buttons: **Add to project** (writes `res/drawable/ic_<name>.xml`, closes the sheet, shows a toast "Added", and the project's icon count goes up) and **Copy as Compose code** (copies an `ImageVector` and shows a toast "Copied").

57 Compose tab

Lists the icons the user added, each with a **Copy** button and a one-line example of how to use it in Jetpack Compose. Empty state: "Icons you add appear here as Compose code."

```
// ---- data
data class IconPack(val id: String, val name: String, val count: Int, val bytes: Long, val url: String)
data class LibIcon(val name: String, val pathData: String)        // 24 x 24, filled path

// ---- download once, then keep on the phone
suspend fun downloadPack(pack: IconPack, onProgress: (Float) -> Unit): File = withContext(Dispatchers.IO) {
    val out = File(ctx.filesDir, "iconpacks/${pack.id}.json").apply { parentFile!!.mkdirs() }
    val tmp = File(out.path + ".part")
    http.newCall(Request.Builder().url(pack.url).build()).execute().use { r ->
        check(r.isSuccessful) { "Download failed (${r.code})" }
        val total = r.body!!.contentLength().takeIf { it > 0 } ?: pack.bytes
        r.body!!.byteStream().use { input -> tmp.outputStream().use { o ->
            val buf = ByteArray(16 * 1024); var done = 0L
            while (true) { val n = input.read(buf); if (n < 0) break; o.write(buf, 0, n); done += n
                onProgress((done.toFloat() / total).coerceIn(0f, 1f)); ensureActive() } } }
    }
    tmp.renameTo(out); out
}

// ---- SVG path data -> Android vector drawable file
fun toVectorXml(icon: LibIcon, colorHex: String, sizeDp: Int) = """<vector xmlns:android="http://schemas.android.com/apk/re
s/android"
    android:width="${sizeDp}dp" android:height="${sizeDp}dp" android:viewportWidth="24" android:viewportHeight="24">
    <path android:fillColor="$colorHex" android:pathData="${icon.pathData}"/>
</vector>"""

suspend fun addToProject(p: Project, icon: LibIcon, fileName: String, colorHex: String, sizeDp: Int) = withContext(Dispatch
ers.IO) {
    require(Regex("[a-z][a-z0-9_]*").matches(fileName)) { "Use lower case letters, numbers and underscores" }
    val f = File(p.dir, "app/src/main/res/drawable/$fileName.xml")
    require(!f.exists()) { "A file with this name already exists" }
    f.parentFile!!.mkdirs(); f.writeText(toVectorXml(icon, colorHex, sizeDp))
}

// ---- Compose code for the same icon
fun toComposeCode(icon: LibIcon, name: String, sizeDp: Int) = """val AppIcons.$name: ImageVector by lazy {
    ImageVector.Builder("$name", ${sizeDp}.dp, ${sizeDp}.dp, 24f, 24f).addPath(
        pathData = PathParser().parsePathString("${icon.pathData}").toNodes(),
        fill = SolidColor(Color.Black)).build()
}"""

// ---- the grid
@Composable
fun IconGrid(icons: List<LibIcon>, onTap: (LibIcon) -> Unit) {
    val c = CcTheme.colors
    LazyVerticalGrid(GridCells.Fixed(3), contentPadding = PaddingValues(16.dp),
        horizontalArrangement = Arrangement.spacedBy(8.dp), verticalArrangement = Arrangement.spacedBy(8.dp)) {
        items(icons, key = { it.name }) { i ->
            Column(Modifier.clip(RoundedCornerShape(18.dp)).background(c.field).clickable { onTap(i) }.padding(vertical = 1
4.dp),
                horizontalAlignment = Alignment.CenterHorizontally) {
                Icon(rememberVector(i.pathData), null, tint = c.text2, modifier = Modifier.size(26.dp))
                Spacer(Modifier.height(9.dp))
                OneLine(i.name.replace('_', ' '), CcType.caption, c.textSoft, Modifier.padding(horizontal = 6.dp))
            }
        }
    }
}
```

Tab bar code: use `TabRow` with `containerColor = c.content`, `contentColor = c.accent`, a 2 dp `indicator` in accent, and `divider` 1 dp in `c.divider`. Unselected tab text is `c.muted`.

Rules for this feature: nothing is typed in (icon names, counts, sizes and project icons all come from the real pack index, the real downloaded file and the real project folder); works offline after the first download; never block the screen while a download runs (the user can leave and come back, the download continues in a foreground service or WorkManager); show the real error text if anything fails.

2.6 Build order for the developer

1. Add fonts. Add `CcTheme.kt`. Add `CcIcons.kt`. Check an icon renders.
2. Add the components in 2.3 and look at them on one test screen.
3. Add `AppNav`, `SidebarHost`, `ToolRail`, `SidePanel` and wire them in `MainScreen`. Check swipe, close, Back, double Back to exit.
4. Restyle the existing screens one by one, replacing old UI with the components. Do not touch their logic.
5. Add the empty, loading and error states to every list.
6. Test with zero projects, one project and many projects. Test the keyboard. Test large font size.

PART 3. THE VISUAL SPEC

Read this first

CloseCode is a **working app** with many features already built. This document changes how it looks and how screens connect. It must not remove, break or replace any feature that already works.

Scope

Follow the mockups exactly. Build exactly what the mockup pictures and the numbers in this document show: the same spacing, sizes, icons, colours, corners, order of items and texts. **Do not invent anything that is not described here.** Do not add, remove, move, resize, recolour or rename anything because it seems better. Do not use a library default look (Material default colours, shapes, icon sets, fonts, elevations) where this document gives a value. If something is missing or two things seem to disagree, keep the existing behaviour of the app, copy the closest mockup, and ask. Before you call a screen finished, put your screenshot next to the mockup picture and compare spacing, icons and text one by one. Everything in the pictures that is a name, number or list is an example and must come from real data.

- **OpenCode must not be included in this development.** Leave everything about it exactly as it is today.
- Do not add features that are not already in the app. Only restyle what exists and add the navigation rules below.

Real data only (no fake content)

The pictures use example names so the layout can be seen. **Every name, number, date, list and state in a picture is an example.** Nothing in the app may be hard-coded to match a picture. Read every value from where the app already gets it:

Shown in a picture	Must come from
Project names, letters, colours, types, last-opened times	The user's real projects on this phone (created, cloned or imported by the user). The letter is the first letter of the real name. Tile colour is stored per project or derived from the name. Empty list when there are none

Shown in a picture	Must come from
Recent projects and project counts	Real project list sorted by last opened
File tree, open files, tabs, file icons	The real project folder and real open-file state. Icon by real extension
Code, line numbers, cursor	The real file content in the editor
Git marks (M, A, D), counts, branch, commits, authors, hashes, times, diffs	The real repository (git status, log, diff)
GitHub account, repositories	The real signed-in account. Not signed in means the not-connected screen
Problems, errors, warnings	Real results of the project's checks and build
Build steps, percentages, sizes, times, file names	The real running build
Logcat and Console lines	The real log stream
Terminal output, sessions, saved commands	Real shells and the user's saved commands
Versions, build number, storage sizes, download progress	Real values from the app and device
Settings values and toggles	The real saved settings (toggle shows what is saved and saves on change)
Package name, SDK levels, version name and code	The real project configuration
Templates	The real list of templates the app ships with

If a value does not exist yet, show the empty state (see States), never an example. No placeholder projects, no demo files, no sample repositories, no fake commits.

Not every screen is shown here

The app already has more screens and features than the pictures cover. **The pictures are a design system, not a full screen list.** Any screen or feature that is not shown must be restyled with the same design: same colours, type, spacing, shapes, icons, components and navigation rules from this document. See "Screens that are not shown" for the recipe. Do not leave an old-style screen next to new-style screens, and do not remove a screen because it has no picture.

How to read the numbers

All sizes are in dp (px in the pictures equal dp) on a 360 dp wide phone. Text is in sp. Where a picture and a number here differ, the number wins. Scale spacing together on wider phones.

Design numbers

Define every value once as a theme token and use the token everywhere. No screen may contain a loose hex colour, font size or corner radius.

Colours (light theme)

Token	Hex	Used for
rail	#E3E5EA	Left tool rail, status bar area, outer background behind the panel
panel	#F1F2F5	Side panel, bottom sheets, popup menus
content	#FAFAFB	Editor and every full screen background. Never pure white
field	#E9EBEF	Text fields, grouped settings cards, unselected chips, key row keys, back button, result cards
selectedRow	#DCDFE5	Selected row in lists
selectedFile	#DDE0FB	Selected file in the tree (text #2E3036)
border	#D5D8DE	Card, menu and sheet borders (1 dp)
divider	#E3E5EA	Lines between header and content, between rows (1 dp)
toggleOff	#B8BCC5	Toggle track when off
text	#1F2024	Titles, main text
text2	#2E3036	Body text, code text
textSoft	#4F535C	Secondary button text, unselected chip text
muted	#626772	Descriptions, sub lines, section labels
faint	#858A95	Line numbers, hints, disabled icons
accent	#5865F2	Primary buttons, selected rail item, selected chip, links, toggle on, progress, focus ring
green	#248046	Run button, success, added lines, "Ready"
amber	#C98A0A	Modified, warnings
red	#DA373C	Errors, delete, badges, "Not installed"
blue	#2F6FE0	Info and config file icons
purple	#7B5CE0	Code file icons
yellow	#D99A06	Folder icons
teal	#1F9D6B	Success icons
orange	#D97706	Warning triangle
pink	#DB4C9B	Image icon
diffAddBg / diffAddText	#DDF3E4 / #1A7F37	Added lines
diffDelBg / diffDelText	#FBE4E5 / #DA373C	Removed lines, error rows
matchBg / matchText	#FBE7A1 / #6B4E00	Search match highlight
dim	rgba(30,35,50,0.30)	Layer behind sheets and menus (0.38 behind first-time dialogs, 0.18 behind small menus)
shadow	rgba(30,35,50,0.25), blur 30, y 12	Popup menus and the sliding editor card

Icon tile tint: the tile behind a coloured icon is the icon colour at 13% opacity (hex alpha 22).

Syntax colours (light): keyword #8250DF, type #1B7C83, string #A5461D, function #6F5A00, number #0A7D4F, comment #5E7F46, variable #0A5EA8, plain #2E3036. Apply only to real code tokens of the file's language. Plain text and Markdown stay plain.

Project tile colours: each project has its own colour (saved with the project or derived from its name from this set: #2563EB, #16A34A, #EA580C, #0E7490, #7C3AED, #BE185D). Letter on the tile is white, weight 800.

Type

UI font **Inter**. Code and terminal font **JetBrains Mono**. Line height 1.4 of the size unless noted.

Style	Size / weight	Used for
Hero	33 sp / 800, tracking -1	Welcome headline
Setup title	27 sp / 800	First-time setup title
Panel title	21 to 22 sp / 800	Title at the top of a side panel
Sheet title	19 to 20 sp / 800	Bottom sheet titles
Screen title	16.5 sp / 700	Header title of full screens
Item title	14.5 sp / 600	List and settings row titles
Body	14 sp / 400 to 500	Buttons in menus, descriptions
Small	13 to 13.5 sp	Field text, tree rows
Caption	12 to 12.5 sp	Sub lines, labels above fields
Section label	11 sp / 700, uppercase, tracking 0.8	Group titles
Code	11.5 sp mono, line height 21	Editor
Terminal and logs	10.5 to 12 sp mono	Terminal, Logcat, Console

Spacing and shape

- Screen side padding 14 to 18. Row padding 8 by 14. Gap between related items 8 to 12.
- Radius: tiles 13 to 15, buttons 12 to 16, cards 18 to 20, grouped settings cards 18, chips and pill fields 15 to 22, sheets 26 (top corners), panel 26 (top-left), phone preview 30.
- Borders 1 dp. Selected outline 2 dp accent. Focus ring 1.5 dp accent.
- Status bar area 34 dp. Header 56 dp. Home indicator area 16 dp.

Motion

- Sidebar swipe: the editor card follows the finger, rests 300 dp to the right, spring (damping 0.85, stiffness 400). Release past 40% opens; otherwise it snaps back.
- Sheets slide up in 250 ms ease-out. Menus fade and scale from 0.95 in 120 ms.
- Screen push slides in from the right in 220 ms; back reverses it.
- Toggle knob 160 ms. Progress bars animate 200 ms between values.
- Respect the system setting for reduced animation.

Icons

All icons are simple line icons on a **24 by 24 grid**, drawn as vectors (SVG path or Compose ImageVector) with stroke width 1.8 (2.2 for chevrons and back, 2.4 for ticks), round caps, round joins, no fill (only Play is filled). Colour is the token named in the table. Use these everywhere in the app. If the app uses an icon that is not in this list, draw a new one in the same style (same grid, same stroke, rounded) and add it to your icon set.

Icon sizes

Place	Size	Stroke
Rail tool icon	22	1.8
Back button chevron	20	2.2
Header action (undo, redo, share, clear)	17 to 19	1.9
Panel action button (plus, folder, refresh)	17 to 18	1.9
List and settings tile	18 (settings) / 21 (panel rows)	1.8
Button icon	15 to 17	2.0
Run button play	17 filled	none
Tree row icon	17 to 18	1.7
Chevron in rows	15 to 17	2.2
Tick in lists	18	2.4
Empty-state and hero icon	36 to 38	1.6
Badge glyph	10 to 12	3

Touch target for any icon button is at least 38 by 38 dp (44 preferred).

Icon path data (24 by 24)

Name	Path
folder	`M4 20h16a2 2 0 0 0 2-2V8a2 2 0 0 0-2-2h-7.9a2 2 0 0 1-1.69-.9L9.6 3.9A2 2 0 0 0 7.93 3H4a2 2 0 0 0-2 2v13c0 1.1.9 2 2 2Z`
refresh	`M3 12a9 9 0 0 1 9-9 9.75 9.75 0 0 1 6.74 2.74L21 8M21 3v5h-5M21 12a9 9 0 0 1-9 9 9.75 9.75 0 0 1-6.74-2.74L3 16M8 16H3v5`
branch	`M6 3v12M18 9a3 3 0 1 0 0-6 3 3 0 0 0 0 6zM6 21a3 3 0 1 0 0-6 3 3 0 0 0 0 6zM18 9a9 9 0 0 1-9 9`
history	`M3 12a9 9 0 1 0 3-6.7L3 8M3 3v5h5M12 7v5l4 2`
down	`M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4M7 10l5 5 5-5M12 15V3`
up	`M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4M17 8l-5-5-5 5M12 3v12`
more	`M12 5v.01M12 12v.01M12 19v.01`
plus	`M12 5v14M5 12h14`
check	`M20 6L9 17l-5-5`

Name	Path
cd	`M6 9l6 6 6-6`
cl	`M15 18l-6-6 6-6`
cr	`M9 18l6-6-6-6`
search	`M11 19a8 8 0 1 0 0-16 8 8 0 0 0 0 0 16zM21 21l-4.3-4.3`
filter	`M3 6h18M7 12h10M10 18h4`
copy	`M9 9h11v11H9zM5 15H4V4h11v1`
file	`M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8zM14 2v6h6`
undo	`M9 14L4 9l5-5M4 9h10a6 6 0 0 1 0 12h-3`
redo	`M15 14l5-5-5-5M20 9H10a6 6 0 0 0 0 12h3`
save	`M19 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h11l5 5v11a2 2 0 0 1-2 2zM17 21v-8H7v8M7 3v5h8`
term	`M4 17l6-6-6-6M12 19h8`
menu	`M4 6h16M4 12h16M4 18h16`
x	`M18 6L6 18M6 6l12 12`
bolt	`M13 2L3 14h9l-1 8 10-12h-9z`
kbd	`M4 5h16a2 2 0 0 1 2 2v10a2 2 0 0 1-2 2H4a2 2 0 0 1-2-2V7a2 2 0 0 1 2-2zM6 9h.01M10 9h.01M14 9h.01M18 9h.01M7 15h10`
code	`M16 18l6-6-6-6M8 6l-6 6 6 6`
layers	`M12 2l10 5-10 5L2 7zM2 17l10 5 10-5M2 12l10 5 10-5`
image	`M5 3h14a2 2 0 0 1 2 2v14a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2zM9 8a1 1 0 1 0 0 .01M21 15l-5 -5L5 21`
trash	`M3 6h18M8 6V4h8v2M19 6l-1 14H6L5 6`
share	`M4 12v8h16v-8M16 6l-4-4-4 4M12 2v13`
bug	`M8 2l1.9 1.9M14.1 3.9L16 2M9 7.1V6a3 3 0 0 1 6 0v1.1M12 20a6 6 0 0 1-6-6v-3a4 4 0 0 1 4-4h4a4 4 0 0 1 4 4v3a6 6 0 0 1-6 6zM12 20v-9M6.5 13H2M17.5 13H22`
flask	`M10 2v7.5L4 20a1 1 0 0 0 .9 1.5h14.2A1 1 0 0 0 20 20l-6-10.5V2M8.5 2h7`
alert	`M12 9v4M12 17h.01M10.3 3.9L1.8 18a2 2 0 0 0 1.7 3h17a2 2 0 0 0 1.7-3L13.7 3.9a2 2 0 0 0-3.4 0z`
gh	`M15 22v-4a4.8 4.8 0 0 0-1-3.5c3 0 6-2 6-5.5.08-1.25-.27-2.48-1-3.5.28-1.15.28-2.35 0-3.5 0 0-1 0-3 1.5-2.64-.5-5.36-.5-8 0C6 2 5 2 5 2c-.3 1.15-.3 2.35 0 3.5A5.403 5.403 0 0 0 4 9c0 3.5 3 5.5 6 5.5-.39.49-.68 1.05-.85 1.65-.17.6-.22 1.23-.15 1.85v4M9 18c-4.51 2-5-2-7-2`
send	`M22 2L11 13M22 2l-7 20-4-9-9-4z`
cog	`M12 15a3 3 0 1 0 0-6 3 3 0 0 0 0 6zM19.4 15a1.65 1.65 0 0 0 .33 1.82l.06.06a2 2 0 1 1-2.83 2.83l-.06-.06a1.65 1.65 0 0 0-1.82-.33 1.65 1.65 0 0 0-1 1.51V21a2 2 0 1 1-4 0v-.09A1.65 1.65 0 0 0 9 19.4a1.65 1.65 0 0 0-1.82.33l-.06.06a2 2 0 1 1-2.83-2.83l.06-.06A1.65 1.65 0 0 0 4.68 15a1.65 1.65 0 0 0-1.51-1H3a2 2 0 1 1 0-4h.09A1.65 1.65 0 0 0 4.6 9a1.65 1.65 0 0 0-.33-1.82l-.06-.06a2 2 0 1 1 2.83-2.83l.06.06A1.65 1.65 0 0 0 9 4.68a1.65 1.65 0 0 0 1-1.51V3a2 2 0 1 1 4 0v.09a1.65 1.65 0 0 0 1 1.51 1.65 1.65 0 0 0 1.82-.33l.06-.06a2 2 0 1 1 2.83 2.83l-.06.06A1.65 1.65 0 0 0 19.4 9a1.65 1.65 0 0 0 1.51 1H21a2 2 0 1 1 0 4h-.09a1.65 1.65 0 0 0-1.51 1z`
ext	`M18 13v6a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V8a2 2 0 0 1 2-2h6M15 3h6v6M10 14L21 3`
play	`M7 4.5v15l13-7.5z`
user	`M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2M12 11a4 4 0 1 0 0-8 4 4 0 0 0 0 8z`
panel	`M5 3h14a2 2 0 0 1 2 2v14a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2zM9 3v18`
globe	`M12 22a10 10 0 1 0 0-20 10 10 0 0 0 0 0 20zM2 12h20M12 2a15 15 0 0 1 0 20 15 15 0 0 1 0-20z`
flag	`M4 22V4M4 4h13l-2 4 2 4H4`

Name	Path
moon	`M21 12.8A9 9 0 1 1 11.2 3a7 7 0 0 0 9.8 9.8z`
sun2	`M12 17a5 5 0 1 0 0-10 5 5 0 0 0 0 10zM12 1v2M12 21v2M4.2 4.2l1.4 1.4M18.4 18.4l1.4 1.4M1 12h2M21 12h 2M4.2 19.8l1.4-1.4M18.4 5.6l1.4-1.4`
lock	`M5 11h14v10H5zM8 11V7a4 4 0 0 1 8 0v4`
info	`M12 22a10 10 0 1 0 0-20 10 10 0 0 0 0 20zM12 16v-4M12 8h.01`
db	`M12 8c4.4 0 8-1.3 8-3s-3.6-3-8-3-8 1.3-8 3 3.6 3 8 3zM4 5v14c0 1.7 3.6 3 8 3s8-1.3 8-3V5M4 12c0 1.7 3.6 3 8 3s8-1.3 8-3`

Tool colours in the rail (unselected): Files yellow, Search blue, GitHub text2, Terminal blue, Logcat green, Problems orange, Settings textSoft. Selected tool: white icon on an accent tile.

File icons are chosen by the real file extension: code files (kt, java, ts, js, py, html, css, xml, json, gradle) use the code icon in purple; folders use the folder icon in yellow; everything else uses the file icon in blue. Use the app's existing extension mapping where it has one.

Components

Build each component once and reuse it. Sizes in dp, text in sp.

Screen frame

Part	Spec
Status bar area	34 tall, colour rail, system icons dark
Header (full screens)	56 tall, colour content, 1 dp divider below. Left padding 12 (with back button) or 18 (editor). Right padding 14. Items 10 apart
Back button	36 by 36, radius 12, fill field, chevron-left 20 stroke 2.2 colour text2. Always the first item. Touch target 44
Header title	16.5 sp bold colour text, one line, ends with ellipsis. Origin screen name is the title, current screen name follows it in 12 sp / 500 colour muted, 7 dp gap
Header actions	36 or 38 square buttons, radius 12, icon 17 to 19. Up to two
Bottom area	16 dp for the home indicator. Keyboard pushes content up; nothing is hidden behind it

Tool rail

Part	Spec
Rail	66 wide, colour rail, padding left 14, top 12. Only shown while the sidebar is open
Item	46 by 46 tile, 11 dp below the previous one. Unselected: tile colour panel, radius 23, icon 22 in its tool colour. Selected: tile colour accent, radius 15, icon white. Selected also shows a white pill on the left edge, 5 wide, 30 tall, 10 dp left of the tile, right side rounded 4. Hovered or pressed: pill 8 tall
Project avatar (top)	46 tile, radius 15, project colour, project letter 18 sp / 800 white, small chevron-down badge 16 at bottom right (circle, colour rail, white chevron 10). Below it a 30 wide, 2 tall divider line in border colour, 8 dp inset

Part	Spec
Badge	Circle 19 tall (min 19 wide), colour red, white 10 sp bold number, 3 dp ring in rail colour, top-right of the tile. Hidden at 0. Shows 99+ above 99
Settings	Last item, pinned to the bottom, 16 dp from the bottom edge

The tools are the ones the app already has, in this order: Files, Search, GitHub, Terminal, Logcat, Problems, and Settings at the bottom. If the app has another tool, add it in the same style before Settings.

Side panel

Part	Spec
Container	Fills the width right of the rail. Colour panel. **Top-left AND top-right radius 26.** Content scrolls inside
Title area	Padding 16 top, 16 left, 12 right. Title 21 sp / 800 colour text. Sub line 12 sp colour muted (for example the project name). Action buttons (for example New file and New folder): **each is a 38 by 38 rounded button, radius 12, fill field colour, with the icon 18 inside in text2.** An icon without its grey button behind it is wrong. They sit on the right, 6 apart
Filter field	Height 40, radius 12, fill field, padding 12, search icon 16 muted, hint 13.5 sp muted
Section label	11 sp / 700 uppercase, tracking 0.8, colour muted, padding 12 16 4
Run card	Floating, 10 from each side of the panel, 12 from the bottom. Height 64, radius 22, fill content, 1 dp border, padding 12. The Run button in the mockups is **icon only**: a 40 by 40 green square, radius 14, with the play icon 17 filled. Left of it: tile 36 (radius 12, project colour, letter 15 / 800 white), 8 gap, then name 14 sp / 700 and the line below 11.5 sp muted with branch icon 11 and "branch - state" (state green when ok, red when failed). Name and line are one line each and are cut with an ellipsis. Nothing may overlap. Shown on Files, Search, Terminal, Logcat and Problems panels

Lists

Part	Spec
Panel row	Min height 58, padding 8 14, gap 12. Tile 42 by 42, radius 13, tint 13% of its colour, icon 21. Title 14.5 sp / 600 colour text. Sub line 12.5 sp colour muted, one line. Right side: badge or chevron. Selected row: fill selectedRow, radius 14, side margin 8
Status dot on a tile	14 circle at bottom right, 3 dp ring in panel colour. Green running, grey idle, amber warning
Tree row	Height 38, margin 1 8, radius 11, indent 16 per level (14 in small trees), chevron 13, icon 17 to 18, name 13.5 to 14 sp, git letter at the right 11 sp mono bold (M amber, A green, D red). Selected: fill selectedFile
Settings group	Label (section label, padding 16 18 6), then card margin 0 12, radius 18, fill field, rows padding 12 14. Row: tile 36 (radius 12, tint 13%), icon 18, title 14.5 sp / 600, sub 12 sp muted, line height 1.4, right value 13 sp muted with chevron 15. Dividers 1 dp starting 62 from the left
Result card (search)	Margin 2 14 0, padding 6 9, radius 9, fill content, mono 11 sp, line number muted, match highlighted with matchBg and matchText

Buttons, fields, chips

Part	Spec
Primary button	Height 48 (52 on Welcome), radius 15 (16), fill accent, text 15 sp / 700 white, icon 17 with 8 gap. Pressed: 90% opacity
Success button	Same, fill green. Used for Run and Open app only
Secondary button	Same size, fill field, text textSoft or text
Destructive text button	Fill field, text red
Disabled button	Fill field, text muted, no press effect
Small action button	40 tall, radius 12, fill field, text 13.5 sp / 600, icon 16, 7 gap (Pull, Push, Copy code)
Text field	Height 44, radius 13, fill field, padding 14, text 12.5 sp (mono for paths, links and commands). Label above 12 sp / 600 colour textSoft, 6 dp gap. Hint below 11.5 sp muted. Focus: 1.5 dp accent ring. Error: red text line under the field
Search field	Pill, height 42, radius 21, fill field, focus ring accent
Chip	Height 30, padding 0 13, radius 15, 12 sp / 600. Selected: fill accent, text white. Unselected: fill field, text textSoft. Chips scroll sideways
Segmented control	Track fill rail, radius 13, padding 3. Segment height 34, radius 10, 13 sp / 600. Selected: fill B8BCC5 (or white with shadow), text white or text
Toggle	42 by 25, radius 13, knob 19 white. Off: track toggleOff, knob left 3. On: track accent, knob left 20
Progress bar	Height 6 to 7, radius 4, track rail, fill accent, animated. Label above: left text accent 12.5 sp / 700, right text muted
Tick box	20 by 20, radius 6. Checked: fill green with white tick 12
Badge pill ("Ready")	Height 24, padding 0 10, radius 12, fill diffAddBg, text green 12 sp / 700

Sheets and menus

Part	Spec
Bottom sheet	Top corners 26, fill panel, padding 10 16 24. Grab handle 40 by 5, radius 3, colour B8BCC5, centred, 12 below. Title 19 sp / 800. Layer behind is dim. Closes on Back, tap outside or swipe down
Popup menu	Width 200 to 220, radius 16, fill panel, 1 dp border, shadow, padding 6 0. Row height 42, padding 0 14, icon 17 + 11 gap + 14 sp / 500 text. Divider 1 dp, 4 margin. Destructive row in red
Confirm dialog	Width 300, radius 24, fill panel, padding 20. Title 17 sp / 800, text 13.5 sp, two buttons 44 tall at the bottom (Cancel secondary, action primary or red)
Toast	Pill radius 20, fill text2, white 13 sp, 12 above the bottom area, shows 2 s

Code area

Mono 11.5 sp, line height 21, line number column 34 wide (right aligned, 12 padding, faint). Current line fill field. Gutter marks for git changes: 3 wide bars, accent for modified, green for added, red for removed. Word wrap off by default; horizontal scroll moves in whole characters. Key row above the keyboard: 32 tall keys, radius 9, 6 gap, fill field, mono 12 sp / 600.

Navigation and back rules

Layout

The editor is the root screen. It fills the full width. There is no permanent sidebar and no bottom bar.

Action	Result
Swipe right anywhere on the editor (mostly horizontal, and the code cannot scroll further left). A swipe from the left 40 dp always works	The editor card slides right 300 dp and keeps a 60 dp strip visible. Its left corners are rounded, radius 26. The tool rail (66) and the side panel appear behind it. Follows the finger, settles open past 40% or on a fast fling
Tap the dimmed editor card, or swipe left	Sidebar closes
Tap a rail tool	The panel changes to that tool. The sidebar stays open
Tap a file in the Files panel	The file opens in the editor, the sidebar closes
Tap the project avatar	Project switcher sheet
Tap a tool's action button (for example Clone)	A full screen opens over the sidebar, with a back button

There is no menu button that opens the sidebar. Swipe is the only way, and it works from anywhere on the editor.

Back rules

1. Every screen that is not the editor has the back button as the first item in the header.
2. The back button and the system Back do exactly the same: go back one step, to the screen the user came from.
3. A sheet, menu or dialog closes first, before any screen goes back.
4. When the sidebar is open, Back closes the sidebar.
5. At the editor with the sidebar closed, the first Back shows the toast "Press back again to exit" for 2 seconds. A second Back inside that time leaves the app. Do not exit on the first press.
6. Unsaved text in a form is kept when going back and forth. Ask before discarding only when the user has typed something that cannot be rebuilt.
7. Keep the back stack as a real stack. Opening Settings then Developer then back returns to Settings, not to the editor.

Header shows where you came from

Every nested header shows two texts: the origin screen in bold (the screen the user came from) and the current screen in muted text after it. The origin is whatever screen opened this one, so build it from the real back stack and do not hard-code a table. Typical results:

Current screen	Origin shown in bold	Muted text
Clone a repository	GitHub	Clone a repository
Source control	GitHub	Source control
History	Source control	History
A commit	History	the short hash of that commit

Current screen	Origin shown in bold	Muted text
Shell	Terminal	the shell name
Preview	Run	Preview
Console (preview)	Preview	Console
Project settings	the project name	Project settings
Any settings page	Settings	the page name
Developer	Settings	Developer

If the origin name is long, it is cut with an ellipsis before the muted text is cut. If there is no origin (the screen was opened from outside), show only the current name in bold.

Project switcher

Bottom sheet. Lists the user's real projects (name, tile in the project colour, letter, last-opened time). The current one is marked with a check. Buttons: New project, Open folder, Clone a repository. When there are no projects the list is replaced by the empty state described in the States section.

Every screen: layout and where the data comes from

The pictures are examples. Names, files, commits and numbers in them are samples only. In the app every value comes from the user's real projects, files, git repository, device and settings. Screens are numbered like the PNG files (01 to 57; there is no screen 39).

No.	Screen	Layout	Data comes from
01	Welcome	Logo tile, app name, one line of text, buttons Start a project, Clone a repository, Open a folder (52 tall, 12 apart). Recent projects list below when any exist	Recent projects from the app's project database. No projects: only the buttons
02	Start a project	Header with back. Template cards (2 per row), name field, location row, Create button at the bottom	Templates the app really supports. Location from the real storage path chosen by the user
03	Switch project	Bottom sheet over the editor	Project database; current project marked
04	Clone a repository	Header with back. URL field, branch field (optional), destination, Clone button	User input. Sign-in state from the stored GitHub account
05	Cloning progress	Progress bar, current step text, Cancel button	Real clone progress from git. Real error if it fails
06	First-time setup	Checklist cards with progress	Real check of each tool the app needs on this device. Only tools the app really uses
07	Editor	Header (file name, tabs), code area, key row	The open file from disk. Tabs are the files the user opened
08	Editor with keyboard	Same, keyboard up, key row above it	Same
09	Swipe sidebar	Editor card slid right, rail and panel visible	Panel that was last open

No.	Screen	Layout	Data comes from
10	Files	Title "Files", project name, new file and new folder buttons, filter, tree, Run card	Real folder tree, real git status letters
11	Files menu	Popup menu on a row	Actions the app supports (New file, New folder, Rename, Duplicate, Delete, Copy path). Only supported ones
12	Search	Query field, replace toggle, results grouped by file, Run card	Real search across project files
13	Search options	Chips: match case, whole word, regex, include and exclude globs	User input
14	GitHub	Account card, repositories and actions list	Signed-in account and real repository state
15	GitHub not connected	Empty state with Sign in button	Shown when no account is stored
16	GitHub sign in	Header with back, the sign-in steps the app really uses, code field if the flow has one	Real sign-in flow
17	Source control	Branch row, Pull and Push small buttons, changes list (staged, unstaged), commit message field, Commit button	Real git status
18	Source control more	Popup menu	Real supported git actions
19	History	List of commits (hash, message, author, time)	Real git log
20	Commit detail	Header with hash, message, files changed list	Real commit
21	Diff	File name header, unified lines with the diff colours	Real diff
22	Terminal	Panel with the sessions list and New session button, Run card	Real sessions
23	Terminal session	Full screen output (mono 12), input row, key row (Tab, Ctrl, Esc, arrows)	Real shell
24	New command	Bottom sheet with command field and recent commands	Command history stored by the app
25	Run / build	Full screen with step list and progress, log below	Real build output
26	Build finished	Ready badge, time taken, Open app and Share buttons	Real build result. On failure: red badge and the real error text
27	Website preview	Header with back, address pill, page area, Console button	The real local page
28	Console	Mono log lines coloured by level	Real console messages
29	Compose previews	List of preview cards	Real preview functions found in the project. None: empty state
30	Logcat	Filter chips, search, log lines (time, level letter, tag, message)	Real device log
31	Problems	Counts by level, list grouped by file, line number per row	Real diagnostics

No.	Screen	Layout	Data comes from
32	Settings	Grouped list of categories	Fixed list of the categories the app has
33 to 44	Settings pages	Section labels plus grouped cards with rows and controls from Components	Real stored settings. Each toggle, value and path shows what is saved and writes back on change
45	Project settings	Project name, colour, icon letter, package id, SDK versions, build variants. App icon row shows the real icon. Signing rows show the real key state	Real project config files
46	App icon	Current icon card with circle, rounded and square previews, Choose an image, background colour, Reset and Apply to project	The project's real res/mipmap files and the picture the user picks
47	Signing keys	Debug key card (automatic, Ready), Release key card (Not set) with Create a new key and Import a key file	Real key files in the app's private storage
48	Release key (ready)	Alias, valid until, owner, SHA-256, key file, Back up, Replace, Remove, back-up warning	Real key file
49	Create a key	Alias, name, country, validity chips, password, remember toggle, Create key	User input
50	Import a key	Key file card, store password, key inside (radio list), key password, Import key	The picked key file
51	Self-test	Summary card with Run again, one row per real check, Share report	The app's real checks
52 to 57	Icon manager (NEW)	Tabs Project, Libraries, Compose. Project: app icon card, Import an SVG, project icons grid (52, empty state 53). Libraries: pack cards with Download (54), icon grid with pack chips (55), Add to project sheet (56). Compose: copyable code (57)	Real project resources, the downloaded icon pack files, the pack index

Settings pages: 33 General, 34 Editor, 35 Code intelligence, 36 Build and toolchain, 37 Terminal, 38 GitHub, 40 Appearance, 41 Storage, 42 Security, 43 About, 44 Developer. Show only the rows that match features the app really has; delete any row in a picture that has no matching feature, and add rows for real settings the pictures do not show.

Rules for every screen

1. Use the header, back button, rail, panel, list and button specs from Components with no changes.
2. Never fill a list with sample items. Lists show what exists. Zero items show the empty state.
3. Loading shows a progress bar or skeleton lines in the field colour, never an empty white screen.
4. Errors show the real message from the system in a red line or card, with a Retry button when retry makes sense.
5. Counts, sizes, times and names are formatted from real values. Times are relative ("2 min ago") up to one day, then a date.

Screens that are not shown

This document does not show every screen. The app already works and has more features than the pictures cover. Do not remove or hide any existing feature because it has no picture. Restyle it with this design system so it looks like part of the same app.

Method for any screen

1. Find the closest pictured screen (a list, a form, a log, a settings page, a sheet) and copy its structure.
2. Replace the header with the standard header: back button first, origin name in bold, current name muted.
3. Page background is the content colour. Cards and groups use the field colour with radius 18. Panels use the panel colour.
4. Replace every colour, font size, radius and spacing with the tokens in Design numbers. No colours outside the table.
5. Replace icons with the ones in the Icons section. If a needed icon is missing, draw a new one in the same style: 24 by 24 grid, stroke 2, round caps and joins, no fill.
6. Keep all existing logic, data and behaviour. Only the look changes.

Recipes

Kind of screen	Build it like
List of items	Panel row (tile 42, title, sub line, chevron) on the content colour
Settings or options	Settings group cards with toggles, values and chevrons
Form	Labelled text fields (44 tall), primary button at the bottom, 48 tall
Confirmation	Confirm dialog, red button for destructive actions
Choice from a few options	Bottom sheet with rows, or segmented control when 2 to 3
Long log or output	Mono 12 sp on field colour, level colours from the syntax and status tokens
Progress	Progress bar with label, Cancel button below
Permission or sign in	Centred icon tile 64, title 19 sp / 800, text 13.5 sp muted, one primary button
Info or about	Settings group with read only rows

Empty, loading and error states

State	Look
Empty	Centred tile 64 (radius 20, field colour, muted icon 28), title 16 sp / 700, one line 13 sp muted saying what appears here, and the action button that creates the first item when one exists
Loading	Progress bar 6 tall or 3 to 5 skeleton rows in field colour
Error	Red tint card (diffDelBg, radius 14, padding 12) with the real message in red, and a Retry button
Offline	Same as error with a wifi-off icon and the text that the action needs network

Existing dialogs, toasts and notifications

Restyle them with Sheets and menus. Keep their text and behaviour. Android system dialogs that the app cannot style stay as they are.

States, errors and acceptance checklist

Empty states to build

Place	Message	Action
No projects (Welcome, switcher)	No projects yet	Start a project, Clone a repository, Open a folder
Empty folder (Files)	This folder is empty	New file, New folder
No search results	No results for the query	Change options
No changes (Source control)	No changes	none
No commits (History)	No commits yet	none
GitHub not connected	Sign in to use GitHub	Sign in
No terminal sessions	No sessions	New session
No logs (Logcat, Console)	Waiting for logs	none
No problems	No problems found	none
No previews	No previews found in this project	none

Acceptance checklist

Tick each line on a real device before handing the build back.

Data

- ☐ No project, file, commit, user, count or path is typed into the code. All of them come from the real app data.
- ☐ A fresh install with no projects shows the empty states, not sample content.
- ☐ Creating, renaming and deleting a project updates every list at once.
- ☐ Settings show and save the real stored values.

Scope

- ☐ The scope statement in "Read this first" is respected, and the existing app keeps working.
- ☐ Every existing feature still works. Features without a picture are restyled using this design system and not removed.

Look

- ☐ Colours, sizes, radii, fonts and icons match the numbers in this document. Accent is blurple 5865F2, not blue.
- ☐ The tool rail is only visible while the sidebar is open, and the sidebar opens only by swipe.
- ☐ Rail items are icon only, with no text labels. The selected item has the accent tile and the white pill.

- ☐ The side panel has rounded top left corner 26 and the Run card floats at the bottom on the panels listed.
- ☐ Text never wraps in headers, buttons and rows; it is cut with an ellipsis. Buttons never overlap.
- ☐ Project sub lines show each value once (no repeated names).
- ☐ There is no Save button, More button, device list or AI button.

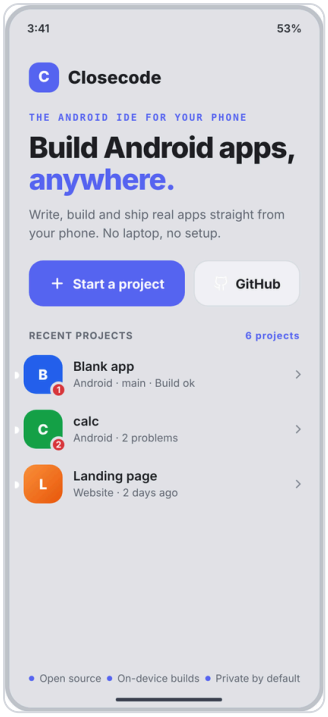
Navigation

- ☐ Every nested screen has the back button first, and it does the same as system Back.
- ☐ Headers show the origin in bold and the current screen muted, built from the real back stack.
- ☐ At the editor, Back shows the toast first and exits on the second press within 2 seconds.
- ☐ Sheets and menus close before the screen goes back.

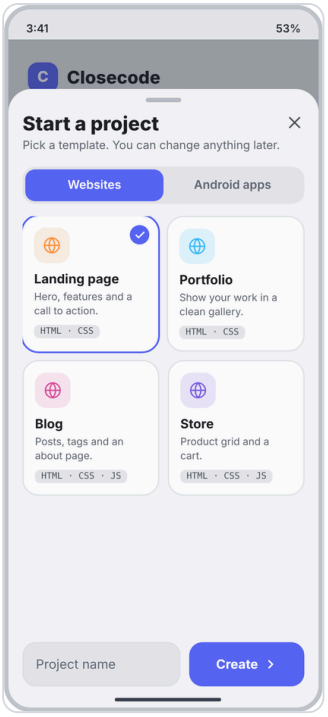
Quality

- ☐ Works on screens 360 to 430 wide and with large system font sizes.
- ☐ Keyboard never hides the focused field or the key row.
- ☐ Touch targets are at least 44 dp.
- ☐ Every error shows the real message with Retry where useful.

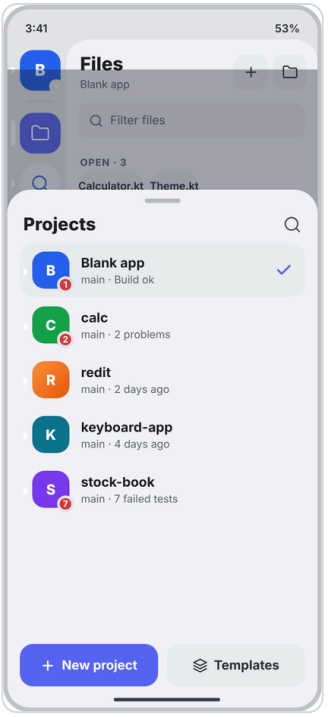
Screen pictures (examples only, all data is sample)



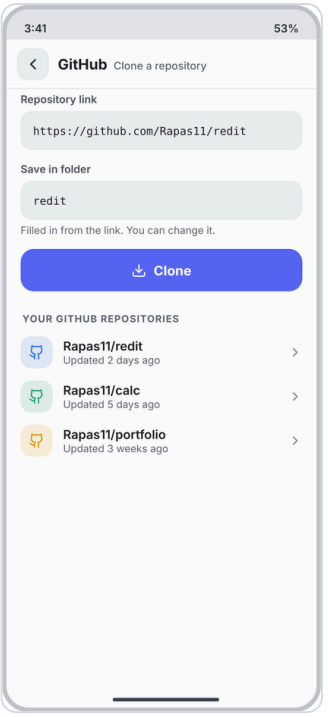
01 welcome



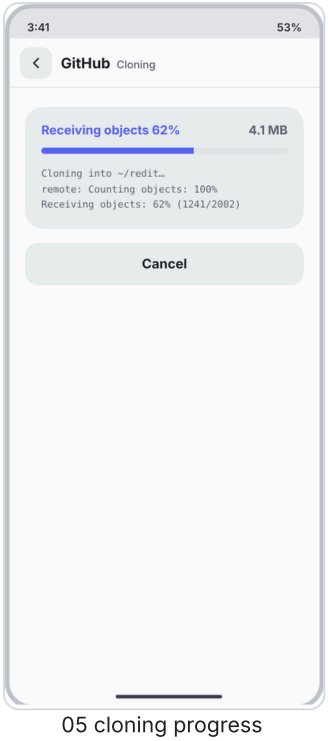
02 start project



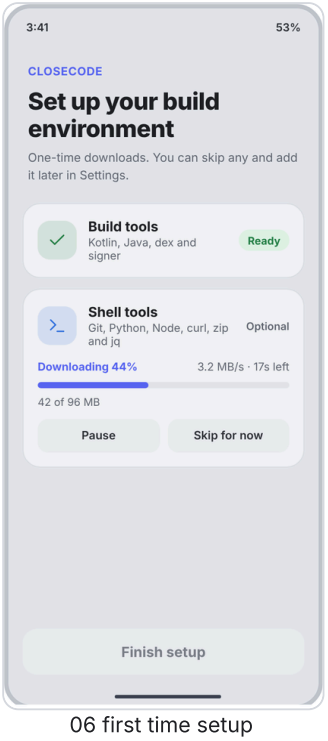
03 switch project



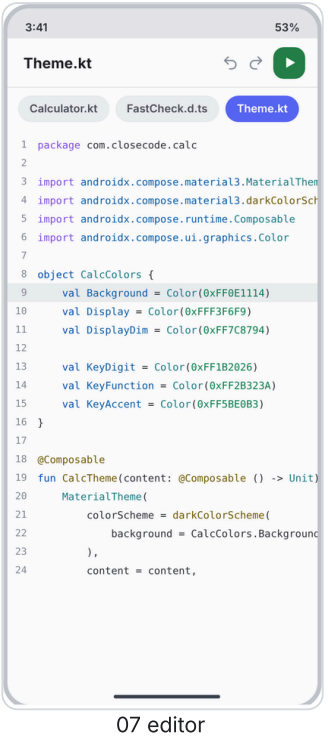
04 clone repository



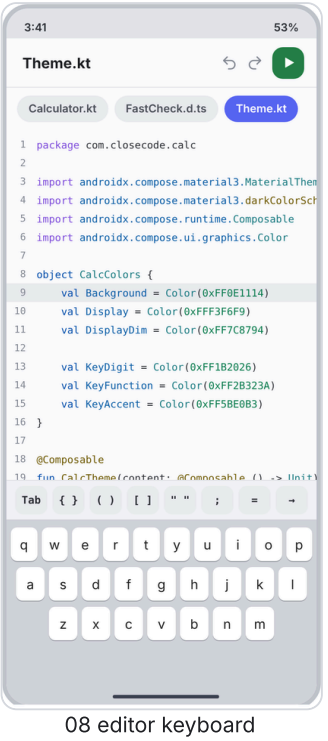
05 cloning progress



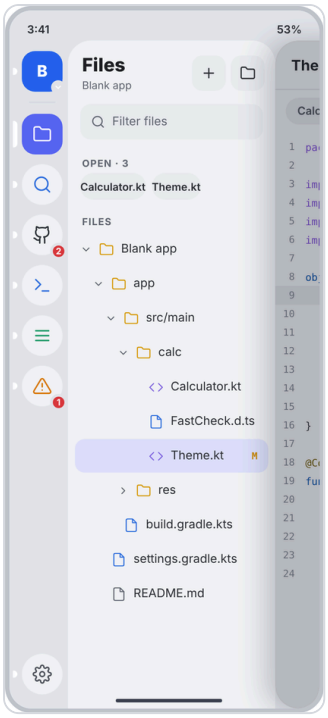
06 first time setup



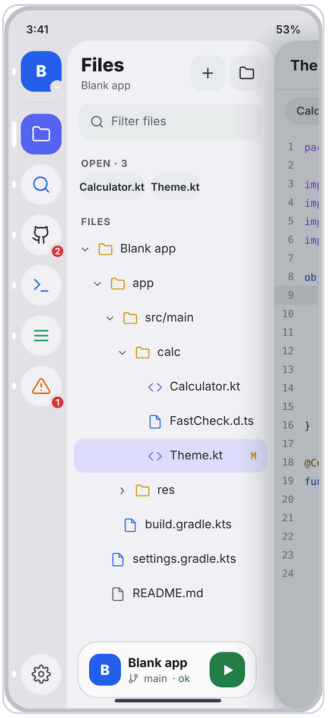
07 editor



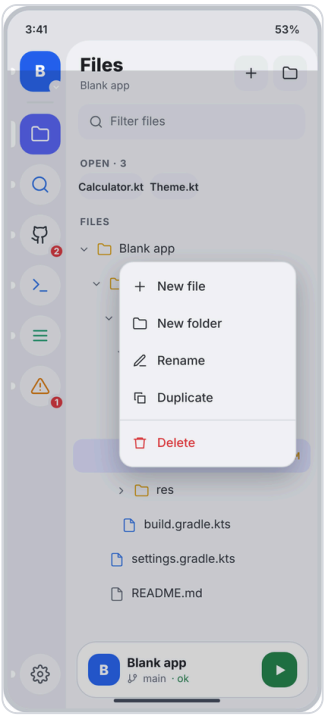
08 editor keyboard



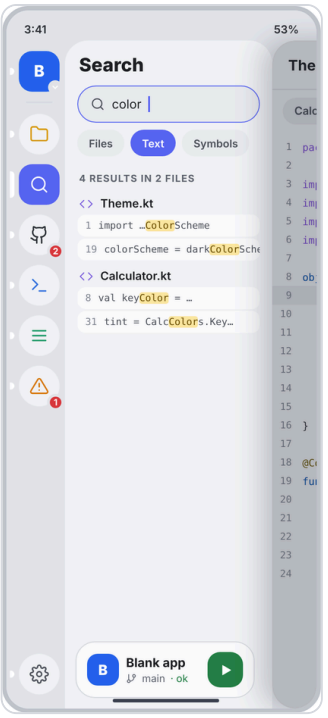
09 swipe sidebar



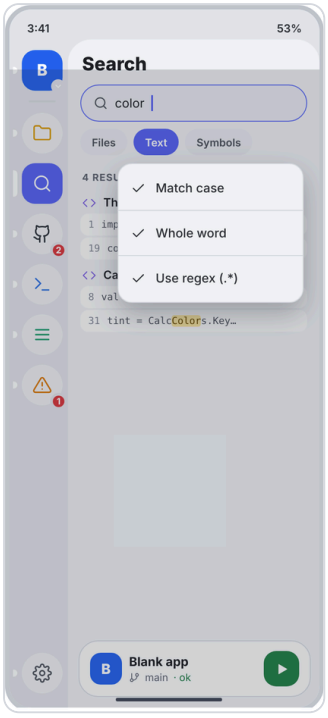
10 files



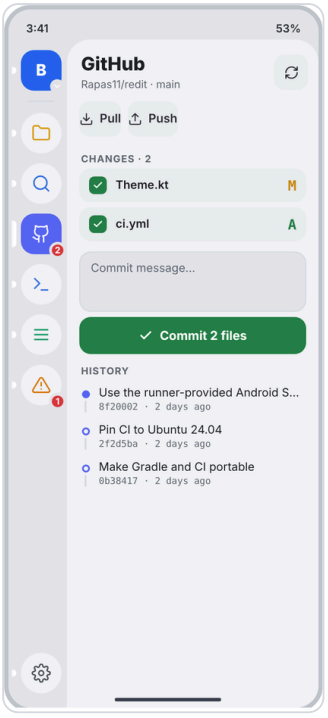
11 files menu



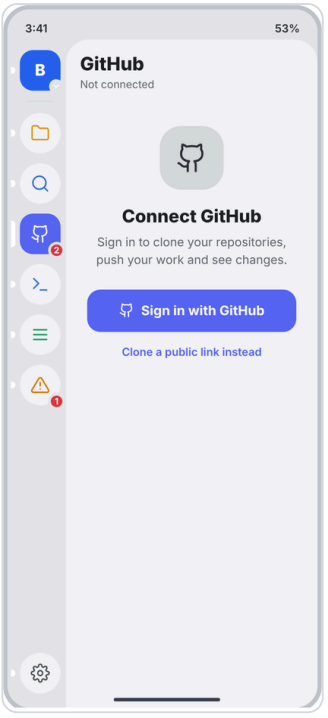
12 search



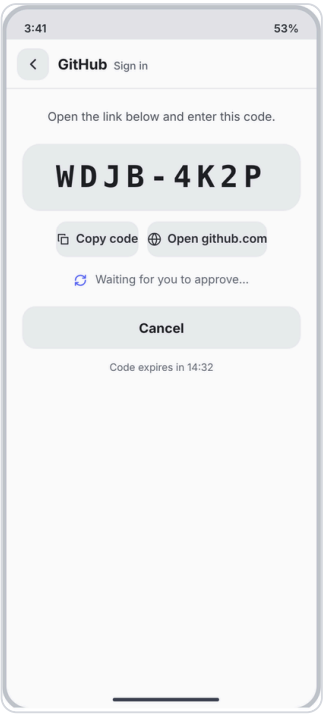
13 search options



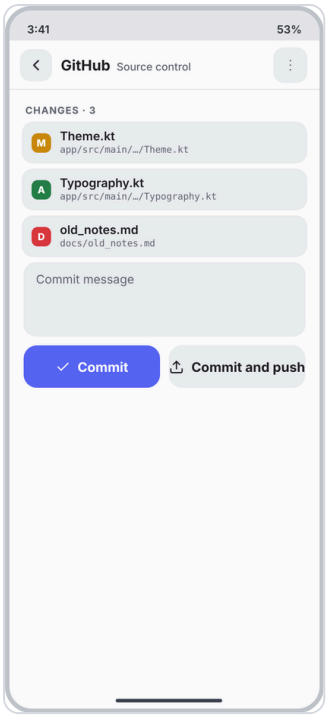
14 github



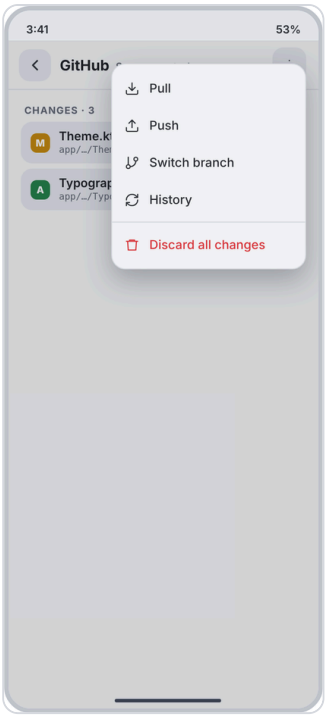
15 github not connected



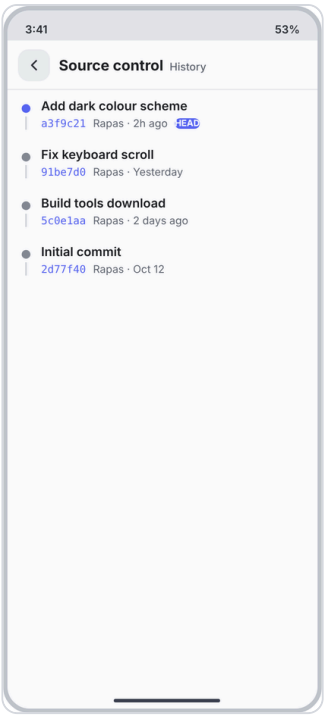
16 github sign in



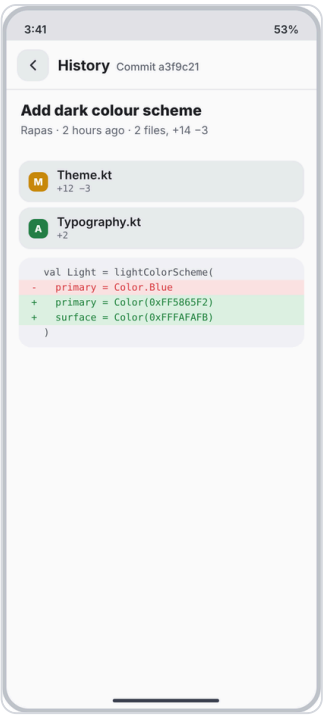
17 source control



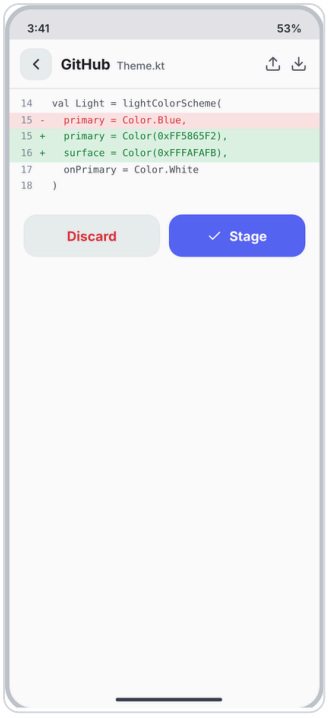
18 source control more



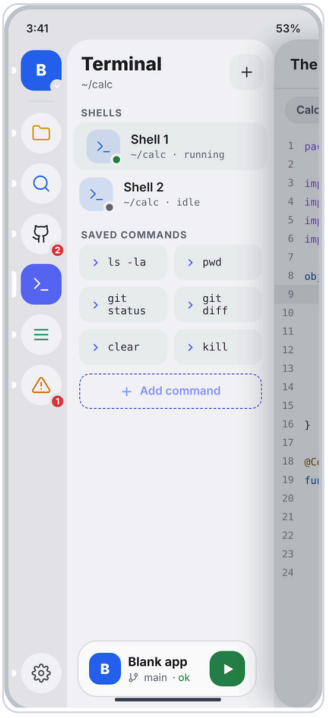
19 history



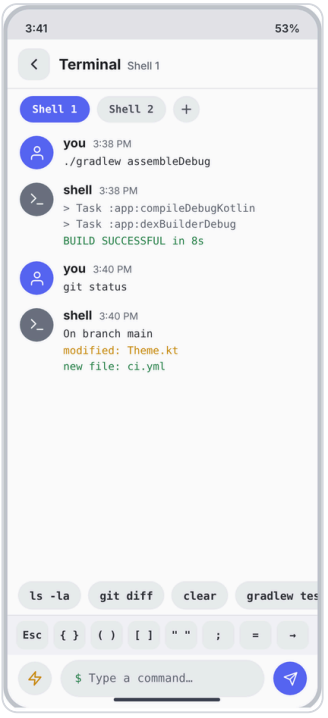
20 commit detail



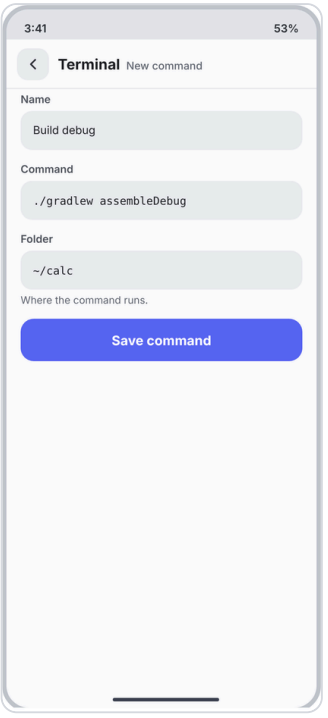
21 diff



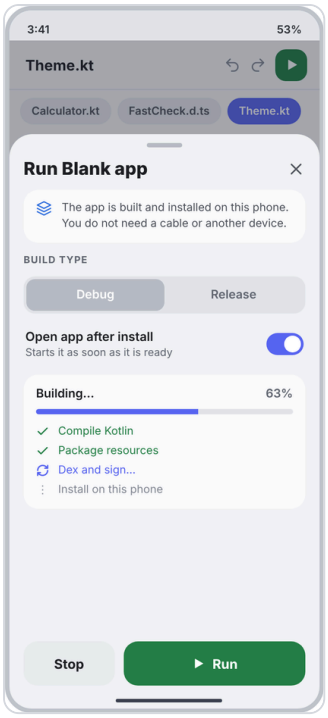
22 terminal



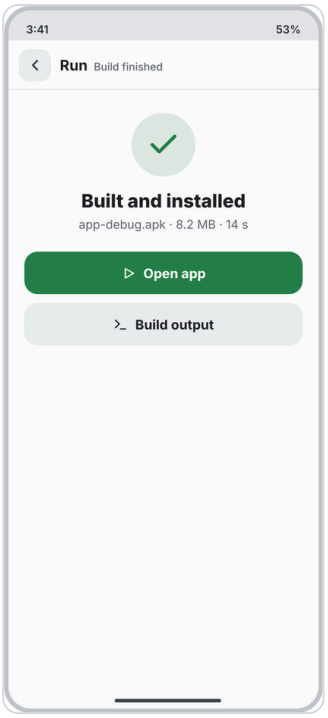
23 terminal session



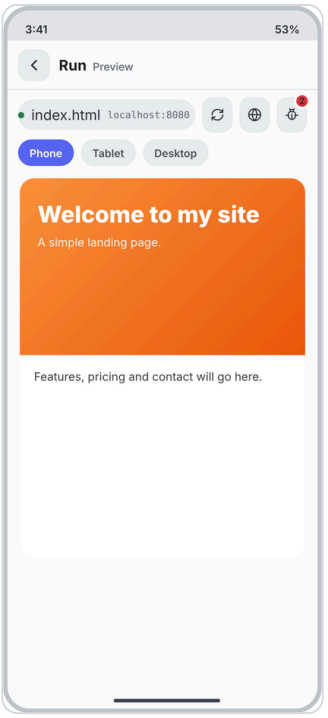
24 terminal new command



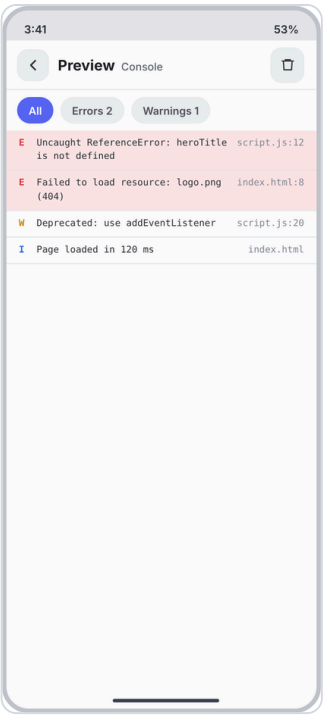
25 run build



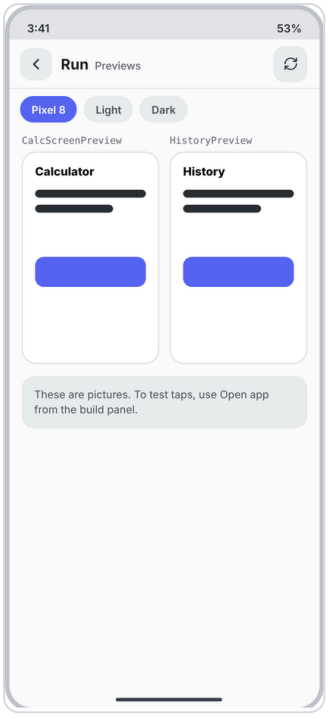
26 build finished



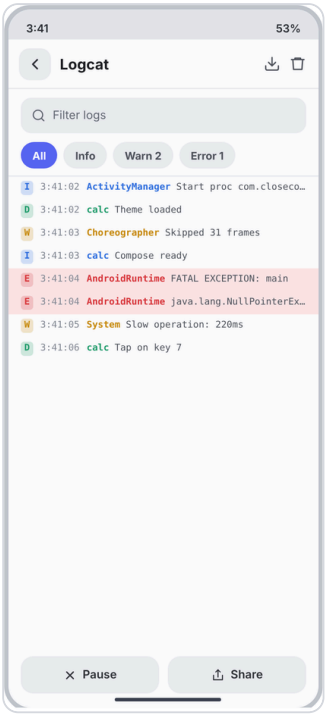
27 website preview



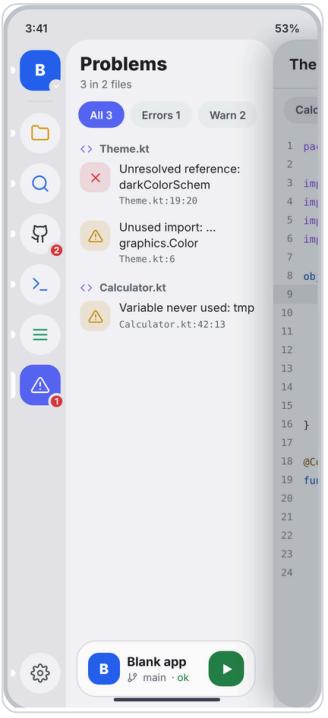
28 console



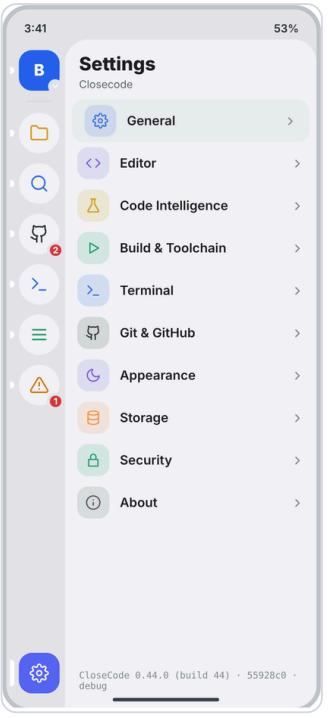
29 compose previews



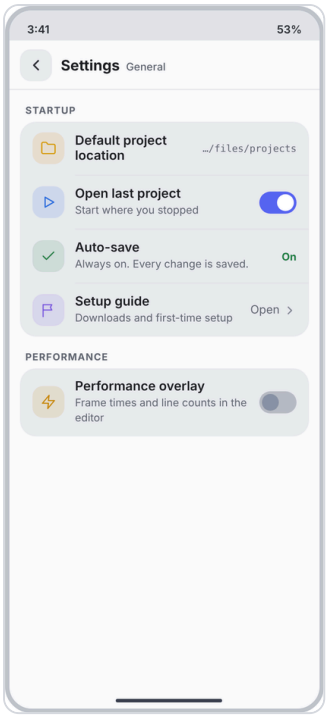
30 logcat



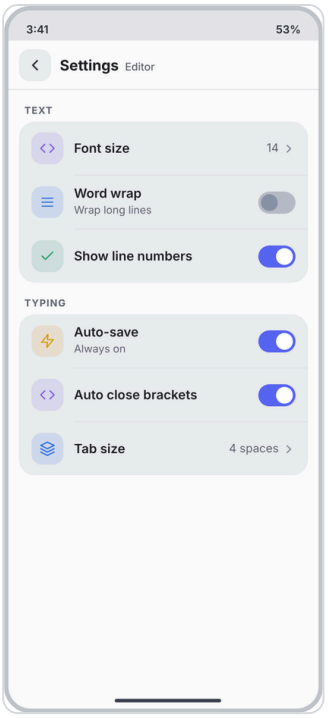
31 problems



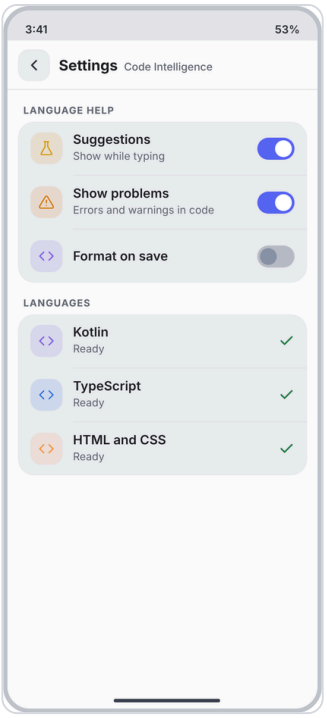
32 settings



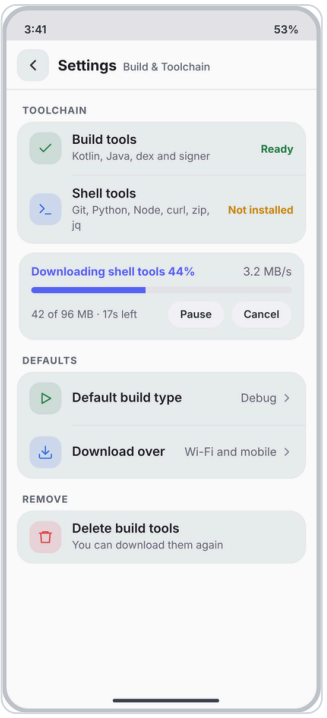
33 settings general



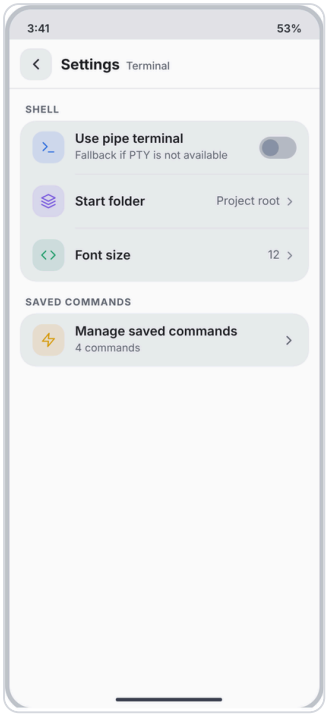
34 settings editor



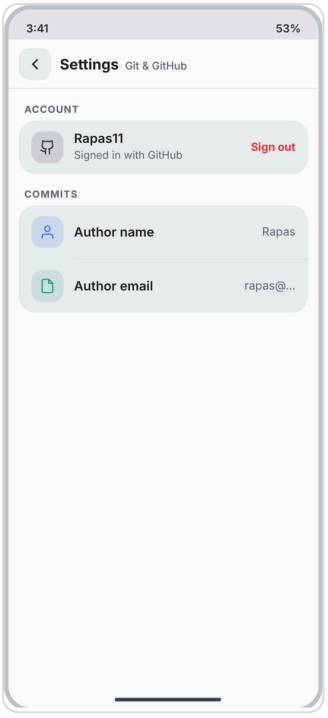
35 settings code intelligence



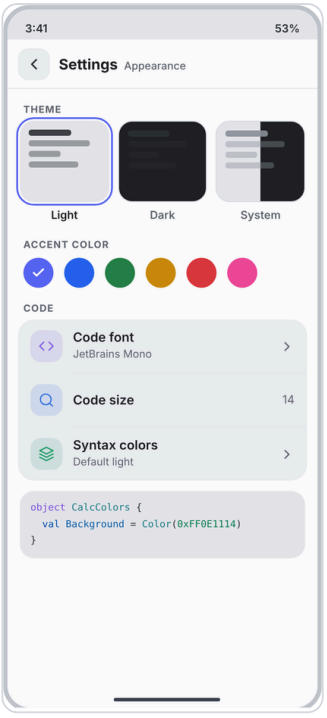
36 settings build



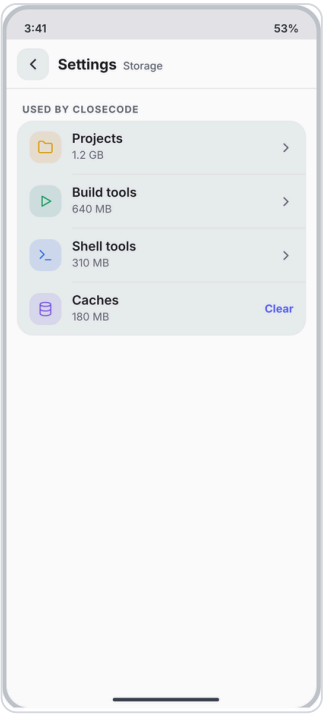
37 settings terminal



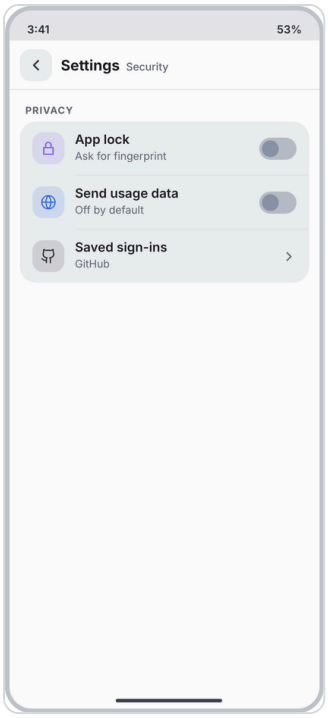
38 settings github



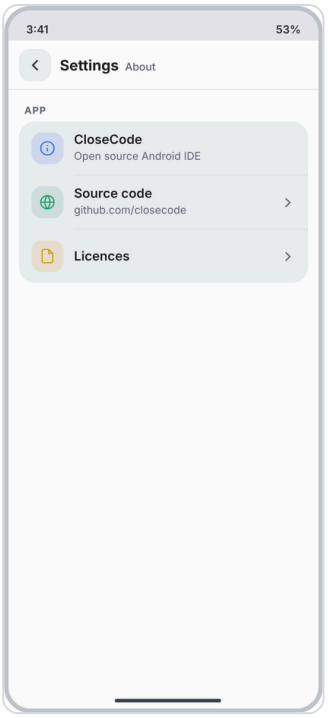
40 settings appearance



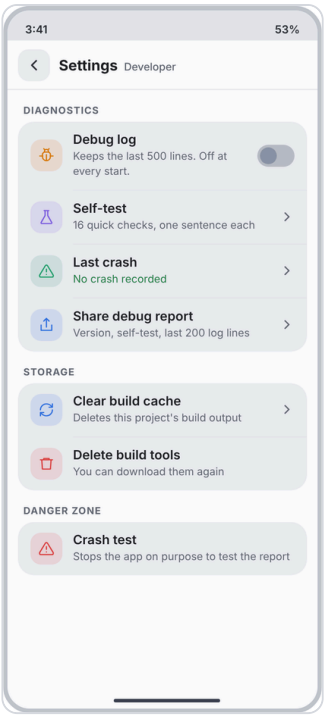
41 settings storage



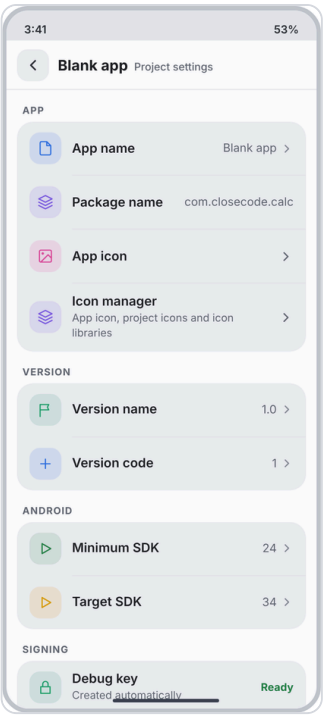
42 settings security



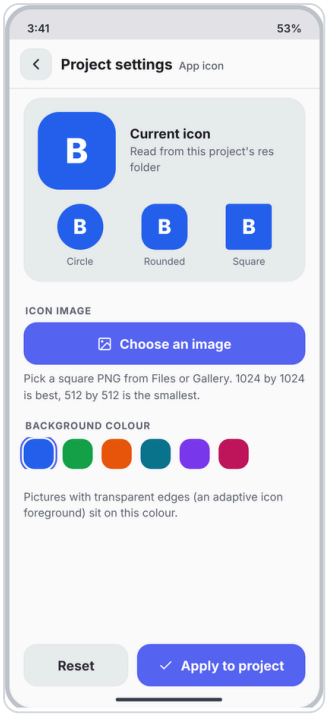
43 settings about



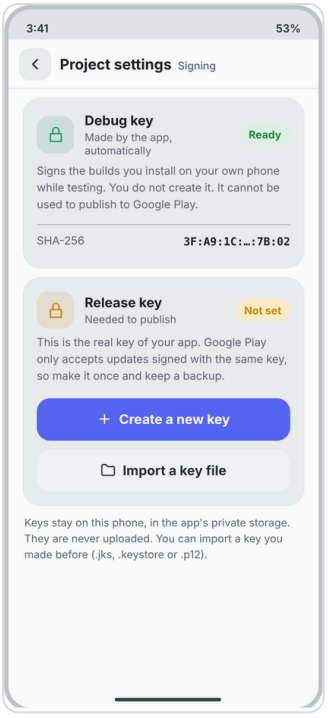
44 settings developer



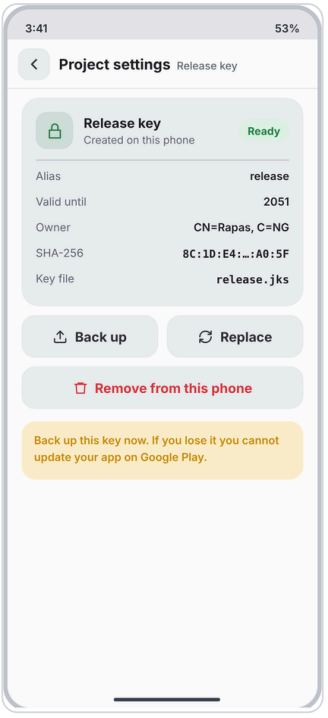
45 project settings



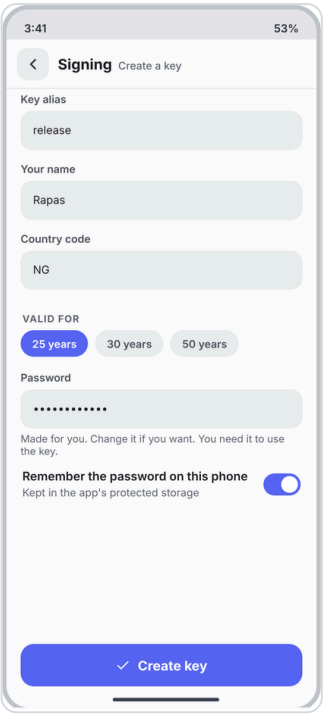
46 app icon



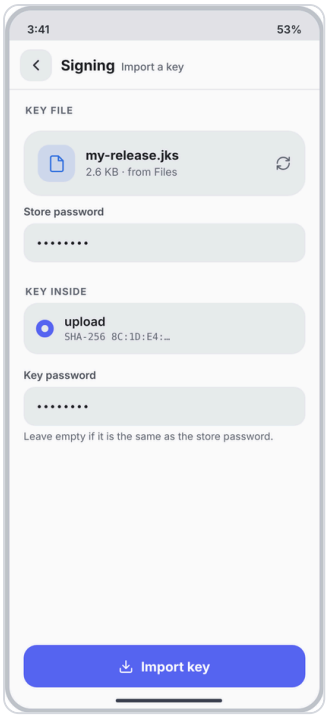
47 signing keys



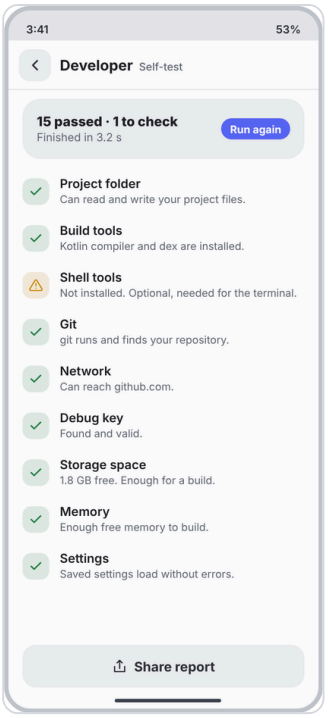
48 release key ready



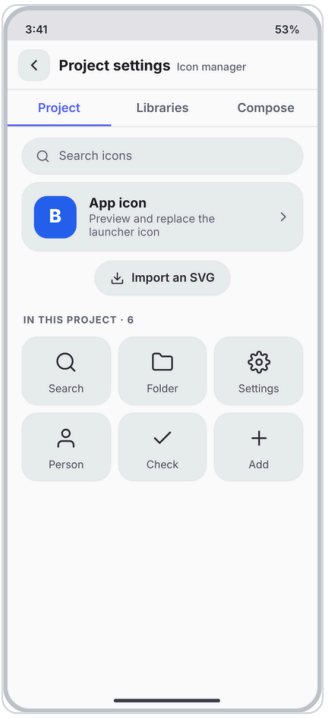
49 create key



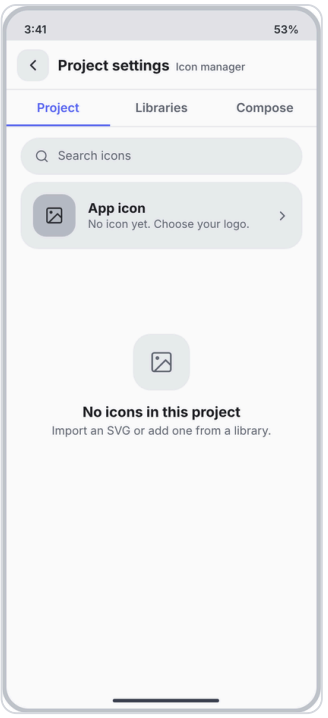
50 import key



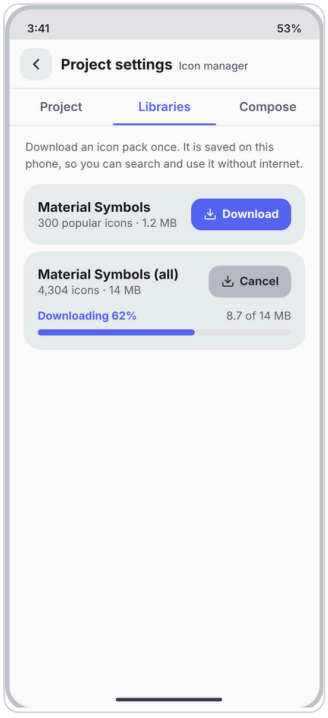
51 self test



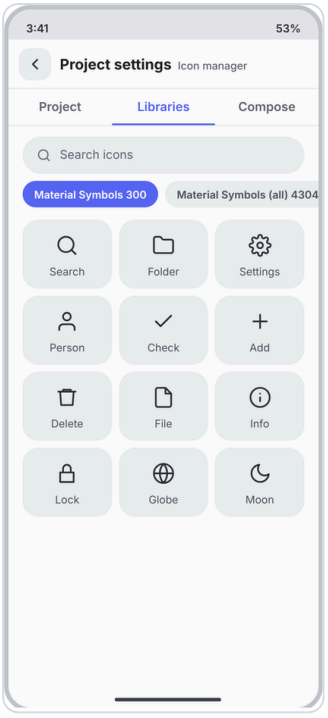
52 icon manager project



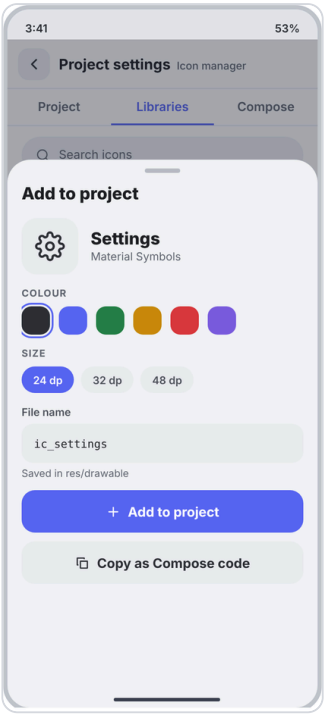
53 icon manager empty



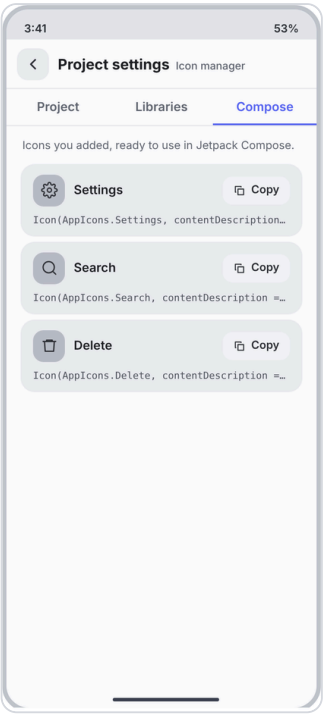
54 icon manager download



55 icon manager library

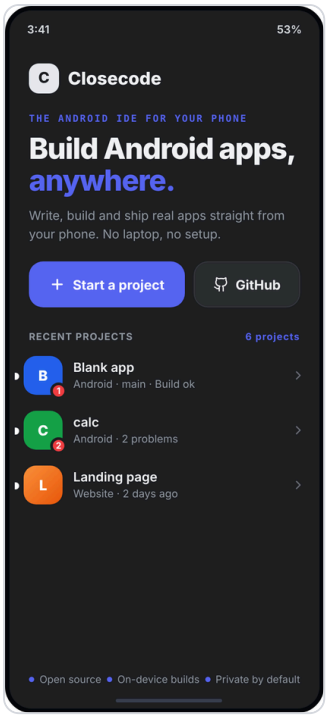


56 icon manager add

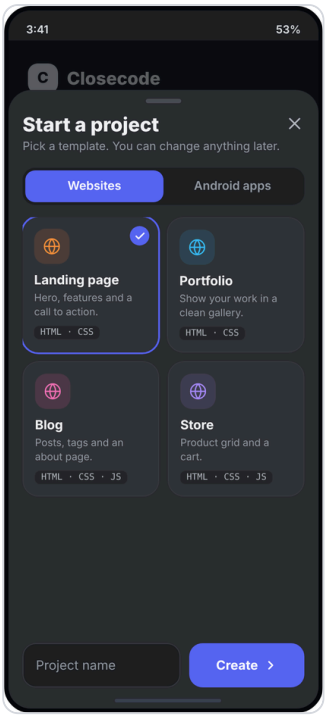


57 icon manager compose

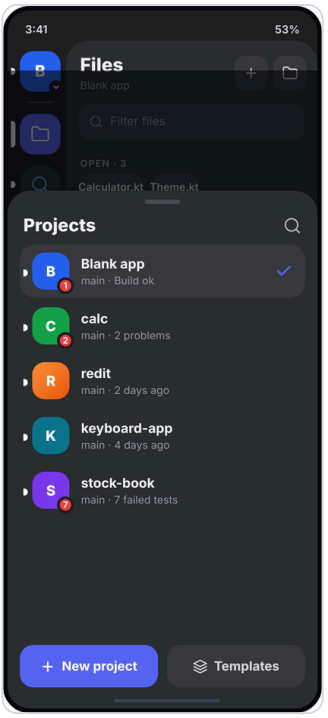
Screen pictures, dark theme (examples only, all data is sample)



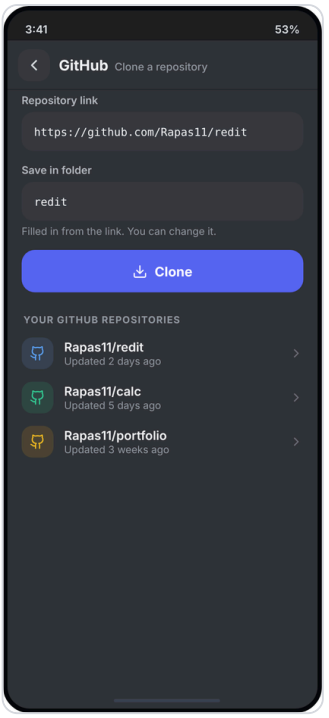
01 welcome



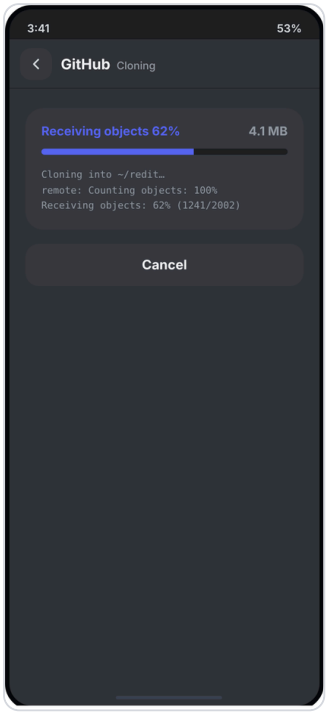
02 start project



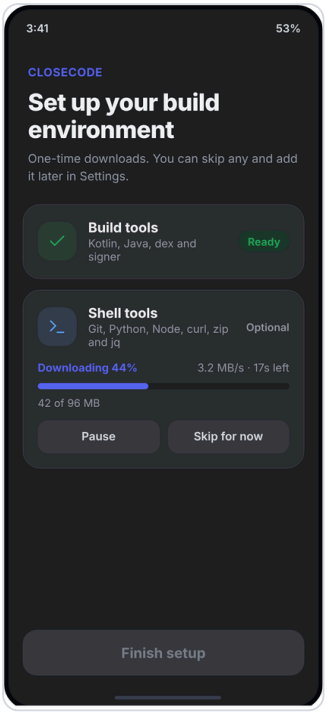
03 switch project



04 clone repository



05 cloning progress



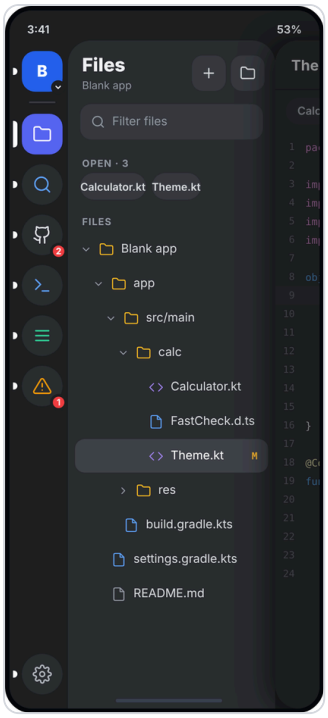
06 first time setup



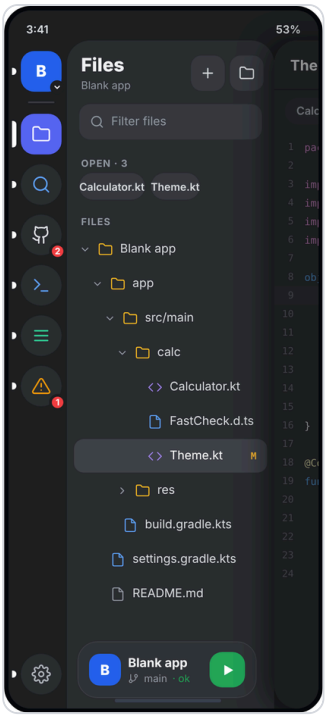
07 editor



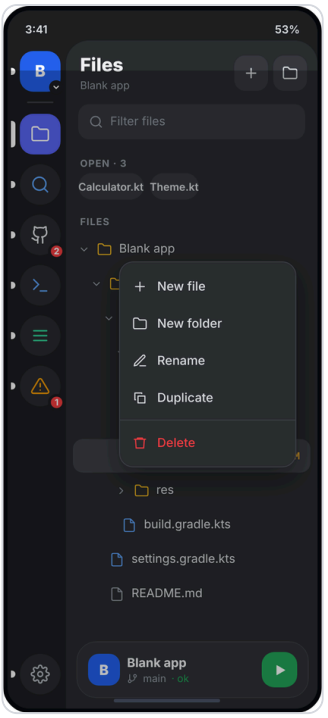
08 editor keyboard



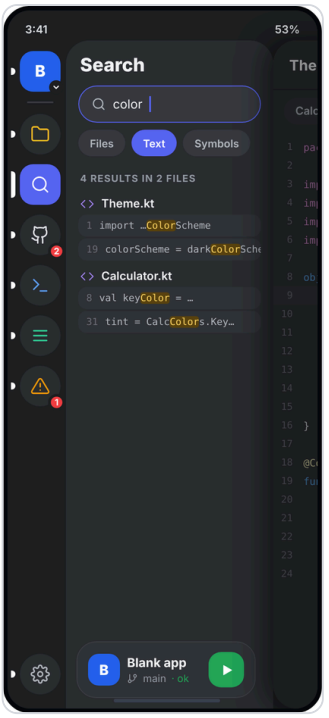
09 swipe sidebar



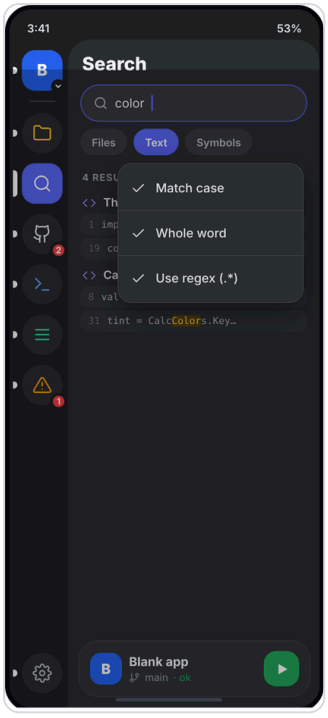
10 files



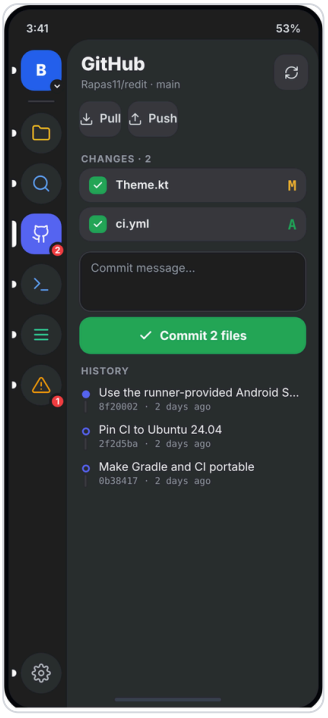
11 files menu



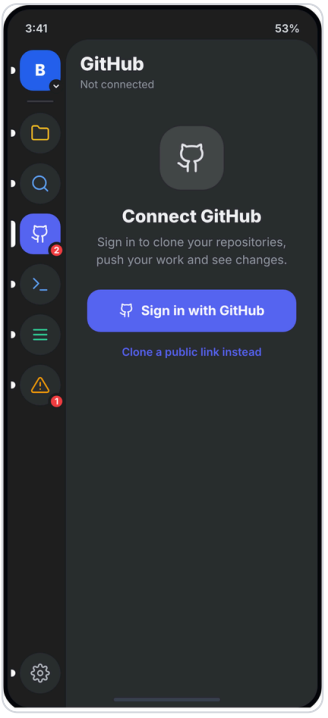
12 search



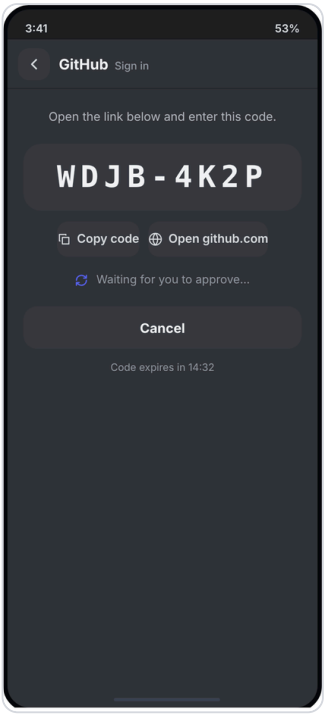
13 search options



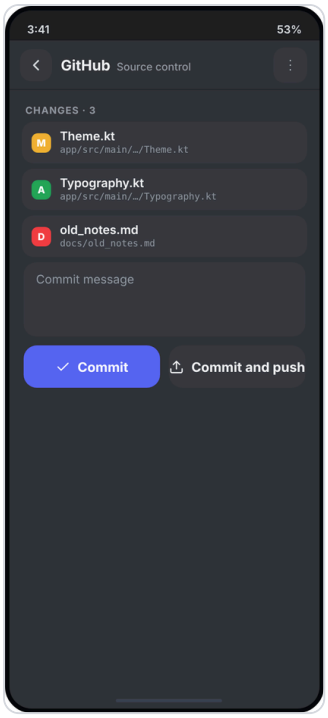
14 github



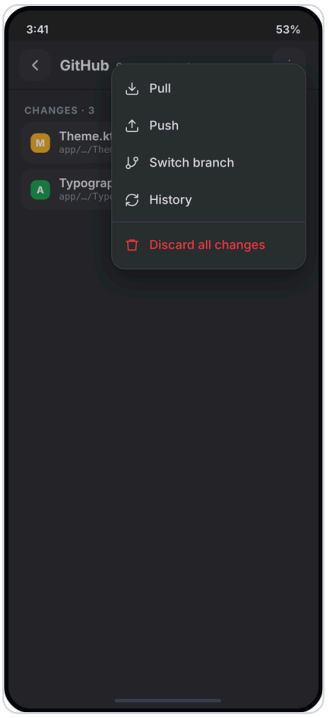
15 github not connected



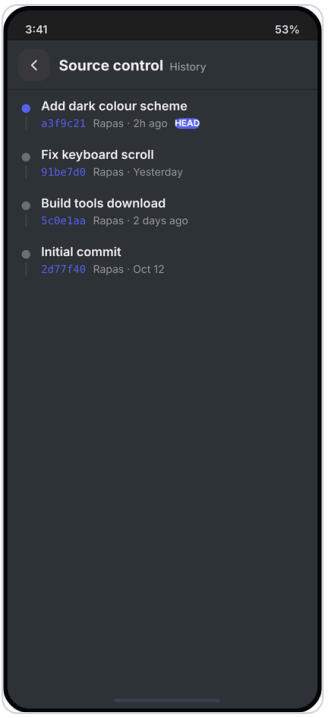
16 github sign in



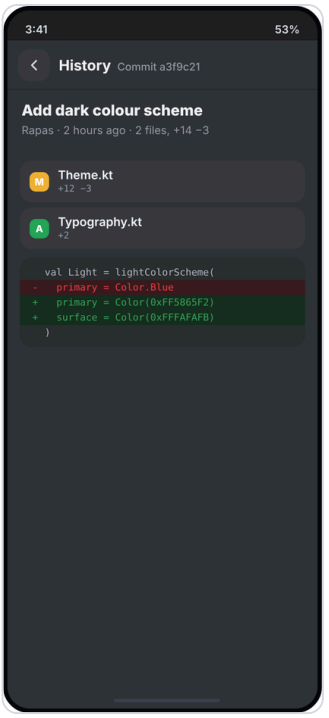
17 source control



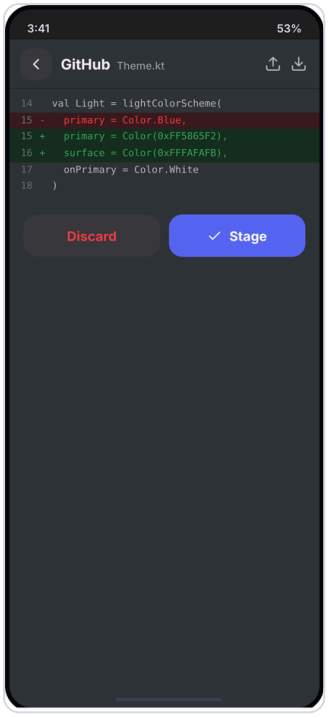
18 source control more



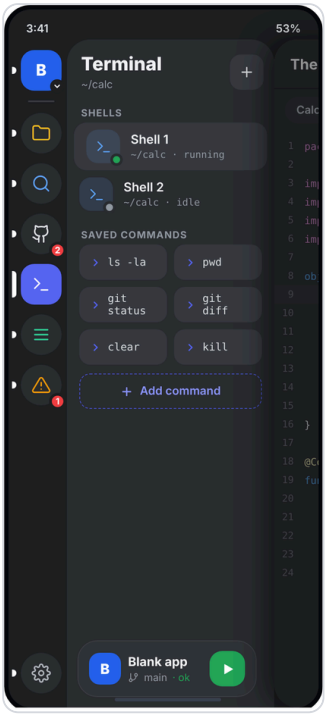
19 history



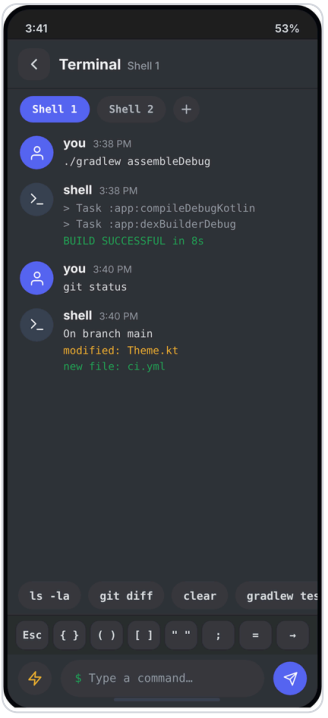
20 commit detail



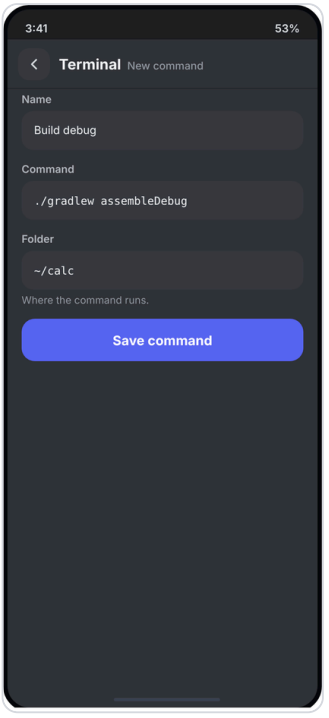
21 diff



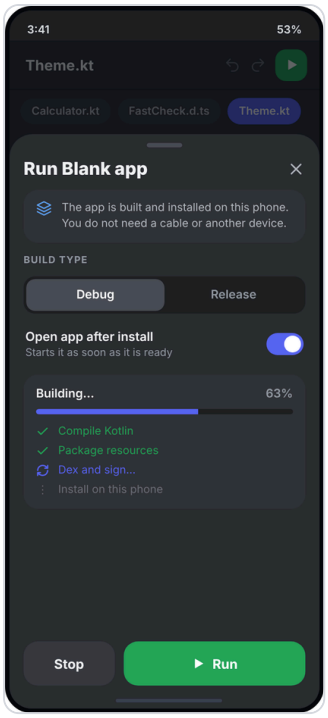
22 terminal



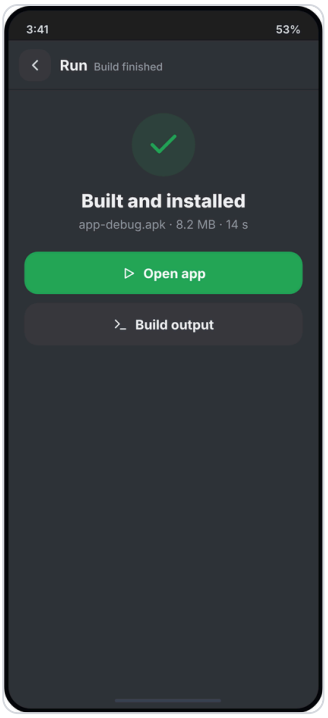
23 terminal session



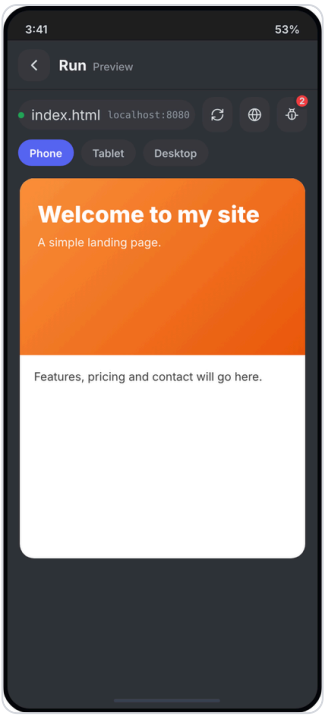
24 terminal new command



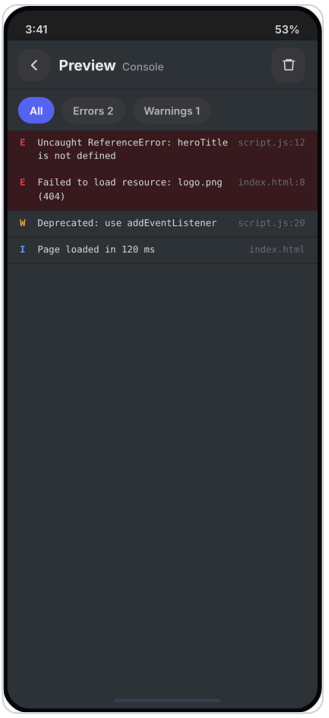
25 run build



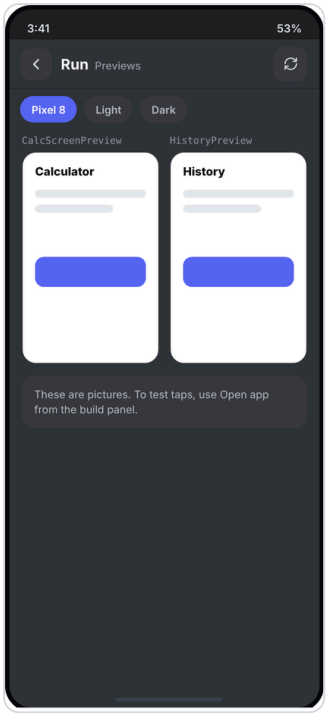
26 build finished



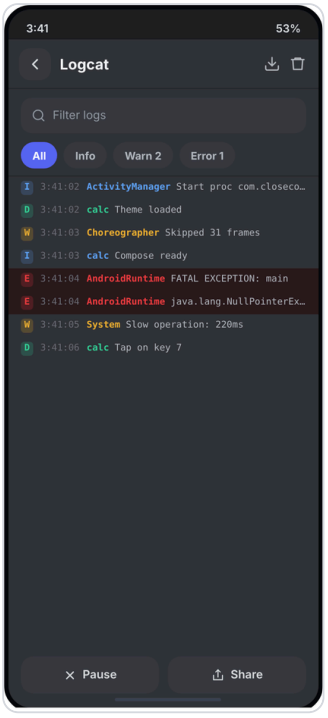
27 website preview



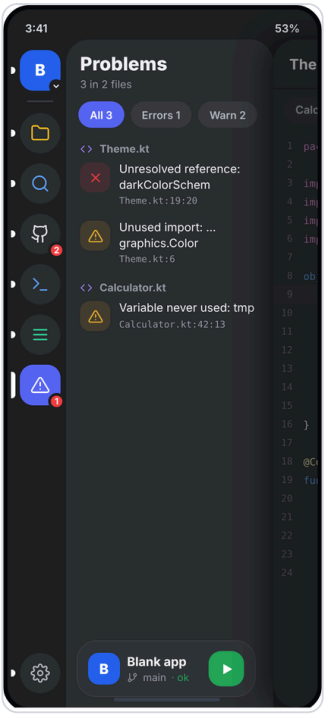
28 console



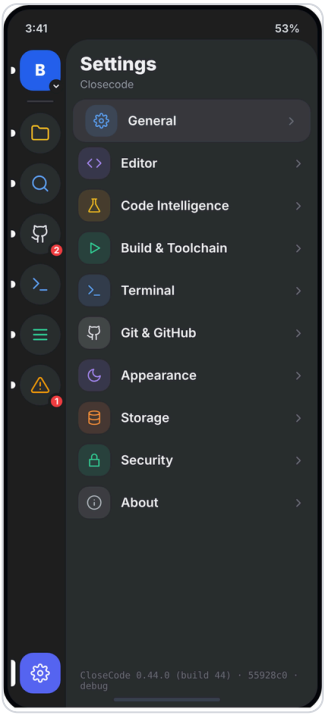
29 compose previews



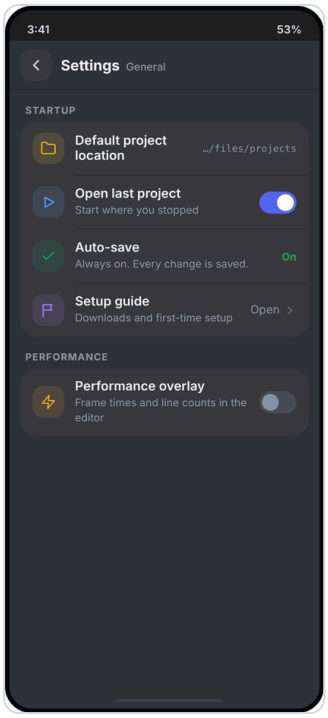
30 logcat



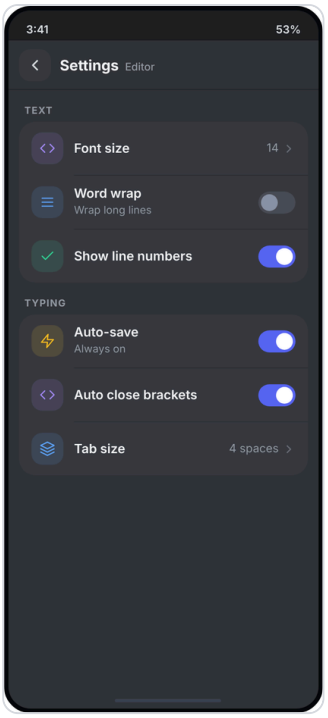
31 problems



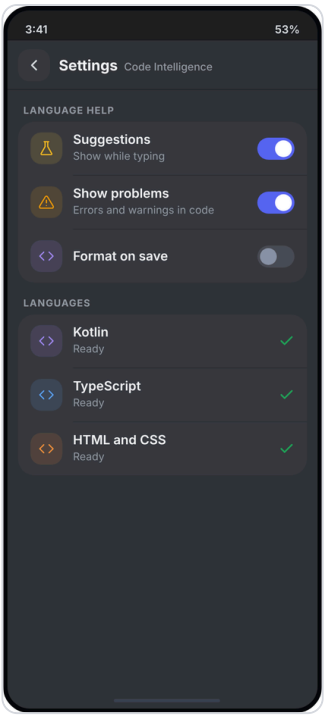
32 settings



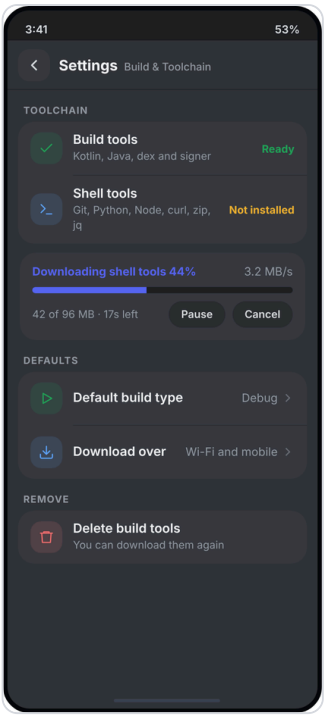
33 settings general



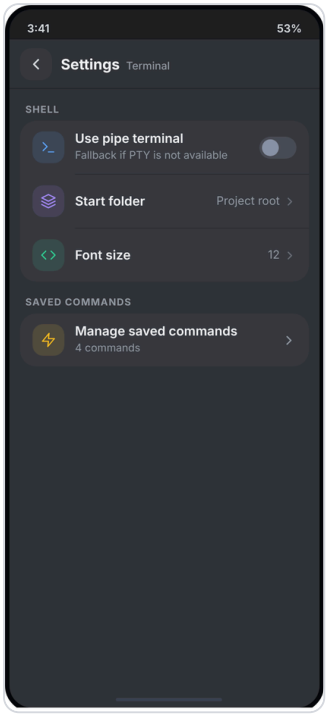
34 settings editor



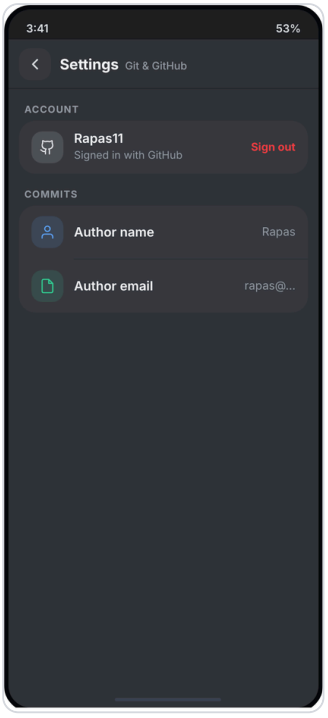
35 settings code intelligence



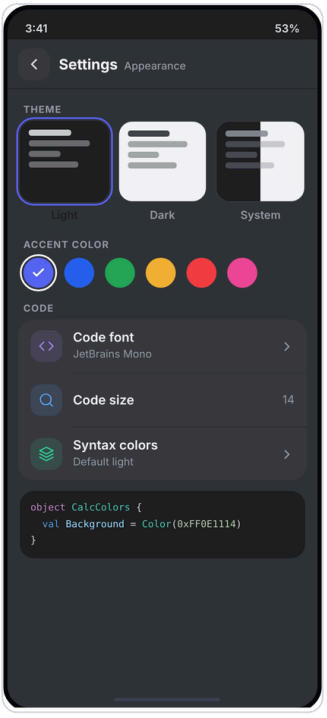
36 settings build



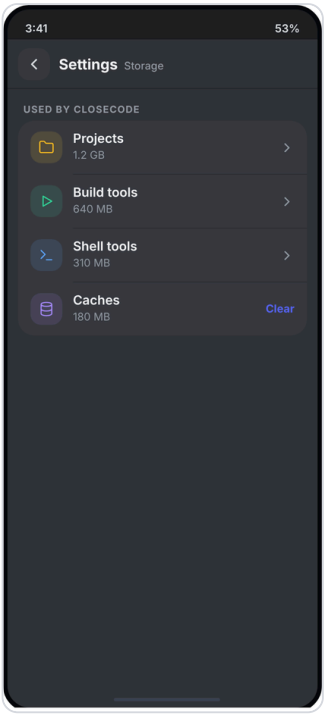
37 settings terminal



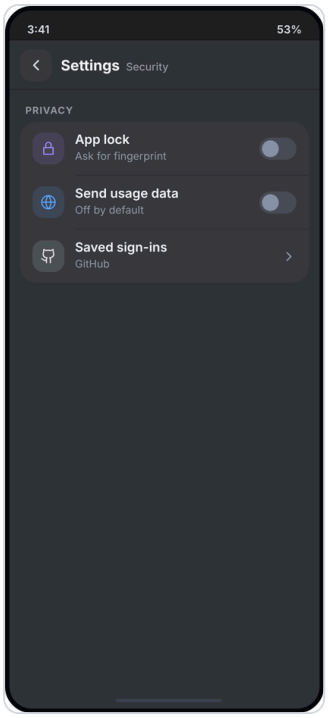
38 settings github



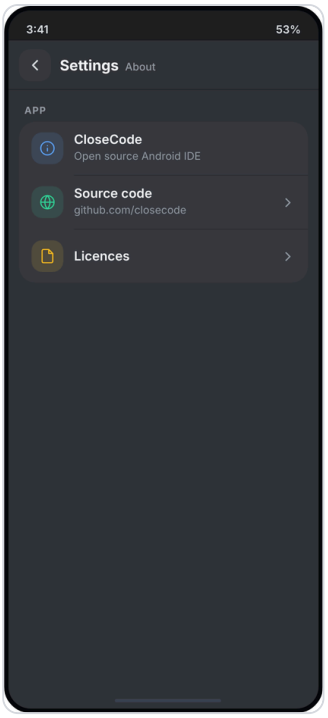
40 settings appearance



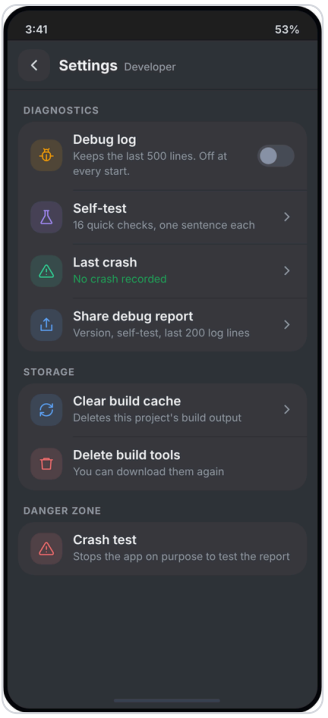
41 settings storage



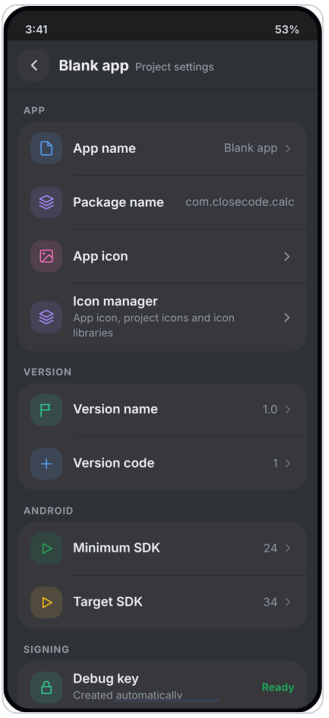
42 settings security



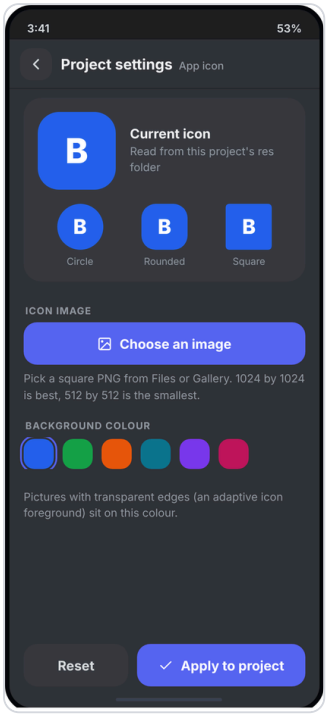
43 settings about



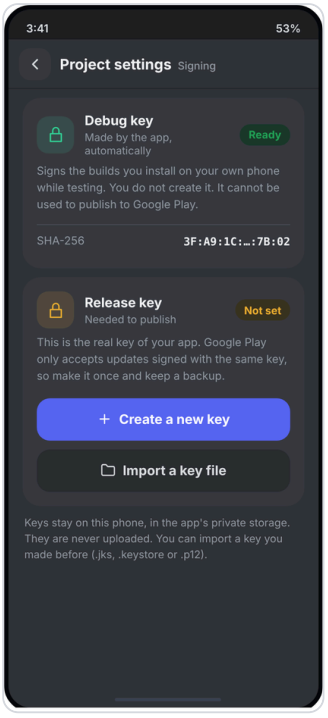
44 settings developer



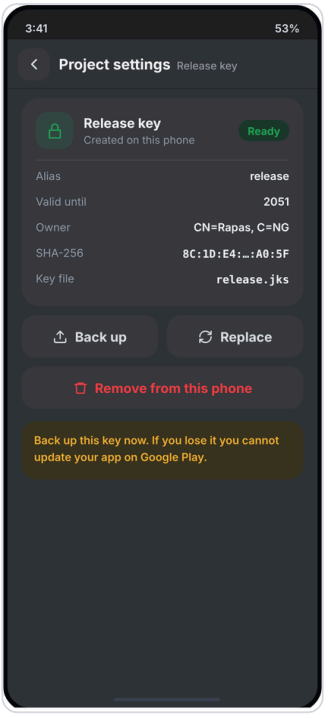
45 project settings



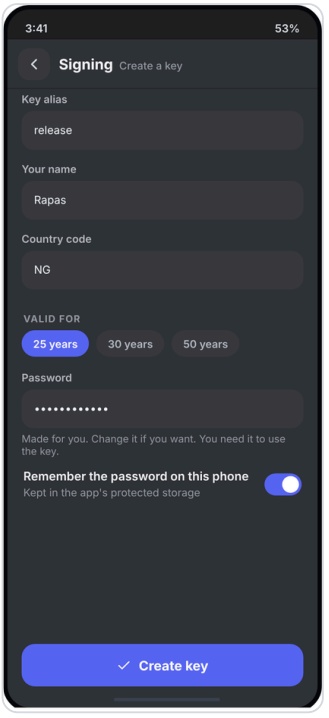
46 app icon



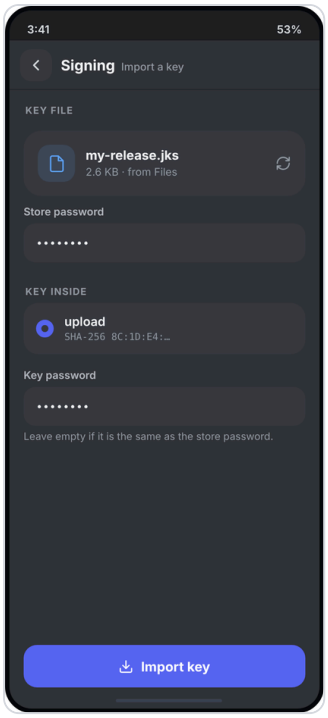
47 signing keys



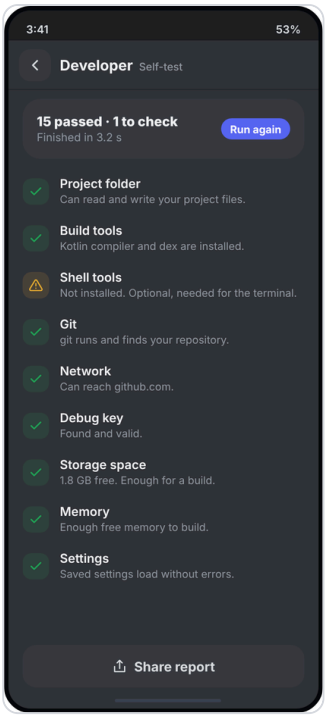
48 release key ready



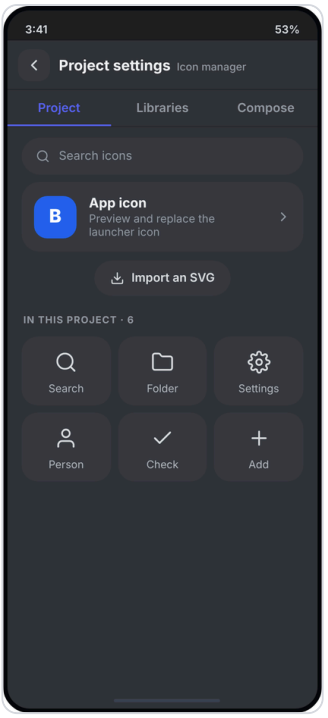
49 create key



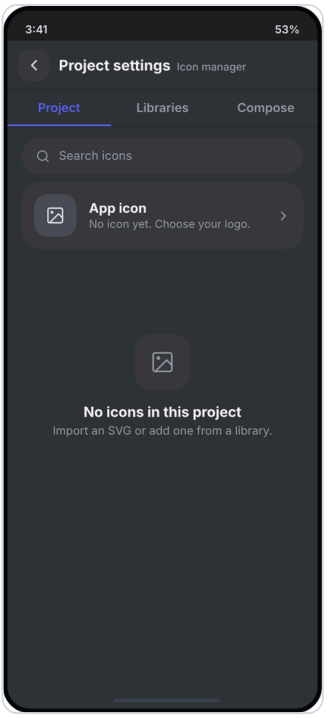
50 import key



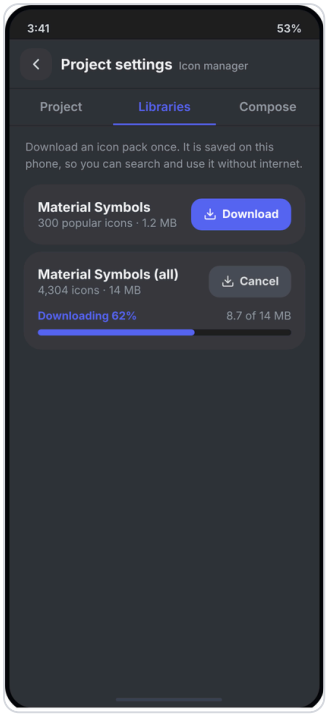
51 self test



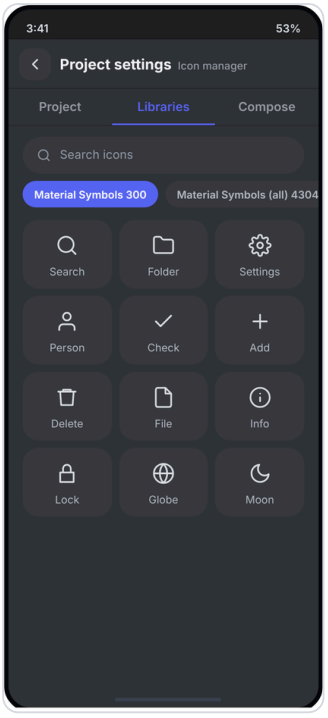
52 icon manager project



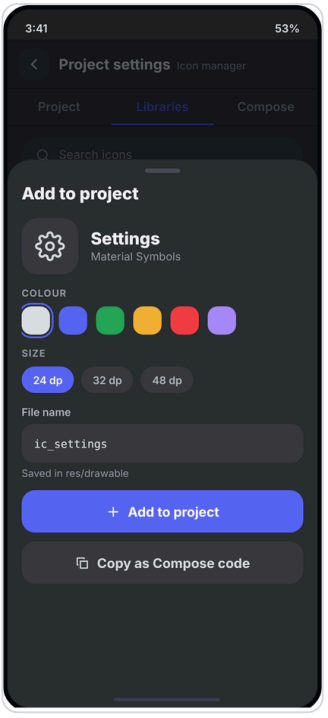
53 icon manager empty



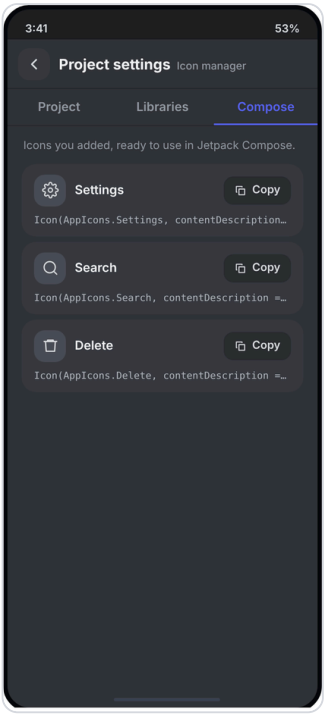
54 icon manager download



55 icon manager library



56 icon manager add



57 icon manager compose